

Thesis

Hierarchical Time Series Forecasting

*A Business - Focused Evaluation of Foundational
Reconciliation Methods in Retail Demand*

Author: Hemanth Jadiswami Prabhakaran
Matriculation No.: 7026000

Course of Studies: Business Intelligence and Data Analytics

First examiner: Prof. Dr. Joachim Schwarz
Second examiner: Prof. Dr.-Ing. Armando Walter Colombo

Submission date: September 28, 2026

Declaration of Authorship

I hereby declare that I, the undersigned, am the sole author of this document. All sources consulted for this document have been listed; all quotations from and references to these sources have been properly cited and included in chapter notes and in the list of references. No version of this document, either whole or in part, has been used to achieve an academic degree or any other examination.

I understand that any false statements made in this declaration may be punishable by law.

Emden, September 28, 2026

Place, Date

Signature

Hemanth Jadiswami Prab-
hakaran

Declaration of Consent

Provided my rights are not affected, I agree that the Master's thesis may be made freely available to members of the Emden/Leer University of Applied Sciences for study/teaching/research purposes.

Emden, September 28, 2026

Place, Date

Signature

Hemanth Jadiswami Prab-
hakaran

Use of AI Tools

During the preparation of this thesis, the AI assistant Claude (Anthropic) was used for the following supporting tasks:

- creating and refining the flowcharts and diagrams (TikZ figures) that illustrate the pipeline, hierarchy and reconciliation concepts;
- writing detailed code comments and explanatory Markdown cells in the Jupyter notebooks and the Streamlit application;
- language editing, consistency checks of figures and cross-references, and proofreading of the $\text{\LaTeX}$ sources of the thesis report, user manual and literature presentation;
- drafting the English abstract and translating it into the German Kurzfassung.

The research questions, the methodology, the implementation of the forecasting pipeline and the dashboard, the experiments, and the interpretation of the results are my own work. I have reviewed, verified and, where necessary, corrected all AI-assisted output, and I take full responsibility for the content of this thesis.

Abstract

Hierarchical forecasting requires forecasts that remain coherent across every level of aggregation, from a company-wide total down to individual products. Reconciliation methods that enforce this coherence are usually compared with uniform accuracy measures that treat every series as equally important, although in business practice a small share of products drives most of the demand. This thesis evaluates three foundational reconciliation methods – Bottom-Up, Top-Down with historical proportions, and MinTrace with structural scaling – in combination with three base forecasting models (HistoricAverage, AutoETS and Theta) on the M5 Walmart dataset. The daily sales data is aggregated to monthly frequency and organised as a six-level hierarchy (Total, State, Store, Category, Department, Item) of 30,354 series. Following the Knowledge Discovery in Databases process, all nine model/reconciler combinations are evaluated with sMAPE on a twelve-month hold-out period, both by hierarchy level and by cumulative volume segments ranked on training-period sales.

At the hierarchy-level view, Bottom-Up is the most accurate reconciler at every aggregate level, and structurally scaled MinTrace does not realise its theoretical advantage. The volume view reverses this picture: on the 29 item-store series that carry the top 5% of sales volume, the flat HistoricAverage baseline (16.61% sMAPE) and Top-Down-reconciled forecasts (16.98% for ETS) clearly outperform Bottom-Up-reconciled ETS and Theta (22.97%), whereas across all bottom-level series Bottom-Up-reconciled Theta performs best (36.32% against 40.54% for Top-Down). No single combination dominates; the choice of method should therefore be matched to the business question and the volume segment it concerns. The study is implemented as a reproducible Python pipeline based on **statsforecast** and **hierarchicalforecast**, and its results are made explorable through an interactive Streamlit dashboard.

Keywords: hierarchical forecasting, forecast reconciliation, MinTrace, retail demand, M5 competition, volume segmentation, sMAPE

Kurzfassung

Hierarchische Prognosen müssen über alle Aggregationsebenen hinweg kohärent sein, von der unternehmensweiten Gesamtsumme bis hinunter zu einzelnen Produkten. Abstimmungsverfahren (Reconciliation), die diese Kohärenz herstellen, werden üblicherweise mit einheitlichen Genauigkeitsmaßen verglichen, die jede Zeitreihe als gleich wichtig behandeln, obwohl in der betrieblichen Praxis ein kleiner Teil der Produkte den Großteil der Nachfrage verursacht. Diese Arbeit untersucht drei grundlegende Abstimmungsverfahren – Bottom-Up, Top-Down mit historischen Anteilen und MinTrace mit struktureller Gewichtung – in Kombination mit drei Basisprognosemodellen (HistoricAverage, AutoETS und Theta) anhand des M5-Datensatzes von Walmart. Die täglichen Verkaufsdaten werden auf Monatebene aggregiert und in einer sechsstufigen Hierarchie (Gesamt, Bundesstaat, Filiale, Kategorie, Abteilung, Artikel) mit 30.354 Zeitreihen organisiert. Dem Prozess der Knowledge Discovery in Databases folgend werden alle neun Kombinationen aus Modell und Abstimmungsverfahren mit dem sMAPE über einen zwölfmonatigen Testzeitraum bewertet, sowohl je Hierarchieebene als auch nach kumulativen Absatzsegmenten, die anhand der Verkäufe im Trainingszeitraum gebildet werden.

Auf Ebene der Hierarchie ist Bottom-Up auf allen aggregierten Ebenen das genaueste Verfahren, und das strukturell gewichtete MinTrace kann seinen theoretischen Vorteil nicht ausspielen. Die Betrachtung nach Absatzvolumen kehrt dieses Bild um: Bei den 29 Artikel-Filial-Zeitreihen, die die obersten 5 % des Absatzvolumens ausmachen, übertreffen der konstante HistoricAverage (16,61 % sMAPE) und Top-Down-abgestimmte Prognosen (16,98 % für ETS) die Bottom-Up-abgestimmten ETS- und Theta-Prognosen (22,97 %) deutlich, während über alle Zeitreihen der untersten Ebene Bottom-Up-abgestimmtes Theta am besten abschneidet (36,32 % gegenüber 40,54 % für Top-Down). Keine einzelne Kombination ist durchgängig überlegen; die Wahl des Verfahrens sollte sich daher an der betrieblichen Fragestellung und dem jeweiligen Absatzsegment orientieren. Die Untersuchung ist als reproduzierbare Python-Pipeline auf Basis von `statsforecast` und `hierarchicalforecast` umgesetzt, und die Ergebnisse lassen sich über ein interaktives Streamlit-Dashboard erkunden.

Schlagwörter: hierarchische Prognose, Prognoseabstimmung, MinTrace, Einzelhandelsnachfrage, M5-Wettbewerb, Volumensegmentierung, sMA-

x

PE

Contents

Contents	xi
List of Figures	xv
List of Tables	xvii
Acronyms	xxi
1. Introduction	1
1.1. Background and Motivation	1
1.2. The Business Problem	1
1.3. Research Objectives	2
1.4. Methodological Approach	3
1.5. Contribution and Significance	5
1.6. Thesis Structure	5
I. Domain Knowledge	7
2. Domain Knowledge	9
2.1. Retail Hierarchies and Structured Aggregation	9
2.2. The Pareto Principle in Sales Concentration	10
2.2.1. ABC Analysis in Inventory Practice	10
2.3. Intermittent and Sparse Demand in Item-Level Retail Data	11
2.3.1. Croston's Method and the Response to Intermittency	12
II. Domain Machine Learning	13
3. Domain Machine Learning	15
3.1. Base Forecasting Models	15
3.1.1. HistoricAverage	15
3.1.2. Exponential Smoothing (ETS)	16
3.1.3. The Theta Method	16
3.2. Hierarchical Coherence and the Summing Matrix	17
3.3. Reconciliation Theory	18
3.3.1. Bottom-Up	20

3.3.2. Top-Down	20
3.3.3. Minimum Trace (MinT)	20
III. Domain Knowledge - Tools	23
4. Domain Knowledge – Tools	25
4.1. statsforecast	25
4.2. hierarchicalforecast	25
4.3. utilsforecast	28
4.4. Supporting Stack	28
IV. Methodology	31
5. Methodology	33
5.1. Research Design	33
5.2. The KDD Process Framework	33
5.3. Dataset	35
5.3.1. Time Windows and the Live Evaluation Period	36
5.4. Data Preparation	37
5.4.1. Daily-to-Monthly Aggregation	37
5.4.2. Dropping the Partial First Calendar Month	38
5.4.3. Fixing the Item Universe on Training Data Only	39
5.4.4. Adding an Explicit Total Root	39
5.5. Hierarchy Construction	39
5.6. Experimental Design	40
5.6.1. Model Configuration: A 12-Month Seasonal Period	40
5.6.2. Ensuring a Fair Comparison: Clipping Negative Base Forecasts	41
5.7. Evaluation Framework	41
5.7.1. sMAPE	42
5.7.2. Volume-Percentile Evaluation Framework	42
5.8. Limitations of the Design	43
5.9. Fairness/Bias Note	44
V. Application Development	45
6. Application Development	47
6.1. Model Fitting	47
6.2. Clipping Negative Base Forecasts	48
6.3. Reconciliation Implementation	48
6.4. Combining Results and Attaching Ground Truth	49

6.5. Developer Documentation: Pipeline Flow	50
6.6. Naming, Parameter, and Error-Handling Conventions	50
6.7. Testing & Validation	51
VI. Development to Deployment	53
7. Development to Deployment	55
7.1. Code/config.py as the Single Source of Truth	55
7.2. Reproducibility	56
7.3. Repo Layout and Separation of Concerns	57
VII.Application Deployment	59
8. Application Deployment	61
8.1. Data Contracts	61
8.2. Cascading Dropdown Interactivity	62
8.3. Chart Design	64
8.4. Theme Sensing	64
8.5. Deployment on Streamlit Community Cloud	65
8.6. Deployment Flow	65
VIIIEvaluation - Validation - Conclusion	67
9. Evaluation – Validation – Conclusion	69
9.1. Accuracy by Hierarchy Level	69
9.2. The Bottom-Up Surprise	70
9.3. Accuracy by Volume Segment	72
9.4. The High-Volume Baseline Surprise	73
9.5. The Top-Down Advantage on High-Volume Series	73
9.6. The Long-Tail Reversal	75
9.7. Testing & Validation Results	75
9.8. Answering SQ1–SQ3 and the Primary Research Question	76
9.9. Practical Recommendations for Business Forecasting Practice	77
9.10. Conclusion and Future Work	78
9.11. Monitoring & Operationalisation Outlook	79

IX. Appendix	81
10. Appendix	83
10.1. Full sMAPE Tables	83
10.1.1. By Hierarchy Level	83
10.1.2. By Volume Segment	84
10.2. Full Code Listings	85
10.2.1. EDA Notebook	86
10.2.2. Pipeline Notebook	99
10.2.3. <code>config.py</code>	129
10.2.4. Streamlit Dashboard	131
10.2.5. Pipeline Requirements	137
10.2.6. Deployment Requirements	140
Bibliography	141

List of Figures

1.1. The six-level hierarchy used in this thesis, with series counts at each level.	4
3.1. Information flow between hierarchy levels under each reconciliation strategy.	19
4.1. Data objects passed between the three Nixtla libraries used in this thesis’s forecasting pipeline.	27
5.1. The five KDD process stages mapped to this thesis’s own pipeline steps.	35
5.2. Total training-period sales by product category and by state. Source: 01_eda.ipynb	36
5.3. The training, test, and live time windows (schematic, not to scale). The live window is trained on training and test data combined and extends three months past where real sales data actually ends.	37
5.4. Distribution across items of the percentage of days with zero sales, at the daily item-store level. Source: 01_eda.ipynb	38
5.5. The 3×3 base-model / reconciler grid; every cell is further evaluated across 6 hierarchy levels and 6 volume-percentile bands.	40
5.6. Total monthly sales across all items and stores, 2011–2016, showing a recurring annual pattern.	41
6.1. The full pipeline, from loading the raw M5 data to exporting the dashboard’s Parquet files.	50
7.1. Code/config.py as the single configuration module feeding both notebooks and the Streamlit app.	55
8.1. The sidebar’s cascading dropdown controls, partway through drilling down the hierarchy.	63
8.2. The forecast chart in light mode (left) and dark mode (right), showing the shaded train/test/live regions.	64
8.3. The deployment flow from a local pipeline run to the served dashboard.	66

- 9.1. Pareto curve of training-period sales volume: cumulative share of volume against cumulative share of bottom-level series, ranked from the largest seller. Source: `01_eda.ipynb`. 72
- 9.2. Reconciled sMAPE by volume segment for ETS and Theta, Bottom-Up (solid) vs. Top-Down (dashed). 74

List of Tables

6.1. Summary of the pipeline’s built-in validation checks. . .	52
9.1. sMAPE (%) by hierarchy level, base forecasts, test period.	70
9.2. Reconciled sMAPE (%) by hierarchy level, ETS and Theta, test period.	71
9.3. Volume-bucket sizes, ranked on training-period volume. .	72
9.4. sMAPE (%) by volume segment, base forecasts, test period.	73
9.5. Reconciled sMAPE (%) by volume segment: Bottom-Up vs. Top-Down.	74
9.6. Testing and validation results, against the protocol defined in Section 6.7.	76
10.1. sMAPE (%) by hierarchy level – HistoricAverage	83
10.2. sMAPE (%) by hierarchy level – AutoETS	84
10.3. sMAPE (%) by hierarchy level – Theta	84
10.4. sMAPE (%) by volume segment – HistoricAverage	84
10.5. sMAPE (%) by volume segment – AutoETS	85
10.6. sMAPE (%) by volume segment – Theta	85

Listings

10.1. Full code listing: <code>Code/config.py</code>	129
10.2. Full code listing: <code>Code/app/streamlit_app.py</code>	131
10.3. Full listing: <code>Code/requirements-pipeline-raw.txt</code> .	137
10.4. Full listing: <code>Code/app/requirements.txt</code>	140

Acronyms

BU Bottom-Up

DRY Don't Repeat Yourself

EDA Exploratory Data Analysis

ETS Error, Trend, Seasonality (exponential smoothing)

KDD Knowledge Discovery in Databases

LRU Line Replaceable Unit

M5 Fifth Makridakis Forecasting Competition

MinT Minimum Trace (reconciliation)

MRO Maintenance, Repair and Overhaul

RMSSE Root Mean Squared Scaled Error

sMAPE Symmetric Mean Absolute Percentage Error

TD Top-Down

WLS Weighted Least Squares

WRMSSE Weighted Root Mean Squared Scaled Error

1. Introduction

1.1. Background and Motivation

In modern business environments, forecasting problems rarely exist in isolation. Organisations routinely manage demand across structured hierarchies—where totals must equal the sum of their parts at every level of aggregation. In retail operations, total store sales must equal the sum across product categories, departments, and individual items. In industrial operations, total component demand must reconcile across technology classifications, unit types, and individual part numbers. These structures, known as hierarchical time series, present both a methodological challenge and a practical necessity for organisations that rely on coherent, actionable forecasts across multiple planning levels [HA18].

This research is motivated by real-world exposure to hierarchical forecasting challenges encountered during a data science internship in the aviation maintenance, repair and overhaul (MRO) sector in Hamburg. In that operational context, demand forecasts were required across a structured hierarchy spanning technology classifications, line replaceable units (LRUs), component disposition sets, and individual part numbers — a structure directly analogous to the hierarchical retail forecasting problem examined in this thesis. A key practical observation from that experience was that forecast accuracy requirements are not uniform across the hierarchy: accuracy for high-demand, business-critical components matters disproportionately more than accuracy for low-demand, peripheral items. This asymmetry between business importance and equal statistical treatment motivates the central investigation of this thesis.

1.2. The Business Problem

Traditional hierarchical forecasting adopts one of three broad strategies: bottom-up forecasting, which generates predictions at the most disaggregate level and aggregates upward; top-down forecasting, which predicts the aggregate total and distributes downward proportionally [HA18]; or optimal reconciliation, which combines base forecasts across all hierarchy levels to ensure coherence [WAH19]. Each approach addresses the coherence problem differently, yet all share a common characteristic — they are typically evaluated against uniform accuracy measures that treat all

series equally.

In practice, organisational forecasting priorities are rarely uniform. A small subset of high-volume series frequently accounts for the majority of business value, a pattern consistent with the Pareto principle [Par96]. In retail environments, a small proportion of products may generate the majority of demand, while a large number of low-volume items contribute comparatively little. Whether the choice of reconciliation method or base forecasting model produces different outcomes across these volume segments is an open empirical question with direct implications for how practitioners approach hierarchical forecasting.

Furthermore, series within a hierarchy often exhibit distinct statistical characteristics. High-volume, aggregate-level series tend to demonstrate stable, continuous demand patterns, while low-volume, bottom-level series frequently display intermittent demand with sparse observations [SBC05]. Whether a single forecasting approach adequately serves both ends of this spectrum, or whether the choice of base model interacts with reconciliation method performance across hierarchy levels, is a further empirical question this thesis seeks to investigate.

1.3. Research Objectives

This thesis investigates how foundational hierarchical time series forecasting methods perform across business-relevant volume segments in a large-scale retail demand setting. Rather than seeking to identify a universally superior method, the study examines how different method combinations behave across hierarchy levels and volume percentiles, with the aim of informing practical forecasting decisions in business contexts.

The primary research question is:

How do foundational hierarchical forecasting methods perform across different business volume segments in retail demand forecasting, and what practical insights can be drawn for business forecasting decisions?

This is addressed through three sub-questions:

1. **SQ1:** Do Bottom-Up, Top-Down and MinT reconciliation methods differ in forecast accuracy patterns across hierarchy levels?
2. **SQ2:** Does the choice of base forecasting model — HistoricAverage, ETS [Hyn+08], or Theta [AN00] — affect reconciliation performance across the hierarchy?
3. **SQ3:** As the accuracy evaluation is widened from the highest-volume series outward to the full population, how does the gap between

methods change, and what are the implications for business forecasting practice?

Evaluation employs sMAPE as the primary metric, selected for its demonstrated rank stability on hierarchical retail data and its interpretability relative to scale-dependent alternatives [Hew+21]. Volume-based business importance is addressed through a percentile evaluation framework rather than price-weighting, reflecting a deliberate focus on demand planning and logistics accuracy over revenue-weighted financial performance. This framework evaluates cumulative top- $p\%$ volume bands (e.g. the top 5%, top 10%, ..., up to the full population); since each band is nested inside the next, SQ3 is framed around how the accuracy gap changes as the band widens, rather than which single segment a difference "belongs to."

1.4. Methodological Approach

To address the research questions, this thesis implements and systematically compares hierarchical forecasting method combinations across three dimensions: base forecasting model, reconciliation method, and business volume segment.

Base Forecasting Models:

- **HistoricAverage:** A naive baseline producing the historical mean as the forecast, providing a reference point for all comparisons
- **ETS:** A classical exponential smoothing method capturing level, trend, and seasonality [Hyn+08]
- **Theta:** A decomposition-based method with strong empirical performance on real-world demand data [AN00]

Reconciliation Methods:

- **Bottom-Up (BU):** Forecasts are generated independently at the most granular level (item-store) and summed upwards through the hierarchy [OWE68; DWD76]. This preserves all low-level information but can accumulate high noise from sparse, intermittent series.
- **Top-Down (TD):** Forecasts are generated strictly at the highest level of aggregation (Total) and distributed downward using historical proportion averages [GS90]. While structurally stable, this method can introduce systematic bias at lower levels by ignoring localized trends [WAH19].

- **Minimum Trace (MinT)**: A mathematically optimal framework that adjusts forecasts across all levels of the hierarchy simultaneously to minimize forecast error variance [WAH19]. To handle the scale of the retail dataset (30,354 series), this study utilizes a computationally efficient scaling variant of the trace minimization framework [WAH19].

The empirical evaluation utilises the M5 forecasting competition dataset [MSA22], comprising daily retail sales data from Walmart across three US states, ten stores, three product categories, and seven departments. Of the 3,049 products in the raw dataset, 3,024 have recorded sales prior to the train/test split and are retained for modelling, yielding 30,240 bottom-level item-store series in total. Daily data is aggregated to monthly frequency to align with operational planning horizons and to reduce noise in sparse bottom-level series.

The data is structured as a six-level hierarchy, with an explicit total level introduced above the state level to ensure a single, well-defined root for top-down reconciliation, as shown in Figure 1.1.

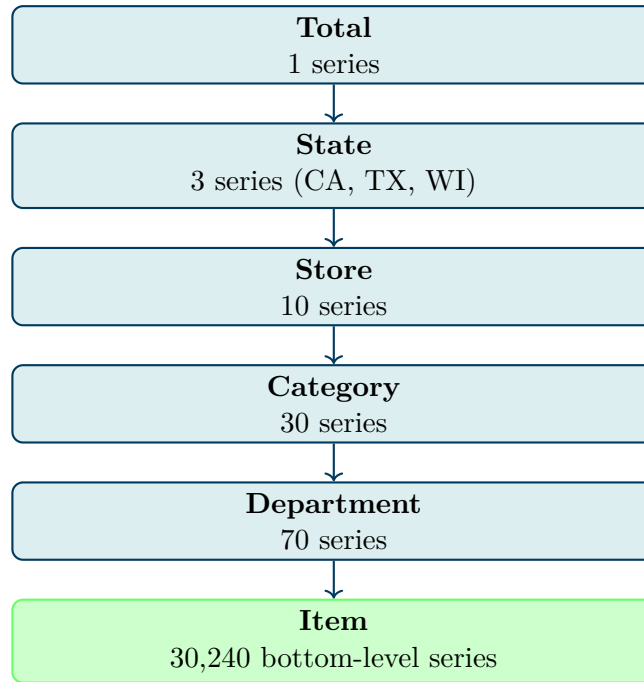


Figure 1.1.: The six-level hierarchy used in this thesis, with series counts at each level.

This dataset exhibits the volume distribution and demand sparsity patterns characteristic of real-world business hierarchies, making it well-suited to investigating the research questions. Implementation leverages Python-based forecasting libraries including `statsforecast` [Gar+22]

and `hierarchicalforecast` [Oli+22], with results explored through an interactive Streamlit dashboard built with Plotly visualisations.

1.5. Contribution and Significance

This research contributes to both academic understanding and business practice in hierarchical time series forecasting:

Academic Contribution: The thesis provides empirical evidence on how foundational reconciliation methods behave across hierarchy levels and volume segments on a large-scale, publicly available dataset. By systematically varying both base model and reconciliation method, it characterises the interaction between these two dimensions of the hierarchical forecasting pipeline.

Practical Contribution: The percentile-based evaluation framework allows forecast accuracy to be assessed where it matters most for business decisions — across defined volume segments rather than as a single aggregate statistic. This perspective is directly motivated by operational forecasting experience in the MRO aviation sector in Hamburg, and the findings provide empirical guidance for practitioners facing similar hierarchical forecasting challenges.

Methodological Contribution: The study establishes a reusable evaluation methodology that combines standard hierarchical forecasting benchmarks with business-oriented volume segmentation, offering a transparent and stable alternative to price-weighted evaluation.

1.6. Thesis Structure

The remainder of this thesis is organised as follows:

Chapter 2 — Domain Knowledge: Provides the business and retail context of the study: how retail sales are organised in hierarchies, the Pareto concentration of sales volume and its use in ABC analysis, and the intermittent nature of item-level demand.

Chapter 3 — Domain Machine Learning: Reviews the statistical forecasting methods applied in this study, including `HistoricAverage`, `ETS`, `Theta`, and hierarchical reconciliation techniques.

Chapter 4 — Domain Knowledge – Tools: Describes the Python forecasting ecosystem used in this study, including `statsforecast`, `hierarchicalforecast`, and supporting libraries.

Chapter 5 — Methodology: Details the experimental design following the Knowledge Discovery in Databases (KDD) process framework, covering dataset preparation (daily-to-monthly aggregation, dropping the partial first calendar month, fixing the item universe on training data

only, and adding the Total root), hierarchy construction, and the evaluation framework, including the sMAPE metric and the volume-percentile evaluation framework used throughout Chapter 9.

Chapter 6 — Application Development: Documents the implementation and fitting of the three base forecasting models and their reconciliation, including the safeguard of clipping negative base forecasts to zero before reconciliation, which keeps the comparison between reconciliation methods fair.

Chapter 7 — Development to Deployment: Discusses code organisation and reproducibility considerations, including how `Code/config.py` serves as the single source of truth for hierarchy levels, time windows, and reconciler naming, so that the pipeline and the Streamlit dashboard never silently drift apart.

Chapter 8 — Application Deployment: Presents the Streamlit dashboard built on the pipeline's four exported data contracts (hierarchy lookup, actuals, base predictions, reconciled predictions), including its cascading State → Store → Category → Department → Item dropdowns and automatic light/dark theme sensing.

Chapter 9 — Evaluation – Validation – Conclusion: Analyses empirical results across hierarchy levels and volume percentiles, characterises the conditions under which different method combinations perform differently, and provides evidence-based recommendations for business forecasting practice.

Chapter 10 — Appendix: Contains the full sMAPE result tables for every base-model/reconciler combination and the complete code listings (both notebooks, `config.py`, the Streamlit dashboard, and the two requirements files).

Part I.

Domain Knowledge

2. Domain Knowledge

This chapter introduces the retail and business context relevant to this thesis: the hierarchical structure through which retail sales are organised, the concentration of sales volume across products, and the intermittent nature of item-level demand. These characteristics motivate the methodological decisions detailed in Chapter 5.

2.1. Retail Hierarchies and Structured Aggregation

Retail organisations rarely plan around a single, undifferentiated demand number. Sales are tracked and reported at several nested levels simultaneously – a national or company-wide total, individual regions, individual stores within those regions, and individual products within each store – with product-level detail commonly further grouped into categories and departments for merchandising and planning purposes. This structure exists because different decisions are made at different levels: a category manager plans assortment and promotions at the category or department level, a regional manager allocates inventory across stores within a region, and executive planning reasons about company-wide totals [CM15]. None of these levels is meaningful in isolation from the others.

This division of planning responsibility across levels is not incidental to how retailers happen to organise their reporting; it reflects a genuine difference in what each level of the business can act on. A category manager cannot meaningfully act on a single store’s daily sales fluctuation, and a regional inventory planner cannot act on a company-wide total without first breaking it back down into the individual stores that actually hold stock. Multi-echelon inventory and distribution planning is built around exactly this observation: decisions about where to hold stock, how much safety stock to carry at each echelon, and how frequently to replenish are all made at a specific level of the hierarchy, using a forecast appropriate to that level [SPP98]. A forecasting approach that ignores this structure – treating every series as an independent, unrelated problem – is consequently a poor match for how the business itself is actually run.

A forecast produced at any one of these levels is only useful to the business if it remains consistent with forecasts produced at every other level: the sum of all store-level forecasts within a region should equal that

region’s own forecast, and so on up the structure. This requirement is what the hierarchical forecasting literature refers to as *coherence*, and it is what distinguishes a genuinely hierarchical forecasting problem – where the levels are structurally linked – from a collection of unrelated, independently forecast time series [HA18]. Reconciling forecasts across such a structure is a well-established problem in the hierarchical forecasting literature, and is the central methodological concern of this thesis (see Chapter 3).

Retail hierarchies of this kind are typically *simple trees*: every store belongs to exactly one region, every item belongs to exactly one department, and so on, with no node belonging to more than one parent. This is in contrast to *grouped* or *cross-product* hierarchies, where a series might be classified along more than one dimension at once (for example, a product organised simultaneously by region and by brand, where neither dimension is subordinate to the other) [HA18]. The distinction matters beyond terminology: a simple tree has a single, unambiguous summing matrix that maps bottom-level series to every level above them, whereas a grouped hierarchy generally requires a different aggregation treatment to express the same coherence requirement. The dataset used in this thesis follows the simple-tree pattern, which is discussed further, together with the specific hierarchy levels used, in Section 5.5.

2.2. The Pareto Principle in Sales Concentration

A recurring observation across retail and inventory management is that a relatively small share of products accounts for a disproportionately large share of total sales volume, while the majority of the assortment – often the long tail of slower-moving items – contributes comparatively little individually. This pattern is commonly referred to as the *Pareto principle*, after the original observation that a small fraction of a population often accounts for a large fraction of some outcome of interest [Par96].

2.2.1. ABC Analysis in Inventory Practice

This observation underpins a standard inventory management practice known as *ABC analysis*, in which products are ranked by their contribution to sales or revenue and grouped into tiers – conventionally labelled A, B, and C – on that basis [SPP98]. "A" items are the small number of high-contribution products that typically receive the tightest inventory control, the most frequent demand review, and the greatest forecasting attention; "C" items are the much larger number of low-contribution products managed with looser, less frequent oversight, often through simple reorder-point rules rather than a fitted forecasting model at all.

"B" items occupy an intermediate tier, reviewed on a standard periodic cycle rather than the intensive or minimal treatment given to the other two.

The underlying rationale is one of proportionate effort: forecasting or stocking errors on a handful of high-volume products carry far greater business consequences – in lost sales, excess holding cost, or both – than the same relative error on a slow-moving item, so analytical and managerial attention should scale with a series' business importance rather than being spread uniformly across the assortment [SPP98]. This is a fundamentally different allocation of effort than treating every series in a catalogue as equally deserving of scrutiny, and it is the same logic, applied to forecast *evaluation* rather than forecast *production*, that motivates the volume-percentile evaluation framework used throughout this thesis.

This concentration is not merely a theoretical concern – it is clearly present in the dataset examined in this thesis, where a small number of item-store series account for a large share of total sales volume, while the majority of series individually contribute very little (the exact figures are presented in Section 5.7.2). This observation motivates the volume-segmented evaluation approach used throughout this thesis, which assesses forecast accuracy separately across cumulative volume bands rather than relying solely on a single, uniformly weighted metric (Section 5.7).

2.3. Intermittent and Sparse Demand in Item-Level Retail Data

A second defining characteristic of item-level retail demand is its *intermittency*: individual products, observed at the level of an individual store and an individual day, frequently record no sales at all on a given day, producing series with long stretches of zero-valued observations interrupted by occasional non-zero sales [SBC05]. This behaviour is a normal feature of low-to-moderate-volume retail products rather than a data quality issue, and it becomes more pronounced the more granular the level of observation – an item's daily sales at a single store are far sparser than that same item's sales summed across an entire region.

Demand of this kind is not a single, uniform category. The forecasting literature typically distinguishes intermittent demand from the related but distinct patterns of *erratic* demand (frequent sales, but highly variable in size) and *lumpy* demand (both infrequent and highly variable in size when it does occur), with "smooth" demand – frequent and stable – as the baseline case classical methods were designed for [SBC05]. Where a given series falls along these two axes, frequency and variability of size, has direct practical consequences: it determines which forecasting method is appropriate, and how much confidence can be placed in any resulting

estimate of variability for safety-stock purposes.

2.3.1. Croston's Method and the Response to Intermittency

Intermittent demand is well known in the inventory management and forecasting literature as a distinct challenge: classical forecasting methods developed for smooth, continuous demand can perform poorly on series where most observations are zero. A method fitted directly to a raw intermittent series – one that does not separate the two questions "how much, when it sells" and "how often does it sell" – tends to produce forecasts that are biased and unstable in a way that does not show up as clearly on smoother series [Cro72].

Croston's method, the specialised approach most commonly cited as the response to this problem, addresses it by decomposing the series into two separate quantities and smoothing each independently: the size of a sale when one occurs, and the interval between successive sales. Simple exponential smoothing is applied to each of these two derived series in turn, rather than to the original, mostly-zero series directly, and the forecast is then reconstructed as the ratio of the smoothed demand size to the smoothed inter-demand interval [Cro72]. Numerous refinements to this basic scheme – correcting known biases in the original formulation, or adapting the smoothing to specific demand regimes – have since been proposed, but they share this same core idea of separating demand *size* from demand *timing* rather than forecasting the raw series as a single quantity.

This decomposition is also why intermittent demand complicates safety-stock and replenishment planning more broadly, independently of which specific forecasting method is used: the variability that matters for setting a safety stock level is no longer a single number describing how much demand varies from period to period, but two separate sources of variability – how much demand varies in size when it occurs, and how much the timing between occurrences varies – that must be combined correctly to arrive at an appropriate stocking policy [SPP98].

The dataset used in this thesis exhibits this pattern clearly at the item level, while aggregate levels of the hierarchy (such as state-wide totals) show almost no sparsity at all, since some purchase of some product is effectively guaranteed to occur somewhere across an entire region on any given day (the exact figures are presented in Section 5.4). This contrast between item-level sparsity and aggregate-level stability is one of the primary reasons forecast accuracy, measured with the same metric, tends to look very different at different levels of a retail hierarchy – a pattern that is examined empirically, and quantified directly, in Chapter 9.

Part II.

Domain Machine Learning

3. Domain Machine Learning

This chapter introduces the forecasting methods compared in this thesis: three base forecasting models used to produce an initial, independent prediction for every series in the hierarchy, and three reconciliation methods used to make those independent predictions consistent with one another across the hierarchy. Section 3.1 covers the base models, and Section 3.2 introduces the concept of hierarchical coherence and the summing matrix that formalises it. Section 3.3 then presents the three reconciliation strategies built on that formalism. The specific configuration of these methods for the M5 dataset – the hierarchy levels used, the seasonal period, and the particular MinTrace variant applied – is a design decision covered in Chapter 5, and their implementation is described in Chapter 6. Empirical results comparing all combinations of base model and reconciliation method appear in Chapter 9.

3.1. Base Forecasting Models

Three base forecasting models are used in this thesis, chosen to span a deliberately narrow range of sophistication: a naive reference model, a classical exponential smoothing method, and a decomposition-based method with a strong track record in forecasting competitions. Each model is fitted independently to every series in the hierarchy – from the single Total series down to each individual item-store combination – with no awareness of the hierarchical structure the series sit within. It is this independence that creates the coherence problem addressed in Section 3.2.

3.1.1. HistoricAverage

The simplest of the three models, HistoricAverage, forecasts every future period as the arithmetic mean of all previously observed values of the series – the "average method" in the standard taxonomy of simple forecasting benchmarks [HA18]:

$$\hat{y}_{t+h} = \frac{1}{T} \sum_{i=1}^T y_i, \quad \text{for every horizon } h.$$

It has no concept of trend or seasonality, and its forecast is flat across the entire horizon. Despite its simplicity, HistoricAverage serves a specific

methodological purpose in this thesis beyond being a discardable baseline: because it is a linear function of the training data, it is already perfectly coherent across a hierarchy without any reconciliation step being applied, since the mean of a sum of series equals the sum of their individual means. This property makes it a natural sanity check for the reconciliation pipeline itself, and it also functions as the fallback model whenever the more sophisticated models below fail to fit a particular series (Chapter 6).

3.1.2. Exponential Smoothing (ETS)

Exponential smoothing methods forecast a series by taking a weighted average of past observations, with the weights decaying exponentially the further back in time an observation lies. In its simplest form, this is written recursively as

$$\hat{y}_{t+1} = \alpha y_t + (1 - \alpha)\hat{y}_t, \quad 0 \leq \alpha \leq 1,$$

so that each new forecast blends the latest observation with the previous forecast, and the smoothing parameter α controls how quickly the influence of older observations decays. The ETS (Error, Trend, Seasonal) framework formalises this idea as a state-space model in which each of the three components – the error structure, the trend, and the seasonal pattern – can independently be absent, additive, or multiplicative [Hyn+08]. Rather than requiring the analyst to specify which combination of components applies to a given series, an automated ETS procedure fits every valid combination and selects the one that best balances fit and parsimony according to an information criterion. This automatic model selection is what makes ETS practical to apply, unmodified, across tens of thousands of series with very different characteristics, as is required in Chapter 6. Where a seasonal component is included, the model requires a seasonal period to be specified; the rationale for the specific period used in this thesis is given in Chapter 5.

3.1.3. The Theta Method

The Theta method decomposes a series into two or more "theta lines" – transformed versions of the original series in which the local curvature (the second difference) is scaled by a coefficient θ . Formally, a theta line $Z_t(\theta)$ is defined as the series whose second differences are a scaled version of the original series' own:

$$\nabla^2 Z_t(\theta) = \theta \nabla^2 y_t,$$

where ∇^2 denotes the second-difference operator and y_t is the original series [AN00]. A θ value below one dampens short-term fluctuations,

producing a smoother, longer-term view of the series' level, while a θ value above one amplifies them, emphasising short-term dynamics. The classical implementation combines a long-term line ($\theta = 0$, equivalent to a linear trend) with a short-term line ($\theta = 2$) in equal weight, after first removing any seasonal component so that the decomposition operates on a deseasonalised series [AN00]. The method was the strongest performer in the M3 forecasting competition, and its combination of a simple decomposition step with strong empirical accuracy is the reason it is included here as the most sophisticated of the three base models.

3.2. Hierarchical Coherence and the Summing Matrix

A hierarchical time series structure arranges a collection of series so that some are the sum of others: a company total is the sum of its regions, each region is the sum of its stores, and so on down to the most granular, or *bottom-level*, series that are not themselves the sum of anything else in the structure. When every node in the structure belongs to exactly one parent, the result is a *simple tree* hierarchy; structures in which a series can be classified along more than one dimension at once (for example, jointly by region and by product brand) instead form *grouped* or *cross-product* hierarchies, which require a different aggregation treatment [HA18]. This distinction, and which of the two applies to the dataset used in this thesis, is addressed in Chapter 5.

Forecasting every series in such a structure independently – fitting a separate model to the company total and a separate model to each individual store, with no link between the two – produces forecasts that are individually reasonable but collectively *incoherent*: there is no guarantee that the sum of the store-level forecasts will equal the independently produced total-level forecast. Hierarchical forecasting formalises the relationship between levels using a *summing matrix*, conventionally denoted S , which encodes exactly how each bottom-level series contributes to every aggregate series above it. If b_t denotes the vector of bottom-level values at time t , the complete set of values across every level of the hierarchy, y_t , is given by

$$y_t = S b_t.$$

A set of forecasts is coherent precisely when it also satisfies this relationship: the forecasts at every aggregate level must be recoverable from the bottom-level forecasts via the same matrix S that defines how the actual data aggregates [HA18]. Since independently fitted base forecasts generally do not satisfy this relationship, a subsequent *reconciliation* step is required to enforce it.

Every reconciliation method considered in this thesis can be written as a single general transformation of the base forecasts:

$$\tilde{y}_t = S G \hat{y}_t,$$

where $\hat{y}_t$ is the vector of independently produced base forecasts across *every* level of the hierarchy, G is a matrix that maps this full, generally incoherent set of base forecasts onto a single reconciled estimate of the bottom level, and post-multiplying by S re-aggregates that reconciled bottom level back up through every level above it – guaranteeing coherence by construction, whatever G turns out to be [WAH19]. The three reconciliation strategies compared in this thesis differ only in how G is constructed; each is presented next as a specific, named choice of this one matrix.

3.3. Reconciliation Theory

Reconciliation methods differ in how much they trust the base forecast produced at each level of the hierarchy, ranging from complete trust in the bottom level, to complete trust in the top level, to a compromise that adjusts every level simultaneously. Figure 3.1 contrasts the three methods by the direction information flows between levels: strictly upward for Bottom-Up, strictly downward for Top-Down, and simultaneously in both directions for MinT.

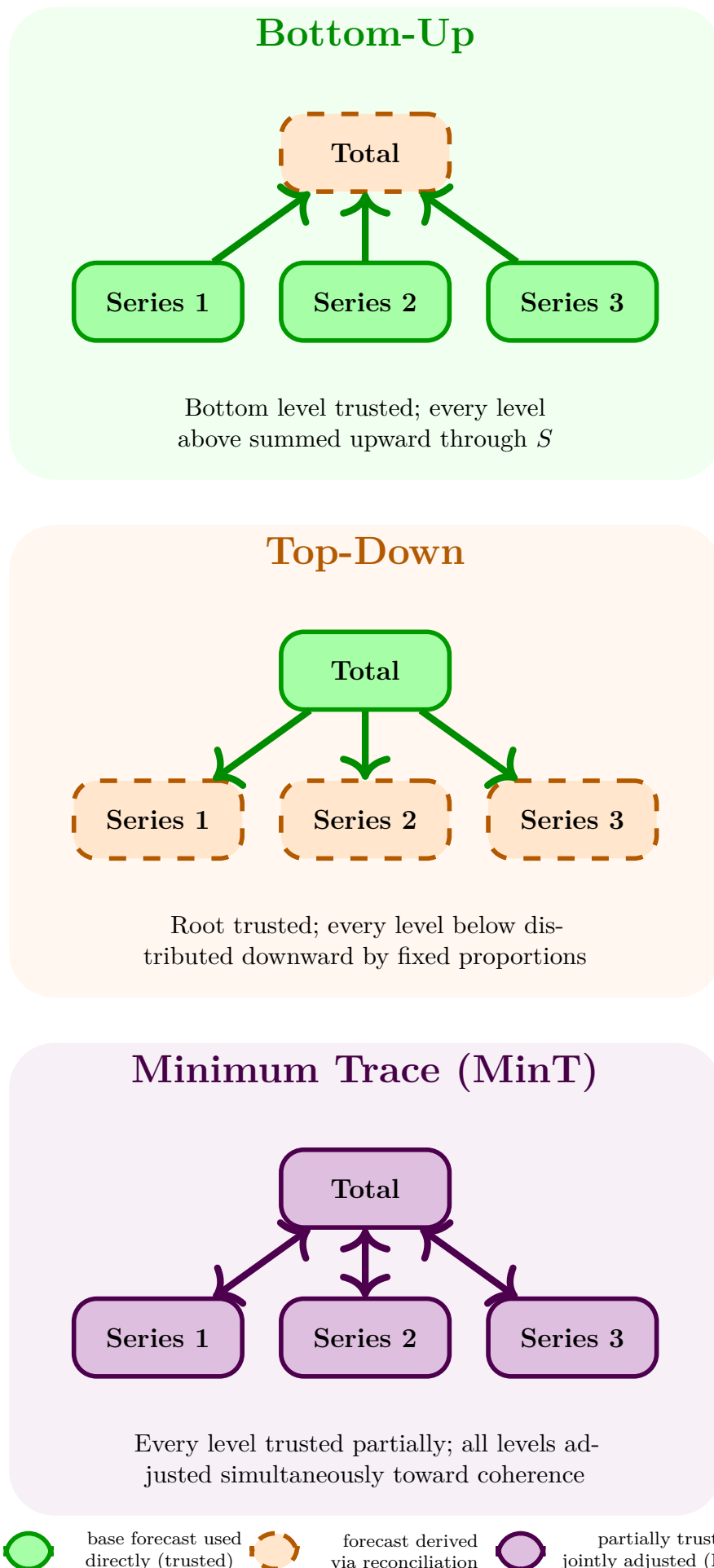


Figure 3.1.: Information flow between hierarchy levels under each reconciliation strategy.

3.3.1. Bottom-Up

The Bottom-Up approach forecasts only the bottom-level series directly, and obtains every aggregate-level forecast by summing the relevant bottom-level forecasts through the summing matrix S [OWE68; DWD76]. In terms of the general reconciliation matrix G introduced in Section 3.2, this is the simplest possible choice available: $G = [\mathbf{0} \mid I]$, a matrix that discards every base forecast produced at an aggregate level entirely and passes the bottom-level base forecasts through completely unchanged. Because it relies exclusively on the most disaggregated forecasts, Bottom-Up preserves all information available at the item level and never overrides it with a top-down assumption. The trade-off is that bottom-level series are typically the noisiest and, in a retail setting, the most intermittent in the hierarchy, so any forecasting error made at that level propagates directly, and without correction, into every aggregate forecast above it.

3.3.2. Top-Down

The Top-Down approach forecasts only the single series at the root of the hierarchy, and distributes that top-level forecast downward to every lower level using a set of disaggregation proportions, most commonly each series' historical average share of the total [GS90]. In terms of the same general matrix G , this corresponds to a matrix with a single non-zero column – the one selecting the top-level base forecast – populated with the disaggregation proportions p : every reconciled bottom-level estimate is simply a fixed share of that one top-level number, and every other base forecast produced anywhere else in the hierarchy is discarded. Because the root-level series is typically the most stable and least intermittent series in the entire hierarchy, its forecast is comparatively easy to produce accurately. The trade-off runs in the opposite direction to Bottom-Up: Top-Down cannot represent a lower-level series' own trend or seasonal pattern if that pattern differs from its historical share of the total, which can introduce systematic bias at the lower levels of the hierarchy [WAH19]. Top-Down reconciliation is only well defined when the hierarchy has a single root series to distribute downward from; how this condition is ensured for the dataset used in this thesis is addressed in Chapter 5.

3.3.3. Minimum Trace (MinT)

Rather than committing entirely to the bottom level or the top level, the Minimum Trace (MinT) framework adjusts the base forecasts at every level of the hierarchy simultaneously, choosing the G matrix that minimises the total variance of the reconciled forecast errors, subject to the requirement that the result remains coherent under S [WAH19].

Formally, this optimal G has the closed form

$$G = (S^\top W^{-1} S)^{-1} S^\top W^{-1},$$

where W is the covariance matrix of the base forecasts' errors [WAH19]. This is a generalised least-squares projection of the base forecasts onto the coherent subspace defined by S , weighted by exactly how much each series' base forecast can be trusted relative to the others. The original formulation allows this covariance estimate W to range from a simple structural approximation,

$$W = \text{diag}(S\mathbf{1}),$$

where $\mathbf{1}$ is a vector of ones – so that each node's weight is simply the count of bottom-level series summing into it, the row sums of S itself – to a full shrinkage estimator of the sample error covariance [WAH19]. At the scale of a large retail hierarchy with tens of thousands of bottom-level series, estimating a full covariance matrix becomes computationally prohibitive, which is why implementations of MinT commonly offer a structurally-scaled variant as a tractable approximation; the specific variant used in this thesis, and the trade-off this choice implies, is discussed in Chapter 5.

MinT is generally presented in the literature as the theoretically preferred reconciliation method, since Bottom-Up and Top-Down can each be shown to be special, restrictive cases of the same G framework introduced above [WAH19]: both are recoverable as particular, fixed choices of G , while MinT is the specific choice proven to be optimal. This generalised least-squares approach to reconciliation predates MinT itself, however: reconciling every level of a hierarchy simultaneously through a regression-style combination of the independently produced base forecasts was first shown to improve on both Bottom-Up and Top-Down at once, using a simple weighting scheme based only on the hierarchy's structure [Hyn+11]. What Wickramasuriya et al. added, and what the "Min" in MinT's name refers to, is the proof that setting W to the true covariance of the base forecast errors minimises the trace of the reconciled error covariance among all reconciliations that preserve unbiasedness ($SGS = S$), provided the base forecasts are themselves unbiased [WAH19]. Top-Down lies outside this class, and clipped base forecasts are no longer strictly unbiased. Because the true covariance is unknown, every practical MinT variant approximates it: the structural scaling $W = \text{diag}(S\mathbf{1})$, introduced by Athanasopoulos et al. [Ath+17], is the simplest such approximation rather than the optimal weighting itself, and the non-negativity constraint used in this thesis follows Wickramasuriya, Turlach, and Hyndman [WTH20].

Whether this theoretical advantage is realised in practice depends on how well the chosen covariance approximation reflects the true error

structure of the series being reconciled – a question this thesis investigates empirically in Chapter 9.

Part III.

Domain Knowledge - Tools

4. Domain Knowledge – Tools

This chapter describes the Python forecasting ecosystem used to implement the pipeline introduced in Chapter 5: **statsforecast** for fitting the base forecasting models, **hierarchicalforecast** for building the hierarchy and reconciling forecasts across it, **utilsforecast** for the accuracy metric used throughout this thesis, and the supporting data and visualisation stack (pandas, NumPy, PyArrow, Streamlit, Plotly). Design rationale for why particular models, reconciliation methods, and the sMAPE metric were chosen is covered in Chapter 5 methodology; how these libraries are actually called and configured is covered in Chapter 6 Application Development.

4.1. statsforecast

statsforecast [Gar+22] is the library used to fit all three base forecasting models used in this thesis – **HistoricAverage**, **AutoETS**, and **Theta** – across every series in the hierarchy at once. Rather than looping over series individually, its **StatsForecast** class accepts a list of model instances and fits them in parallel across the full panel of series in a single call, which is what makes fitting tens of thousands of series computationally practical.

Two configuration options are used throughout the pipeline. First, a **fallback_model** (**HistoricAverage()**) is registered as a safety net: on a panel of this size, some series will be too short, too sparse, or too irregular for ETS or Theta to fit successfully, and rather than the entire run failing on one problematic series, that series is silently given the **HistoricAverage** forecast instead. How often this fallback actually triggers is a pipeline result, reported in Sections 6.7 and 9.7 together with the rest of the testing and validation checks. Second, each model is given a distinct **alias** (**histavg**, **ets**, **theta**), which becomes both its output column name and part of the naming scheme centralised in **config.py** (Chapter 7).

4.2. hierarchicalforecast

hierarchicalforecast [Oli+22] provides the two capabilities that make the hierarchy itself computationally usable: building it, and reconciling

forecasts across it.

Its `aggregate()` function takes the bottom-level (item-store) panel and a specification of the hierarchy's levels, and returns three objects in a single call: `Y_df`, the target values rolled up to every level of the hierarchy (not just the bottom); `S_df`, the summing matrix – a compact representation of exactly how bottom-level series roll up into every higher node, stored in sparse form to keep memory use manageable at this scale; and `tags`, a lookup of which series belong to which hierarchy level, used later when evaluating accuracy separately per level. Because this is built once from a single, already-fixed item universe, the same hierarchy serves the training, test, and live periods without being rebuilt.

Reconciliation itself is performed by a `HierarchicalReconciliation` object, to which the three reconciliation methods used in this thesis are passed as sparse variants: `BottomUpSparse` (Section 3.3.1), `MinTraceSparse` (configured with `method="wls_struct"`, the structural weighting scheme derived formally in Section 3.3.3, and `nonnegative=True`, enforcing the constraint motivated in Section 5.6.2), and `TopDownSparse` (configured with `method="proportion_averages"`, the historical-share disaggregation described in Section 3.3.2). The library also supplies an `evaluate()` function that computes sMAPE per series and averages the result within each hierarchy level, which is what produces the by-level accuracy tables used in Chapter 9.

Figure 4.1 summarises how the three Nixtla libraries introduced in this and the next section hand data to one another: `aggregate()` builds the hierarchy once, `StatsForecast` fits every series against the resulting panel, `HierarchicalReconciliation` combines the base forecasts with the hierarchy's own structure to produce coherent forecasts, and `evaluate()` scores the result. This is a narrower, tool-level view than the full pipeline diagram in Chapter 6 (Figure 6.1): it shows which library is responsible for which data object, not every processing step in between.

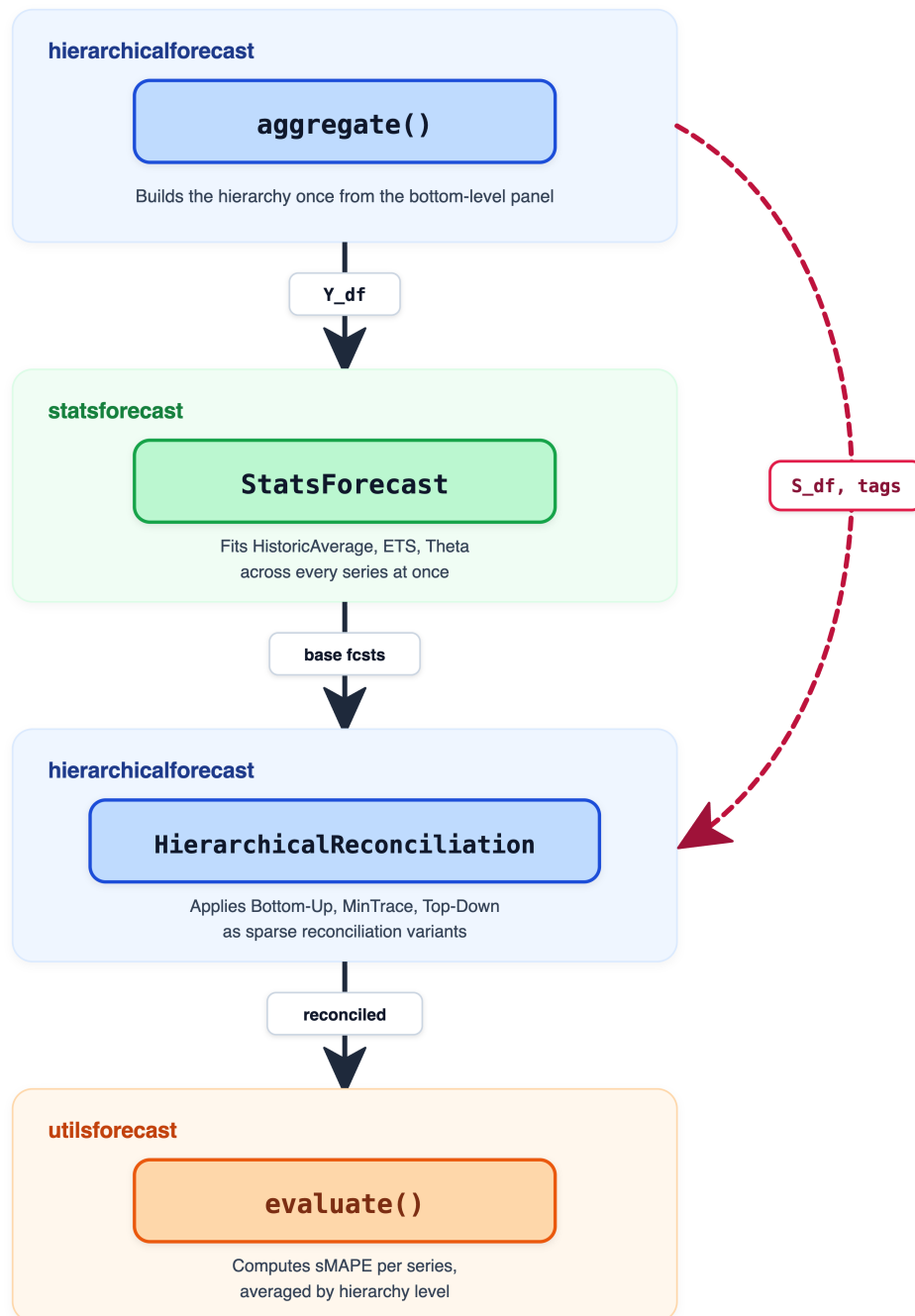


Figure 4.1.: Data objects passed between the three Nixtla libraries used in this thesis's forecasting pipeline.

4.3. utilsforecast

`utilsforecast` [Nix26] supplies two things used throughout this thesis: the `smape` function, and a plotting helper.

The `smape` function is the single accuracy metric used everywhere in this thesis, and its exact formula, scaling convention, and zero/zero handling are presented once, in full, in Section 5.7, and are not repeated here. What belongs in this chapter is a reproducibility check rather than a definition: the pipeline notebook prints the library’s own source code for `smape` alongside this thesis’s independently written implementation, confirming that both compute the identical $|F - A|/(|F| + |A|)$ fraction with the same treatment of the zero/zero case, before the result is scaled to a percentage. Establishing this equivalence once, at the tooling level, means every sMAPE figure reported later in this thesis can be read as coming from a single, unambiguous formula rather than two conventions that merely look comparable.

The library’s `plot_series` function is a secondary tool used to overlay a series’ actual history against its base and reconciled forecasts for visual inspection, as used in the pipeline’s optional visualisation step.

4.4. Supporting Stack

Several general-purpose libraries support the pipeline without being forecasting-specific themselves.

pandas [McK10] and **NumPy** [Har+20] handle every data-wrangling step outside of model fitting and reconciliation: reshaping the raw wide-format sales file (one row per item-store, one column per day) into long format, aggregating daily observations to monthly totals, and constructing the complete item-store-by-month grid that gives every series a value for every month, including months with zero sales.

PyArrow [Apa26] provides Parquet support, used in two places for reproducibility. First, both the EDA and pipeline notebooks cache the raw M5 CSV files to Parquet after their first load, since Parquet is faster to re-read than re-parsing a CSV and preserves column data types (dates remain dates rather than becoming strings). Second, the pipeline’s final step exports four Parquet files – a hierarchy lookup table, the actual sales history, and the base and reconciled forecasts – which are the sole data contract the Streamlit dashboard reads from (Chapter 8).

Streamlit [Str26] is the framework the dashboard (`app/streamlit_app.py`) is built with, and the basis for its deployment on Streamlit Community Cloud. **Plotly** [Plo26] renders the dashboard’s interactive charts and is also used as the plotting engine for `utilsforecast`’s `plot_series` helper within the notebooks. For static exploratory plots that do

not need to be interactive – the histograms and bar charts in the EDA notebook – **Matplotlib** [Hun07] is used instead.

Part IV.

Methodology

5. Methodology

This chapter sets out the design of the empirical study: the dataset and the time windows used to train, test, and generate live forecasts (Section 5.3), the data-cleaning decisions applied before any model is fitted (Section 5.4), the structure of the hierarchy the models are fitted against (Section 5.5), the experimental design comparing base models and reconciliation methods (Section 5.6), the evaluation framework used to score them (Section 5.7), and the limitations and fairness considerations that follow from these choices (Sections 5.8 and 5.9). The forecasting methods themselves were introduced in Chapter 3; their implementation is described in Chapter 6, and the resulting accuracy figures are reported in Chapter 9.

5.1. Research Design

This thesis is structured as an empirical comparative study rather than a hypothesis-testing one: no single method combination is assumed in advance to be superior, and the research questions set out in the introduction ask how accuracy *patterns* differ across hierarchy levels, base models, and volume segments, not whether one specific combination beats another at a fixed significance level. Every combination of base forecasting model and reconciliation method is fitted and evaluated under identical conditions – the same training data, the same forecast horizon, and the same accuracy metric – so that any differences observed can be attributed to the choice of method rather than to inconsistencies in how each was evaluated. The exploratory framing follows directly from SQ1–SQ3: each sub-question asks how a pattern changes across a dimension (hierarchy level, base model, or volume band) rather than positing a specific directional hypothesis to be confirmed or rejected.

5.2. The KDD Process Framework

The overall pipeline is organised around the Knowledge Discovery in Databases (KDD) process [FPSS96], which structures a data-driven study into five stages: selection, preprocessing, transformation, data mining, and interpretation/evaluation. Figure 5.1 maps these five generic stages onto the concrete steps carried out in this thesis. Selection corresponds

to loading the raw M5 sales and calendar files; preprocessing covers the data-cleaning decisions detailed in Section 5.4; transformation covers building the complete item-store-month grid and aggregating it into the hierarchy described in Section 5.5; data mining covers fitting the base forecasting models and applying reconciliation (Chapter 6); and interpretation/evaluation covers the accuracy analysis and the answers to SQ1–SQ3 (Chapter 9). Using this framework keeps the two concerns of this chapter – *what* was decided and *why* – clearly separated from the *how* of Chapter 6, since each KDD stage has a single, unambiguous home in one chapter or the other.

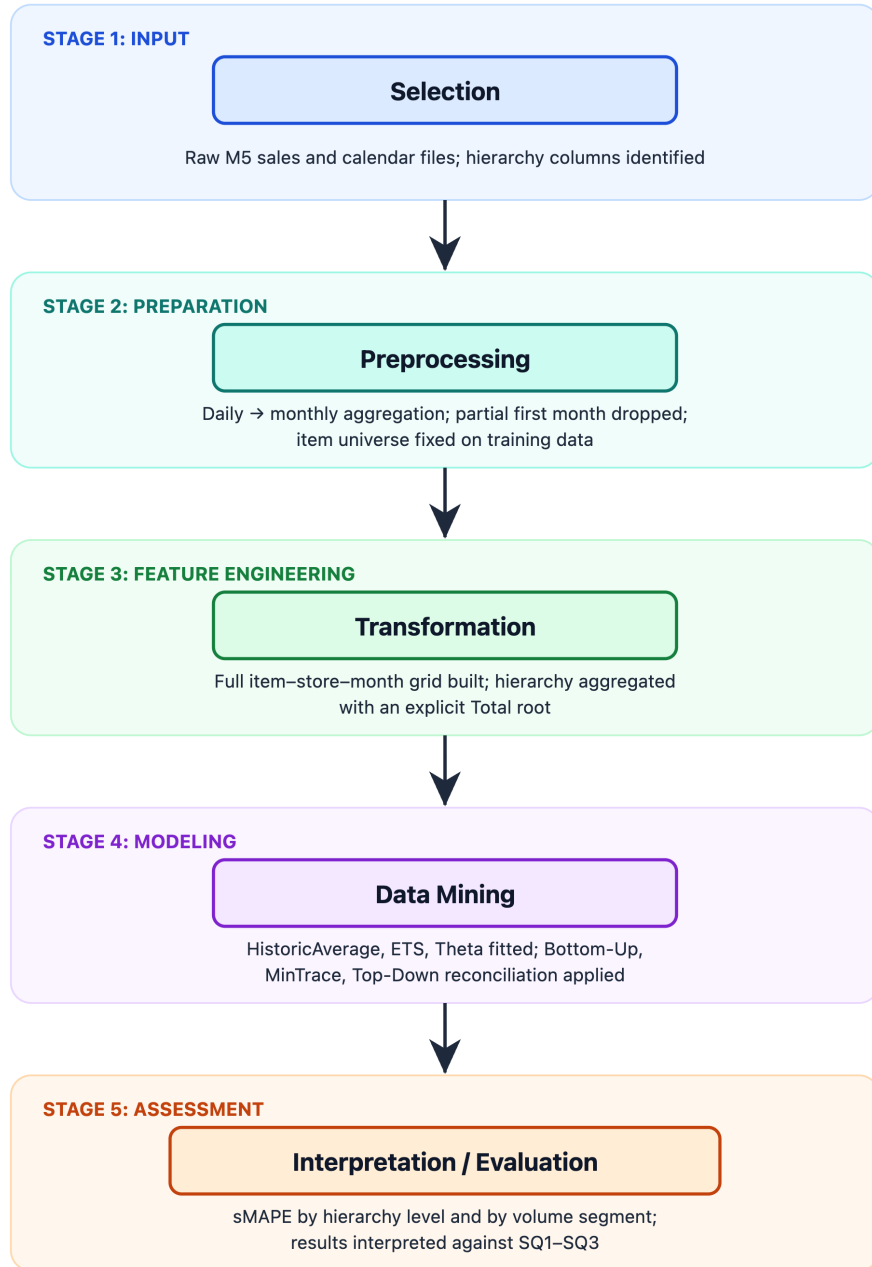


Figure 5.1.: The five KDD process stages mapped to this thesis’s own pipeline steps.

5.3. Dataset

This thesis uses the M5 Forecasting – Accuracy competition dataset [MSA22], comprising daily unit-sales records for Walmart stores across

three US states. The raw sales file contains 30,490 rows, one per item-store combination, and 3,049 unique items. Each series is classified along four nested dimensions: 3 states (California, Texas, Wisconsin), 10 stores (California contributes 4, Texas and Wisconsin 3 each), 3 product categories (Foods, Hobbies, Household), and 7 departments (three within Foods, two each within Hobbies and Household). These are exactly the columns used to define the hierarchy levels in Section 5.5. The daily sales data itself spans 2011-01-29 to 2016-04-24 (1,913 days); an accompanying calendar file extends further, to 2016-06-19, since it also covers the competition’s evaluation period, for which no actual sales figures exist in this dataset. This distinction between the calendar’s date range and the sales data’s actual date range is the reason the live forecast window (Section 5.3.1) needs to be described carefully rather than simply quoted from the calendar file. Figure 5.2 shows how total sales volume is distributed across these categories and states: Foods contributes roughly three times the combined volume of Household and Hobbies, and California’s total sales are noticeably higher than Texas’s or Wisconsin’s despite the state contributing only one more store.

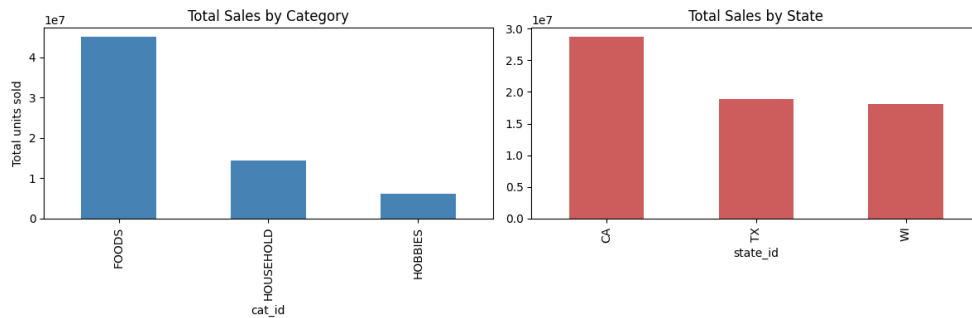


Figure 5.2.: Total training-period sales by product category and by state.
Source: `01_eda.ipynb`.

5.3.1. Time Windows and the Live Evaluation Period

Three non-overlapping periods are defined along the time axis: a training period, a test period, and a live period. Models are fitted once on the training period alone and evaluated on the immediately following test period, whose actual outcomes are known and withheld from model fitting; performance on this test period is what Chapter 9 reports. A second forecast, trained on the training and test periods combined, produces the live period – this is the forecast horizon written into the dashboard described in Chapter 8. Figure 5.3 lays out all three periods on a single timeline, since the boundaries between them do not line up neatly with where the real underlying data itself ends.



Figure 5.3.: The training, test, and live time windows (schematic, not to scale). The live window is trained on training and test data combined and extends three months past where real sales data actually ends.

The live period is deliberately extended to 2016-07-01, three months beyond the point at which the real sales data ends (2016-04-24). This is done purely so the dashboard has a horizon of live forecasts to display; there is no ground truth available for May, June, or July 2016 in this dataset, and the live period is consequently never used for any accuracy evaluation in this thesis, only for the deployed dashboard. Any reference to the data range in later chapters should accordingly be read as "training and test data through April 2016, with the live forecast window extended to July 2016 for the dashboard's benefit" rather than as a single continuous range of real observations running to mid-2016.

5.4. Data Preparation

Four cleaning decisions are applied to the raw daily data before any model is fitted, in the order presented below. This order matters: later steps depend on earlier ones having already run (for example, the item universe fixed further below is defined using monthly, not daily, sales figures).

5.4.1. Daily-to-Monthly Aggregation

At the daily, item-store level, 68.2% of all observations record zero sales, against 0.0% of observations at the state level, where some purchase of some product is effectively guaranteed on any given day across an entire state. This intermittency (introduced conceptually in Section 2.3) is characteristic of item-level retail demand and becomes markedly more pronounced the more granular the level of observation. Figure 5.4 shows this sparsity across the full item catalogue rather than as a single averaged figure: the distribution is skewed heavily toward the sparse end, with

most items recording zero sales on well over half of all days, rather than clustering near zero as a small, uniform intermittency rate would suggest. Aggregating daily sales to monthly totals does not remove this sparsity, but it substantially reduces it: a month in which an item sold at all is no longer a zero observation, even if most of the individual days within it were. Temporal aggregation of this kind is a recognised trade-off in the forecasting literature more broadly: coarsening the sampling frequency of a series trades away sub-period detail in exchange for a more stable, less noisy signal to forecast from [Ath+17]. This aggregation also matches the planning horizon most retail businesses use in practice, and it is the step from which every later part of the pipeline – the calendar-boundary fix below, the hierarchy, and the forecasting models – proceeds.

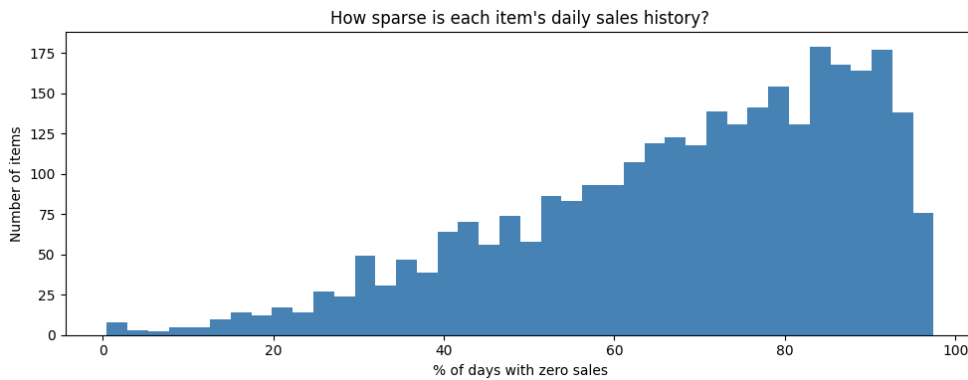


Figure 5.4.: Distribution across items of the percentage of days with zero sales, at the daily item-store level. Source: `01_edu.ipynb`.

5.4.2. Dropping the Partial First Calendar Month

Because the sales data begins on 2011-01-29, resampling to calendar months places only three real days – 2011-01-29 through 2011-01-31 – into the resulting 2011-01-01 monthly bucket, against a normal January’s 31. That three-day bucket records 88,163 units of total sales, compared with 726,375 in the following, complete February bucket: only 12.1% of a normal month’s volume. Left in place, this heavily undercounted bucket becomes one of only four training-period Januaries available to a model configured with a 12-month seasonal period (Section 5.6.1), distorting the January seasonal index those models rely on. The partial bucket is therefore dropped before any further processing, leaving every other monthly bucket untouched.

5.4.3. Fixing the Item Universe on Training Data Only

Not every item was on sale for the full 2011–2016 window; some were introduced only partway through. Checking each item’s monthly sales against the training/test boundary shows that 25 of the 3,049 raw items recorded no sales at all before that boundary, leaving 3,024 items with genuine training history. These 25 items are removed from the dataset entirely, because no model fitted only on training data could ever have learned a pattern for a series with no training-period observations – forecasting them would amount to guessing rather than extrapolating. This filter is applied once, at the training boundary, before the hierarchy or the forecast grid is built, so the training, test, and live periods – and the evaluation built on top of them – automatically share an identical, training-consistent item universe with no later step at which their item sets could drift apart. With 10 stores and 3,024 remaining items, the bottom level of the hierarchy comprises 30,240 item-store series.

5.4.4. Adding an Explicit Total Root

California, Texas, and Wisconsin do not contribute an equal number of stores: California has 4, Texas and Wisconsin have 3 each. Without an explicit node sitting above the state level, the hierarchy is really three separate trees, one rooted at each state, with no single top. Top-Down reconciliation (Section 3.3.2) is only mathematically well defined when there is one root series to distribute downward from; without an explicit root, an implementation asked to reconcile top-down has to infer which of the three state-level series counts as "the top," typically defaulting to whichever aggregates the most bottom-level series. A single constant column, "Total," is therefore introduced above the state level, giving the hierarchy one unambiguous root and making Top-Down reconciliation well defined across the entire structure.

5.5. Hierarchy Construction

With the Total root in place, the hierarchy used throughout this thesis has six levels: Total, State, Store, Category, Department, and Item, exactly as shown in Figure 1.1 in the introduction. Every node belongs to exactly one parent at the level above it – every store belongs to one state, every department instance belongs to one store and one category, and so on – which makes this a *simple tree* hierarchy in the sense introduced in Section 3.2, rather than a grouped or cross-product hierarchy in which a series could be classified along more than one independent dimension at once. This distinction matters because the reconciliation methods compared in this thesis (Section 3.3) are defined in terms of the summing

matrix S that a simple tree naturally produces; a cross-product hierarchy would require a different aggregation treatment not used here. Building this six-level structure over the training-consistent item universe described above yields 30,354 total series: 1 Total node, 3 State nodes, 10 Store nodes, 30 Category nodes, 70 Department nodes, and 30,240 Item-level (bottom) series.

5.6. Experimental Design

The comparison at the centre of this thesis is a full factorial design [Mon04] crossing three base forecasting models (HistoricAverage, ETS, Theta; Section 3.1) with three reconciliation methods (Bottom-Up, MinTrace, Top-Down; Section 3.3), giving nine base-model/reconciler combinations in total. Every one of these nine combinations is then evaluated along two further, independent dimensions: the six hierarchy levels defined above, and the six cumulative volume-percentile bands defined in Section 5.7.2. Figure 5.5 lays out the resulting $3 \times 3 \times 6 \times 6$ design space. Implementing this design – fitting the models and applying the reconciliation methods – is covered in Chapter 6; the two configuration decisions that shape every cell of this design are set out next.



Figure 5.5.: The 3×3 base-model / reconciler grid; every cell is further evaluated across 6 hierarchy levels and 6 volume-percentile bands.

5.6.1. Model Configuration: A 12-Month Seasonal Period

Figure 5.6 plots total monthly sales across every item and store and shows a clear, repeating annual pattern, rather than a series that simply trends up or down at random. ETS and Theta both require a seasonal period to be specified when a seasonal component is used; this thesis sets that period to 12 months for both models. Without it, both models default to a seasonal period of 1 – effectively no seasonality at all – and produce

forecasts that are close to flat, missing the recurring pattern the monthly data actually shows. `HistoricAverage` has no seasonal component and is unaffected by this setting.

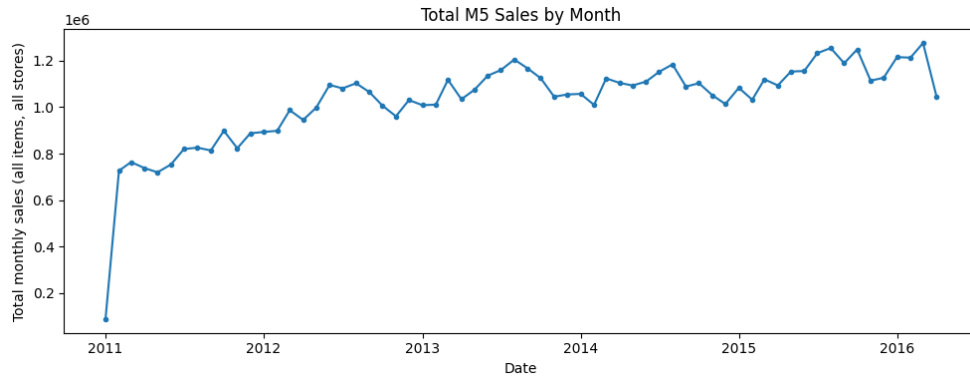


Figure 5.6.: Total monthly sales across all items and stores, 2011–2016, showing a recurring annual pattern.

5.6.2. Ensuring a Fair Comparison: Clipping Negative Base Forecasts

Units sold cannot be negative, and the raw daily data confirms this constraint holds throughout: none of the 58,327,370 raw daily observations is negative, and the minimum observed daily value is 0. The base forecasting models, however, are not aware of this floor – ETS and Theta are free to extrapolate a downward trend straight through zero, while `HistoricAverage`, as a simple mean, cannot. This asymmetry creates a risk of an unfair comparison between reconciliation methods specifically: `MinTrace` is configured to enforce non-negativity internally as part of its reconciliation step, while `Bottom-Up` simply inherits whatever value the base model produced for each bottom-level series. Comparing the two reconcilers on unclipped base forecasts would therefore partly be comparing "reconciliation with an enforced floor at zero" against "reconciliation without one," rather than comparing the reconciliation methods themselves on equal footing. Every base forecast is accordingly clipped at zero before any reconciliation method is applied, so that `Bottom-Up`, `MinTrace`, and `Top-Down` all start from the same, already-nonnegative base forecasts; the resulting clip counts are reported alongside the implementation in Chapter 6.

5.7. Evaluation Framework

Two evaluation lenses are applied to every combination in the design space described above: accuracy by hierarchy level, and accuracy by business

volume segment. Both use the same underlying metric, defined next.

5.7.1. sMAPE

Accuracy throughout this thesis is measured with the symmetric Mean Absolute Percentage Error (sMAPE), chosen for its demonstrated rank stability on hierarchical retail data: across the top-performing submissions to the M5 competition, sMAPE achieves a rank-stability correlation of 0.75–0.83 between comparable dataset splits, compared with just 0.24–0.56 for the competition’s own official metric, price-weighted RMSSE [Hew+21] – and its interpretability relative to scale-dependent alternatives. For a forecast F and an actual value A ,

$$\text{sMAPE} = \text{mean} \left(\frac{|F - A|}{|F| + |A|} \right) \times 100.$$

A worked example: for an actual value of 10 and a forecast of 12, the error is $|12 - 10| = 2$ and the denominator is $|12| + |10| = 22$, giving an sMAPE contribution of $2/22 \approx 9.09\%$. Where both the actual value and the forecast are zero – a dead series correctly predicted as dead – the ratio is defined as zero rather than left undefined, so that a perfect forecast of "no sales" is scored as a perfect forecast rather than penalised or excluded.

This thesis uses the 0–100% scaling convention shown above, rather than the alternative 0–200% version some textbooks define by adding a factor of two to the denominator. Neither convention is more "correct" than the other; they are simply different scalings of the same underlying idea, and using two of them side by side risks producing numbers that look comparable but are not. The 0–100% convention is used here because it is the convention implemented directly by the sMAPE function in this thesis’s forecasting toolchain (Chapter 4), so only one scale is ever in play.

5.7.2. Volume-Percentile Evaluation Framework

Alongside the hierarchy-level view, forecast accuracy is evaluated separately across business-relevant volume segments, reflecting the Pareto-driven motivation introduced in Section 2.2: a small number of series typically account for a disproportionate share of total sales, and treating every series as equally important would obscure how well a method serves the series that matter most operationally. Every bottom-level (item-store) series is ranked by its total *training-period* sales volume alone – never by test-period sales, since a real forecaster would not have access to future sales when deciding which series matter most. Walking down this ranking from the largest seller, cumulative bands are defined at 5%, 10%, 20%, 50%, 80%, and 100% of total training volume; each band contains

whichever series, taken from the top, are needed to reach that cumulative share.

Because sales volume is highly concentrated, these bands correspond to very different numbers of series: across the 30,240 bottom-level series in this dataset, the top 5% of training volume is covered by just 29 series (0.10% of all series), the top 10% by 94 series (0.31%), the top 20% by 326 series (1.08%), the top 50% by 2,151 series (7.11%), and the top 80% by 8,155 series (26.97%); the top 100% band is, by definition, all 30,240 series. "Top 5% of volume" is therefore a statement about sales share, not about the fraction of the product catalogue it covers. This evaluation is carried out strictly at the bottom (item-store) level and is never re-aggregated into a synthetic "state" or "category" total built from only the selected series in a band, since a state built from 5% of its items would not represent a real state-level forecast.

5.8. Limitations of the Design

Several limitations follow from the design choices set out above. First, MinTrace is implemented throughout using structural scaling (`wls_struct`), which weights each hierarchy node only by how many bottom-level series sum into it. The sparse implementation used in this thesis (Chapter 4) does not offer a shrinkage-based covariance estimator; it does offer `wls_var`, which weights each node by the variance of its in-sample forecast errors, but this requires in-sample fitted values for all 30,354 series and was not evaluated. Both variants, and Top-Down with forecast rather than historical proportions, are left as future work. Second, every finding in this thesis is drawn from a single dataset – the M5 Walmart retail data – and while this dataset shares key structural features with the MRO hierarchical forecasting context that motivated this thesis (Chapter 1), the extent to which these specific findings generalise to other domains or hierarchies is not tested here. Third, aggregating daily sales to monthly totals (Section 5.4.1) trades off intra-month granularity for a more tractable, less sparse series, in line with the general temporal-aggregation trade-off noted above [Ath+17]: any weekday effects, promotional spikes, or other sub-monthly demand patterns are smoothed away by this choice and are outside the scope of what this thesis's forecasts can capture. Fourth, all results come from a single forecast origin (one train/test split), use no exogenous drivers such as prices, promotions or events, and evaluate point forecasts only, without prediction intervals. Fifth, sMAPE is discontinuous at zero: whenever the actual value is 0, any forecast above 0 – however small – scores the maximum error, while only an exact 0 scores none [HK06]. At the item level, where many test months have zero sales, this makes the metric sensitive to tiny positive values. It rewards

clipping negative forecasts to exactly zero, and it likely explains much of MinTrace's higher item-level error, since its non-negative optimisation can return small positive values where Bottom-Up returns zero.

5.9. Fairness/Bias Note

The volume-percentile framework defined above is a deliberate methodological choice, and it is worth being explicit about what it optimises for. By design, it directs evaluation attention toward the small number of high-volume series that a business would care about most, which means that accuracy on the long tail of low-volume, individually minor series is structurally given less weight in how "success" is judged throughout this thesis – even though the results chapter shows this long tail is exactly where the gap between the base forecasting models is most pronounced. This is not a flaw to be corrected so much as a framing decision that should be read alongside the results in Chapter 9: a method combination that performs well under this framework is one well suited to business-critical, high-volume forecasting, not necessarily one that serves every individual series in the catalogue equally well.

Part V.

Application Development

6. Application Development

This chapter documents how the design decisions set out in Chapter 5 were built: fitting the three base forecasting models (Section 6.1), clipping their negative forecasts to zero before reconciliation (Section 6.2), applying the three reconciliation methods (Section 6.3), combining the resulting forecast tables and attaching ground truth (Section 6.4), the pipeline's overall developer-facing flow (Section 6.5), the naming, parameter, and error-handling conventions that run through the code (Section 6.6), and the pipeline's built-in test protocol (Section 6.7). The forecasting and reconciliation methods themselves were introduced in Chapter 3, and why each configuration choice was made is covered in Chapter 5; this chapter cross-references those decisions rather than re-arguing them. Code organisation and reproducibility concerns that span the whole repository, including `Code/config.py` itself, are addressed in Chapter 7, and the accuracy figures produced by this implementation are reported in Chapter 9.

6.1. Model Fitting

All three base models are fitted through a single `StatsForecast` instance, configured once and reused for both forecast horizons described below:

```
1000 sf = StatsForecast(  
1001     models=[  
1002         HistoricAverage(alias="histavg"),  
1003         AutoETS(season_length=SEASON_LENGTH, alias="ets"),  
1004         Theta(season_length=SEASON_LENGTH, alias="theta"),  
1005     ],  
1006     freq="MS",  
1007     n_jobs=-1,  
1008     fallback_model=HistoricAverage(),  
1009 )
```

`SEASON_LENGTH` is read from `config.py` rather than hard-coded here; the rationale for setting it to 12 is given in Section 5.6.1 and is not repeated here. `fallback_model=HistoricAverage()` is a safety net: across more than 30,000 series, some are too short, too sparse, or too irregular for ETS or Theta to fit successfully, and rather than letting the whole run fail on one such series, that series is silently given the

HistoricAverage forecast instead. How often this actually happens is measured by the fallback-rate check in Section 6.7.

Two separate forecasts are produced from this configuration. The test forecast is trained on the training period only and scored against the withheld test-period actuals, over a 12-month horizon (2015-01 to 2016-01). The live forecast is trained on the training and test periods combined and produces a 7-month horizon (2016-01 to 2016-07) with no ground truth to check against, consistent with the live-window definition in Section 5.3.1.

6.2. Clipping Negative Base Forecasts

Every base forecast column is clipped at zero, for both the test and live forecast tables, immediately after fitting and before any reconciliation runs:

```
1000 for _df in (pred_test, pred_live):
1001     for _col in ["histavg", "ets", "theta"]:
1002         _n_clipped = (_df[_col] < 0).sum()
1003         _df[_col] = _df[_col].clip(lower=0)
```

The fairness argument for why this step exists – so that BottomUpSparse and MinTraceSparse start from the same, already-nonnegative base forecasts rather than one implicitly benefiting from a nonnegativity floor the other does not have – is given in Section 5.6.2 and is not repeated here. HistoricAverage, being a simple mean of nonnegative sales figures, never produces a negative value and is consequently never clipped. On the test set, 2,300 ETS values and 16,345 Theta values are clipped; on the live set, the corresponding counts are 712 and 6,752. The larger Theta counts, on both horizons, follow directly from the method’s long-term trend line, which is more willing than ETS’s exponential smoothing to extrapolate a downward trend through zero on a declining series.

6.3. Reconciliation Implementation

A single `HierarchicalReconciliation` instance applies all three reconciliation methods together:

```
1000 reconciler = HierarchicalReconciliation(reconcilers=[
1001     BottomUpSparse(),
1002     MinTraceSparse(method="wls_struct", nonnegative=True),
1003     TopDownSparse(method="proportion_averages"),
1004     ])
```

Each reconciler is applied twice – once to the clipped test-period base forecasts, once to the clipped live-period base forecasts – using the summing matrix `S_df` and the level lookup `tags` built once when the hierarchy was constructed (Section 5.5); neither is rebuilt per forecast horizon, since the item universe was already fixed to a single, training-based set before the hierarchy was built. The historical data passed to each call differs by horizon: the test-period reconciliation uses the training rows alone, while the live-period reconciliation uses the training and test rows combined, mirroring exactly which rows each forecast itself was fitted on. `MinTraceSparse`’s own `nonnegative=True` flag is kept even though every base forecast fed into it has already been clipped at zero in Section 6.2; with that clipping in place, the flag no longer carries the burden of enforcing nonnegativity on its own, and instead serves as a belt-and-braces guard against `MinTrace`’s variance-minimising adjustment itself reintroducing a small negative value during reconciliation.

6.4. Combining Results and Attaching Ground Truth

The test and live forecasts are stacked into one table each – one for the three base models, one for the nine reconciled model/reconciler combinations – and labelled by which period each row came from:

```

1000 pred = pd.concat(
1001     [pred_test.assign(**{cat_label_col: "test"}),
1002     pred_live.assign(**{cat_label_col: "live"})],
1003     ignore_index=True,
1004 )
1005 rec_fc = pd.concat(
1006     [rec_test.assign(**{cat_label_col: "test"}),
1007     rec_live.assign(**{cat_label_col: "live"})],
1008     ignore_index=True,
1009 )
1010 assert rec_fc.shape[0] == pred.shape[0], "Shape mismatch
    between rec_fc and pred."

```

Both stacked tables come out at 576,726 rows, and the shape-integrity assertion above confirms they match row-for-row. Ground truth is then attached by merging in the actual sales values directly from `Y_df` – the same table the hierarchy was built from in Section 5.5 – using an exact `(unique_id, date)` match with `validate="1:1"`, so that a silent duplicate join would raise rather than pass unnoticed. A `fillna(0)` is applied afterwards as a safety net only; because every `(id, date)` pair in the forecast tables originated from `Y_df` in the first place, the merge is expected to leave no row without a matching actual, and the pipeline

confirms this: 0 missing ground-truth rows after the merge, against an expectation of 0.

6.5. Developer Documentation: Pipeline Flow

Figure 6.1 lays out the full pipeline end to end, from loading the raw M5 files through to exporting the dashboard’s Parquet files, as a single reference diagram for anyone picking up the codebase. Each box corresponds to one stage of the notebook described across this chapter and Chapter 5: the data-preparation and hierarchy-construction boxes on the left implement the cleaning decisions and hierarchy design covered there, while the model-fitting, clipping, reconciliation, and combination boxes on the right implement the sections above.

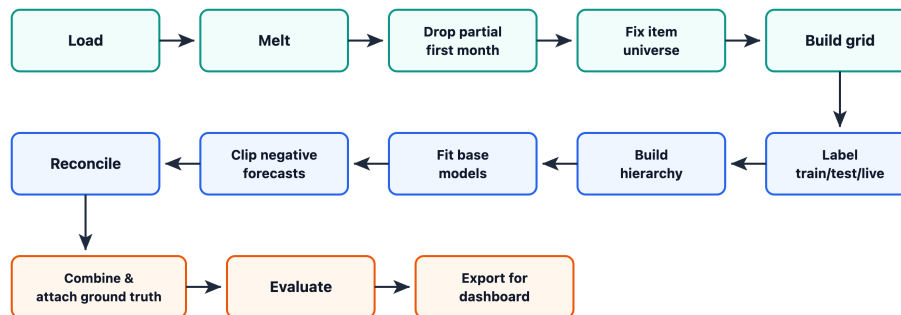


Figure 6.1.: The full pipeline, from loading the raw M5 data to exporting the dashboard’s Parquet files.

No static exported plot from the pipeline notebook itself is included alongside this diagram: the notebook’s one visualisation step calls an interactive Plotly helper to spot-check a handful of reconciled series, and produces no saved image file to draw on here.

6.6. Naming, Parameter, and Error-Handling Conventions

Three conventions run through the pipeline code shown above and are worth stating explicitly, since they are what keeps the notebook and the Streamlit dashboard from silently drifting apart as either one changes.

Naming. `config.py` is the single naming authority for every column and identifier used in this chapter: the hierarchy columns (`COL_TOTAL` through `COL_ITEM`), the aggregation `SPEC` passed to `aggregate()`, the series identifier `ID_COL`, and the period label column `CAT_LABEL_COL` are

all read from `cfg` rather than typed out again in the pipeline notebook. Nothing in this chapter hard-codes a column name.

Parameter handling. The three time-window boundaries (the train/test split date, the forecast start date, and the forecast end date) and `SEASON_LENGTH` are likewise centralised in `config.py`, so that the 12-month test horizon and 7-month live horizon computed in Section 6.1 are always derived from the same dates the dashboard uses to shade its train/test/live regions, rather than a second, independently maintained copy of those dates.

Error handling. `fallback_model=HistoricAverage()` (Section 6.1) is the pipeline's only error-handling mechanism for model fitting: rather than raising on a series ETS or Theta cannot fit, the pipeline substitutes the fallback forecast for that series alone and continues. The fallback-rate check in Section 6.7 finds this happens for 699 of the 30,354 series in the hierarchy (2.30%), identically for both ETS and Theta.

6.7. Testing & Validation

Before any of the accuracy figures in Chapter 9 are treated as meaningful, the pipeline runs three checks against its own output, plus the shape-integrity assertion already shown in Section 6.4.

Coherence check. Actual sales satisfy $y_t = Sb_t$ exactly at every time step (Section 3.2), and `HistoricAverage` forecasts each series as a plain average of its own past values (Section 3.1.1). Averaging both sides of that identity over time shows `HistoricAverage`'s own base forecasts already satisfy the same relationship, $\bar{y} = S\bar{b}$, without any reconciliation step at all: its forecasts are coherent by construction, not by correction. Bottom-Up's reconciliation matrix, $G = [0 \mid I]$ (Section 3.3.1), discards every base forecast above the bottom level and rebuilds the rest through S – exactly the identity `HistoricAverage`'s forecasts already satisfy – so applying Bottom-Up to them should barely change anything. Comparing `HistoricAverage`'s base forecast against its Bottom-Up-reconciled counterpart, matched on `(unique_id, date)`, gives a maximum absolute difference of 0.146244 – consistent with the near-zero result this check is designed to confirm, with the small residual attributable to floating-point accumulation across tens of thousands of summed series rather than to a broken merge or reconciliation step.

Top-Down root-invariance check. With a single "Total" root in place, and Top-Down's G matrix constructed so that only the top-level base forecast is ever read and every other base forecast is discarded entirely (Section 3.3.2), the reconciled Total forecast is definitionally the same number the base model already produced at that node – Top-Down should never alter the top-level forecast, only redistribute it downward.

Comparing ETS's base and Top-Down-reconciled forecasts at the "Total" node gives a maximum absolute difference of 0.000000, confirming the root fix from Section 5.4.4 holds and that the reconciled root has not drifted from the base forecast it should equal exactly.

Fallback-rate check. Counting, per series rather than per row, how often ETS or Theta produced exactly the *HistoricAverage* forecast at every horizon gives the 2.30% figure already stated in Section 6.6: 699 of 30,354 series for ETS, and the identical 699 of 30,354 for Theta.

Table 6.1 summarises all three checks against what each one is designed to confirm.

Table 6.1.: Summary of the pipeline's built-in validation checks.

Check	Confirms	Result
Coherence	<i>HistoricAverage</i> is coherent by construction; Bottom-Up should barely change it	max diff 0.146244
Top-Down root-invariance	The Total forecast is never altered by Top-Down, only redistributed	max diff 0.000000
Fallback rate	How often ETS/Theta fall back to <i>HistoricAverage</i>	699 / 30,354 (2.30%)

All three checks pass as designed, and the shape-integrity assertion from Section 6.4 raises no error, so the 576,726-row combined forecast table carried into Chapter 9 is taken to be trustworthy. The business implications of this test protocol and of the 2.30% fallback rate are discussed alongside the accuracy results in Chapter 9.

Part VI.

Development to Deployment

7. Development to Deployment

This chapter turns from the model fitting and reconciliation implementation described in Chapter 6 to how that implementation is organised as a codebase: the single configuration module shared by both notebooks and the dashboard (Section 7.1), what is required to reproduce the reported results exactly (Section 7.2), and the layout of the repository that holds all of this together (Section 7.3). The dashboard that consumes the artifacts this codebase produces is presented in Chapter 8.

7.1. Code/config.py as the Single Source of Truth

Three separate entry points read from the same underlying definitions: the exploratory notebook, the forecasting pipeline, and the deployed dashboard. `01_eda.ipynb` and `02_pipeline.ipynb` both live in `Code/notebooks/` and import `Code/config.py` by appending its parent directory to `sys.path` before importing it as `cfg`; `app/streamlit_app.py` does the same, appending its own parent's parent directory so the import resolves regardless of the working directory the app is launched from – a detail that matters because Streamlit Community Cloud always sets the working directory to the repository root, not to `app/`. Figure 7.1 shows the resulting relationship: `config.py` sits at the centre, and all three consumers read from it rather than from one another.

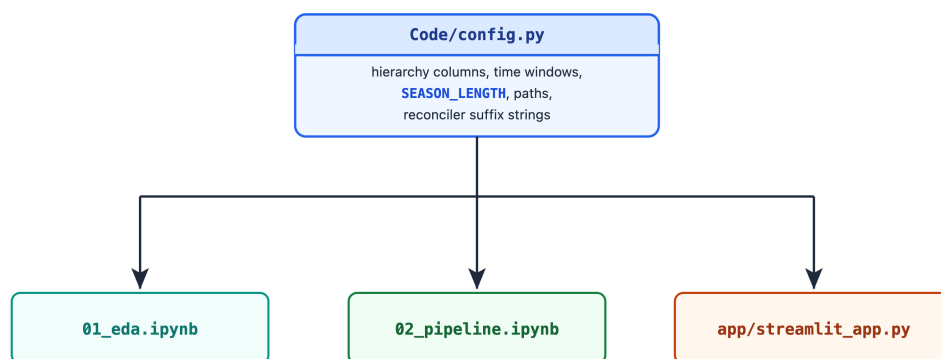


Figure 7.1.: `Code/config.py` as the single configuration module feeding both notebooks and the Streamlit app.

`config.py` centralises everything that would otherwise have to be kept in sync by hand across these three files: the column names used throughout (`DATE_COL`, `TARGET_COL`, `ID_COL`, `CAT_LABEL_COL`), the six hierarchy columns from `COL_TOTAL` down to `COL_ITEM` together with the `SPEC` list that `hierarchicalforecast`'s `aggregate()` function uses to build the hierarchy, the three time-window boundaries (`TRAIN_TEST_SPLIT_DATE`, `FORECAST_START_DATE`, `FORECAST_END_DATE`), the seasonal period `SEASON_LENGTH`, the volume-bucket `PERCENTILES`, the data and cache directory paths, and the mapping from each reconciler to the column-suffix it produces (e.g. `BottomUpSparse`, `MinTraceSparse_method-wls_struct_nonnegative-True`). A representative import looks the same wherever it appears:

```

1000 import sys
      from pathlib import Path
1002 sys.path.append(str(Path(".").resolve()))
      import config as cfg

```

The value of this is that the dashboard's own hierarchy-navigation logic – which builds node labels by joining the same column names with `config.LEVEL_SEP` – and the pipeline's aggregation step both read the same `SPEC` and the same column constants, so a change to the hierarchy definition (adding a level, renaming a column) only has to be made once, in one file, rather than tracked down separately in a notebook and an app. This is the "Don't Repeat Yourself" (DRY) principle applied to configuration rather than to code: every piece of shared knowledge has a single, authoritative representation, so no other file can silently drift out of sync with it [HT99]. `Code/README.md` documents this convention for anyone extending the codebase.

7.2. Reproducibility

Every result reported in this thesis was produced by running `02_pipeline.ipynb` top-to-bottom, once, in a dedicated `m5_forecast` environment, rather than by executing individual cells out of order. This matters beyond general good practice: it is precisely the first of the standard rules for reproducible computational research, keeping track of exactly how every result was produced rather than relying on an undocumented sequence of manual steps [San+13]. Several steps in the pipeline depend on state built by earlier cells, most notably the item-universe fix, which removes items with no training-period sales before the hierarchy, the base models, and every downstream evaluation are built – re-running only a later cell after changing an earlier one would silently leave stale, larger item and forecast tables in memory. The documented rule is accordingly

to re-run the whole notebook from the top whenever the underlying data, `config.py`, or any modelling step changes.

Two further conventions support reproducing these results exactly. First, the raw M5 CSV files (`sales_train_validation.csv`, 30,490 rows of one item-store series each, and `calendar.csv`) are cached to Parquet the first time they are loaded, and every subsequent run reads the cached Parquet file instead of re-parsing the CSV. This is purely a performance measure – Parquet is a columnar binary format that reads back faster than a CSV and preserves column types, so dates remain dates rather than being re-parsed from strings on every run – and it has no effect on the values produced, since the cache is a lossless copy of the same CSV. This raw cache lives under `Code/artifacts/raw_cache/` and is excluded from version control, since it is fully regenerable from the Kaggle source files. Second, the pipeline environment itself is version-pinned, following the same guidelines’ third rule of archiving the exact versions of every external program a result depends on [San+13]: `requirements-pipeline.txt` fixes the exact versions of every package the notebooks depend on (for example `pandas 2.3.3`, `numpy 2.2.6`, `statsforecast 2.1.1`, `hierarchicalforecast 1.5.1`, `utilsforecast 0.2.16`, and `pyarrow 24.0.0`), so that re-creating the `m5_forecast` environment from this file reproduces the same library behaviour rather than whatever happens to be the current release of each package at some later date.

The four small Parquet files the pipeline exports for the dashboard (Section 7.1 lists the shared paths that govern where these are written) are treated differently from the raw cache: at a few megabytes each, they are committed to the repository directly, so the deployed dashboard has data to read without needing to re-run the pipeline in the deployment environment at all – a point taken up in Chapter 8.

7.3. Repo Layout and Separation of Concerns

The repository is organised so that each of the three consumers described in Section 7.1 owns only the part of the codebase specific to it, with `config.py` as the one file all three share:

- `Code/config.py` – the shared configuration module described above.
- `Code/notebooks/` – `01_eda.ipynb` and `02_pipeline.ipynb`, the exploratory analysis and the forecasting pipeline.
- `Code/app/` – `streamlit_app.py` and its own `requirements.txt`, the minimal package list Streamlit Community Cloud actually installs at deployment time.

- **Code/artifacts/dashboard_data/** – the four committed Parquet files the app reads.
- **Code/artifacts/raw_cache/** – the gitignored Parquet cache of the raw Kaggle CSVs, described in Section 7.2.
- **Code/data/** – a **README.md** explaining where to place the Kaggle competition files, plus **Code/data/m5/**, itself gitignored since the competition data cannot be redistributed.
- **Code/requirements-pipeline.txt** – the pinned local research environment described in Section 7.2, distinct from **Code/app/requirements.txt**.
- **.streamlit/config.toml** – kept at the true repository root rather than nested under **Code/** or **Code/app/**, because Streamlit Community Cloud only looks for this file at the root of the deployed repository.

The notebooks themselves are kept fully self-contained, with all logic written inline rather than factored out into an importable **src/** package. Given that the codebase consists of exactly two notebooks feeding one dashboard script, introducing a separate package layer would add a level of indirection without a corresponding benefit; **config.py** already serves as the only module genuinely shared across more than one file, and nothing else in the pipeline is reused elsewhere. This separation is also what keeps the app itself simple: as its own module docstring states, **streamlit_app.py** reads exclusively from the four Parquet files in **Code/artifacts/dashboard_data/** and never re-runs the pipeline itself, since Streamlit Community Cloud only ever executes that one file.

Part VII.

Application Deployment

8. Application Deployment

This chapter covers the deployed Streamlit dashboard itself: the Parquet data contract between the pipeline and the app (Section 8.1), the cascading dropdown controls used to navigate the hierarchy (Section 8.2), the design of the forecast chart (Section 8.3), how the app senses the browser’s colour theme (Section 8.4), and how the app is deployed and kept in step with the pipeline’s output (Sections 8.5 and 8.6). The pipeline that produces the data read here was implemented in Chapter 6; `config.py`, the single source of truth shared between the notebooks and the app, was introduced in Chapter 7.

8.1. Data Contracts

The dashboard never re-runs the pipeline itself – Streamlit Community Cloud only ever executes `app/streamlit_app.py` – so the entire interface between the pipeline and the app is file-based. `02_pipeline.ipynb`’s final step exports four Parquet files into `Code/artifacts/dashboard_data/`, and the app’s `load_data()` function reads all four back in on startup:

- `hierarchy.parquet` – the distinct state/store/category/department/item combinations, with duplicates dropped. This is the lookup table the cascading dropdowns filter against (Section 8.2); it carries no dates or sales figures.
- `actuals.parquet` – the full actual sales history for every hierarchy node, keyed by the series identifier, the date, the sales value, and the train/test/live label produced during data preparation.
- `pred_with_gt.parquet` – the base model forecasts, already restricted to the forecast horizon rather than carrying the full training history.
- `rec_with_gt.parquet` – the reconciled forecasts for the same horizon, with ground truth already attached.

Restricting the two forecast files to the horizon, rather than shipping the training-period fitted values as well, keeps the files small enough to commit to the repository alongside the code, in contrast to the gitignored

raw data cache. The app wraps `load_data()` in `@st.cache_data`, so the four files are read once per session rather than on every dropdown interaction.

8.2. Cascading Dropdown Interactivity

The sidebar exposes one dropdown per hierarchy level – State, Store, Category, Department, Item – each populated from `hierarchy.parquet` and filtered by every selection made above it: choosing a state restricts the store options to that state’s stores, choosing a store restricts the category options to categories sold in that store, and so on down to Item. Every dropdown carries an explicit `"(All)"` option, and leaving a level on `"(All)"` stops all levels below it from narrowing further.

These selections are combined into the hierarchy node’s identifier by walking State through Item and stopping at the first level left on `"(All)"`, which mirrors how the pipeline’s aggregation spec names its own nodes (`Total`, `Total/CA`, `Total/CA/CA_1`, and so on down to the bottom level) – the dropdown ordering is not an independent interface decision, it is the same six-level `Total/State/Store/Category/Department/Item` structure defined in Section 5.5. Leaving every dropdown on `"(All)"` therefore resolves to `Total` itself, the grand total series. Because Streamlit’s `selectbox` widgets are searchable by default, a user can also jump straight to a specific store or item by typing rather than scrolling. Alongside the hierarchy controls, a radio button selects the base model and a multiselect chooses one or more reconciliation methods to plot at once; both draw their option lists from `config.py` so they can never drift out of sync with what the pipeline actually produced. Figure 8.1 shows the sidebar mid-drill-down, with the higher levels resolved to a specific state and store and the remaining levels still open.

Hierarchy

State

CA

Store

CA_2

Category

HOBBIES

Department

(All)

Item

(All)

Forecast

Base model

☐ histavg ☒ ets ☐ theta

Reconciliation method(s)

MinTrace (wls_struct) ×

Bottom-Up × Top-Down × ×

Figure 8.1.: The sidebar’s cascading dropdown controls, partway through drilling down the hierarchy.

8.3. Chart Design

The forecast chart is built with Plotly and always shows three kinds of trace for the currently selected hierarchy node: the actual sales history, the chosen base forecast, and the chosen reconciled forecast(s). The actual line is drawn only over the train and test periods; the live window is excluded from it entirely, since live-period rows have no real ground truth to plot as if they were observed sales (Section 5.3.1). The base forecast is drawn as a dotted line and the reconciled forecast(s) as solid lines, so that even with several reconciliation methods selected at once the base model remains visually distinct from all of them.

Train, test, and live regions are shaded across the full width of the chart using Plotly’s `add_vrect`, each with its own fill colour and an annotation label, so the three phases are visible at a glance without needing to read axis dates closely – the same three windows laid out schematically in Figure 5.3, rendered here interactively for whichever specific series and horizon the user has selected. The chart’s background is left fully transparent and its colour palette – line colour, grid colour, font colour, and shading opacity – is chosen at render time depending on whether the surrounding app is in light or dark mode (Section 8.4), so the chart never clashes with the rest of the page. The legend is placed horizontally above the plot area rather than beside it, keeping the full chart width available for the time series itself. Figure 8.2 shows the resulting chart for the same hierarchy node in both palettes: the shaded train/test/live regions and the dotted base-forecast line are visible in either mode, with only the colour choices differing between them.

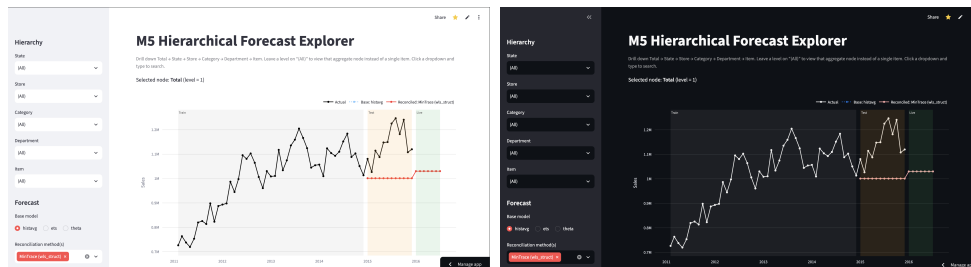


Figure 8.2.: The forecast chart in light mode (left) and dark mode (right), showing the shaded train/test/live regions.

8.4. Theme Sensing

Rather than adding a separate light/dark toggle to the app’s own interface, the dashboard reads Streamlit’s actual active theme directly from the browser via the `st_theme()` call [tem26]. This package is distributed on PyPI as `st-theme` and imported as `streamlit_theme` – a naming

mismatch worth stating explicitly, since `streamlit-theme` is in fact a separate, unrelated PyPI package for setting a custom theme rather than reading the active one; `app/requirements.txt` documents this distinction to prevent the wrong package being installed by name alone. `st_theme()` itself reflects whatever the user has set at the platform level, including a "use system setting" choice. The result is cached in `st.session_state` the first time it is available, and a single boolean derived from it decides which colour palette the chart in Section 8.3 uses. This keeps the dashboard's own theme behaviour consistent with the rest of the user's Streamlit session, without asking them to set the same preference twice.

8.5. Deployment on Streamlit Community Cloud

Because Streamlit Community Cloud only ever executes `app/streamlit_app.py` and never re-runs the notebooks, the four Parquet files described in Section 8.1 have to already exist in the repository at deploy time; the app has no way to regenerate them itself. The app imports `config.py` by explicitly appending the repository root to `sys.path` before importing it, which is necessary because Community Cloud always sets the working directory to the repository root rather than to the `app/` folder the script itself lives in – a relative import that works when run locally from `app/` would otherwise fail on the deployed service. The dependencies actually installed on Community Cloud come from `app/requirements.txt`, a separate, smaller manifest than the `requirements-pipeline.txt` used to run the notebooks locally (Chapter 7), since the deployed app needs Streamlit, Plotly, and `st-theme` (Section 8.4) but none of the modelling libraries the pipeline depends on. The same script can also be run locally with `streamlit run app/streamlit_app.py` against a local checkout of the same Parquet files, which is how the app is checked before every deploy.

8.6. Deployment Flow

Figure 8.3 traces the full path from a local pipeline run to a served dashboard. Running `02_pipeline.ipynb` writes the four Parquet files into `Code/artifacts/dashboard_data/`; committing and pushing that folder to GitHub is what makes updated forecasts available to the deployed app, since Community Cloud has no other way to see new data. A push to the connected repository triggers Community Cloud to rebuild the app, which then boots, loads the Parquet files through `config.py`'s paths

exactly as a local run would, and serves the resulting dashboard to the browser. Because this whole chain is triggered by a Git push rather than a scheduled job, the dashboard reflects whatever was last pushed, not necessarily the most recently run pipeline – re-running the pipeline without pushing the updated Parquet files leaves the deployed dashboard unchanged.

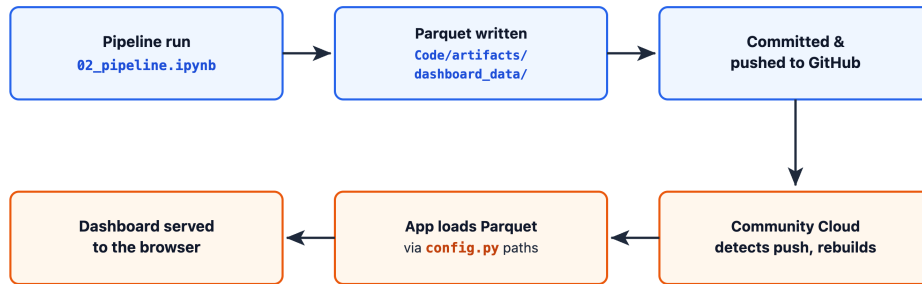


Figure 8.3.: The deployment flow from a local pipeline run to the served dashboard.

Part VIII.

**Evaluation - Validation -
Conclusion**

9. Evaluation – Validation – Conclusion

This chapter reports the accuracy results the design decisions of Chapter 5 and the implementation of Chapter 6 were built toward: sMAPE broken out by hierarchy level (Section 9.1) and by business volume segment (Section 9.3), together with four patterns that emerge from those two slices – the Bottom-Up surprise, the high-volume baseline surprise, the Top-Down advantage on high-volume series, and the long-tail reversal. Section 9.7 reports results against the testing protocol defined in Section 6.7. Sections 9.8–9.10 answer the sub-questions and the primary research question set out in Chapter 1, draw out practical recommendations for business forecasting practice, and revisit the limitations flagged in Section 5.8. Section 9.11 closes with a monitoring and operationalisation outlook for the dashboard introduced in Chapter 8. All figures below are computed on the test period only, at the point where actual outcomes are known and were withheld from model fitting (Section 5.3.1). These sMAPE figures are not directly comparable to the M5 competition’s own official leaderboard rankings, which score submissions on price-weighted RMSSE (WRMSSE) rather than sMAPE – a deliberate choice, not an oversight (Section 5.7.1).

9.1. Accuracy by Hierarchy Level

Table 9.1 reports sMAPE for the three unreconciled base models at each of the six hierarchy levels. Two patterns hold across all three models: accuracy degrades monotonically from Total down to Item, and the degradation is not gradual – it is largest by far in the final step, from Department to Item. HistoricAverage’s error roughly triples between these two levels (13.07% to 40.91%), and ETS’s and Theta’s error roughly quintuples. This mirrors the intermittency pattern established in Section 2.3 and quantified in Section 5.4: item-level series are sparse and irregular in a way that state- and store-level totals, smoothed by aggregation across many items, are not.

Between models, Theta is the most accurate base forecaster at every level except Department, where ETS is marginally ahead (7.88% vs. 8.26%); HistoricAverage is the least accurate base model at every

Table 9.1.: sMAPE (%) by hierarchy level, base forecasts, test period.

Level	Historic Average	ETS	Theta
Total	6.58	5.53	4.24
State	6.85	4.88	4.32
Store	8.19	5.95	5.35
Category	11.32	7.31	6.73
Department	13.07	7.88	8.26
Item	40.91	37.72	36.32

level, although its gap narrows in relative terms at Item (40.91% against Theta’s 36.32%), where intermittency dominates and seasonality is harder to exploit. This is consistent with the seasonal pattern established in Section 5.6.1: since `HistoricAverage` cannot represent the annual seasonality that `season_length=12` lets ETS and Theta exploit, it is structurally disadvantaged wherever seasonality is present, and most clearly so at the smoother aggregate levels.

Reconciliation changes this picture substantially for ETS and Theta, and barely at all for `HistoricAverage`. Table 9.2 reports reconciled sMAPE for ETS and Theta under all three reconciliation methods (`HistoricAverage` is omitted here because, being already linear and therefore already coherent, `Bottom-Up` and `Top-Down` leave its values unchanged to two decimal places at every level, as Section 9.7’s coherence check confirms; `MinTrace` does the same on every level above Item, and only at Item does it raise the error from 40.91% to 41.76% – see Table 10.1 in the appendix).

9.2. The Bottom-Up Surprise

Table 9.2 shows a largely consistent ranking at every level above Item, for both base models: `Bottom-Up` is the most accurate reconciler, `MinTrace` is usually second, and `Top-Down` is usually the least accurate. The one exception is Theta at the Total level, where `Top-Down` (4.24%) edges out `MinTrace` (4.36%) – unsurprisingly, since `Top-Down` reproduces the base forecast at the root, and Theta’s base Total forecast is already the most accurate of all base forecasts at that level. At the Total level, for example, ETS’s `Bottom-Up` forecast reaches 3.94% sMAPE against `MinTrace`’s 5.19% and the unreconciled base forecast’s own 5.53% – `MinTrace`’s theoretically motivated cross-level adjustment leaves the series worse off than simply summing the item-level forecasts upward. The gap between `Bottom-Up` and `MinTrace` is largest at the top of the hierarchy and narrows moving down it: for ETS it shrinks from 1.25 percentage points

Table 9.2.: Reconciled sMAPE (%) by hierarchy level, ETS and Theta, test period.

Base model	Level	Bottom-Up	MinTrace	Top-Down
ETS	Total	3.94	5.19	5.53
	State	3.97	5.11	5.88
	Store	4.77	5.58	7.60
	Category	6.22	7.19	10.63
	Department	7.91	8.00	12.38
	Item	37.72	40.62	40.74
Theta	Total	3.78	4.36	4.24
	State	3.76	4.36	4.77
	Store	4.86	5.25	6.97
	Category	6.18	6.52	9.75
	Department	7.60	7.98	11.49
	Item	36.32	39.24	40.54

at Total to 1.14 at State, 0.81 at Store, 0.97 at Category and only 0.09 at Department, while for Theta it stays between roughly 0.35 and 0.6 points throughout. At Item the gap reopens (2.9 points for both models), because Bottom-Up is there definitionally identical to the unreconciled item-level forecast, while MinTrace’s non-negative adjustment moves many item-level forecasts away from it.

This runs against the usual framing of MinTrace as the theoretically optimal reconciler, in the specific sense proved in Section 3.3.3 [WAH19], so it is worth setting out two candidate explanations rather than treating MinTrace’s superiority as a given. First, the `wls_struct` weighting scheme used here – $W = \text{diag}(S\mathbf{1})$, as derived in Section 3.3.3 and adopted here for the reasons given in Section 5.8 – scales each node purely by how many bottom-level series sum into it, not by an estimate of that node’s own forecast-error variance – a coarser signal than the shrinkage-based covariance estimator a full MinTrace implementation would use, and one that gives the method less information to act on than its reputation assumes. Second, the upper levels of this hierarchy are already comparatively stable and non-intermittent, as established in Section 2.3 and Section 5.4: with little genuine incoherence left for Bottom-Up’s plain summation to correct, MinTrace’s cross-level adjustment has more room to introduce noise than to remove it. Both explanations point the same way – toward `wls_struct` specifically, rather than reconciliation in general, being the weaker link – which is why Section 5.8 flags a shrinkage-based MinTrace variant as future work rather than concluding that Bottom-Up is simply the better method.

9.3. Accuracy by Volume Segment

The volume-percentile framework defined in Section 5.7.2 ranks the 30,240 bottom-level (item-store) series by training-period sales volume and buckets them by cumulative share of that volume. Figure 9.1 shows this concentration directly: plotting the cumulative share of training-period volume against the cumulative share of series, ranked biggest seller first, produces a curve far steeper than the perfectly-even diagonal it is plotted against. Table 9.3 restates the same concentration numerically: the top 5% of training volume comes from only 29 series (0.10% of all series), and even the top 80% of volume comes from just 8,155 series (26.97% of all series) – the remaining roughly three-quarters of all series account for only the last fifth of total volume.

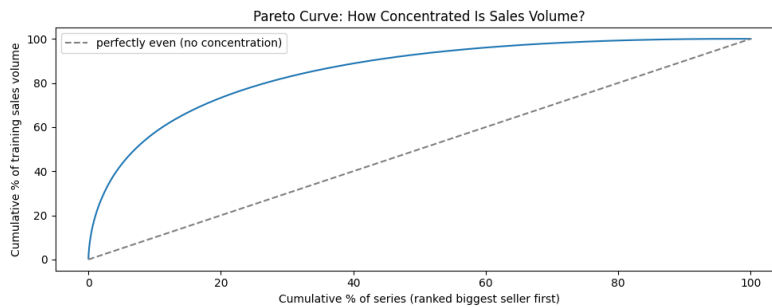


Figure 9.1.: Pareto curve of training-period sales volume: cumulative share of volume against cumulative share of bottom-level series, ranked from the largest seller. Source: `01_eda.ipynb`.

Table 9.3.: Volume-bucket sizes, ranked on training-period volume.

Top % of volume	Series in bucket	% of all 30,240 series
5	29	0.10
10	94	0.31
20	326	1.08
50	2151	7.11
80	8155	26.97
100	30,240	100.00

Table 9.4 reports base-forecast sMAPE within each bucket. Because these buckets are cumulative and nested – the top 10% bucket contains the top 5% bucket, and so on – the top 100% row is identical to the Item row of Table 9.1, confirmed to six decimal places in the pipeline’s own output (a consequence of the evaluation grid being perfectly balanced, since every series contributes exactly the same 12 zero-filled test-period rows).

Table 9.4.: sMAPE (%) by volume segment, base forecasts, test period.

Top % of volume	Historic Average	ETS	Theta
5	16.61	22.97	22.97
10	22.51	29.88	30.41
20	27.89	33.43	32.79
50	28.88	31.93	31.04
80	29.68	31.84	31.10
100	40.91	37.72	36.32

9.4. The High-Volume Baseline Surprise

Table 9.4 shows `HistoricAverage` as the most accurate base model in every bucket up to and including the top 80% of volume, and by a wide margin in the smallest, highest-volume buckets: at the top 5% of volume, `HistoricAverage` reaches 16.61% sMAPE against ETS's and Theta's 22.97%, roughly six percentage points better. Only at the top 100% bucket – once the long tail of low-volume, high-intermittency series is folded in – does this ranking invert, with Theta (36.32%) and ETS (37.72%) overtaking `HistoricAverage` (40.91%).

This is the opposite of what the seasonal argument in Section 5.6.1 might suggest on its own: ETS and Theta were configured specifically to exploit the annual seasonality visible in aggregate sales, yet on the series that carry the most sales volume, the flat, no-seasonality `HistoricAverage` baseline wins. This is not an isolated anomaly; it echoes one of the most robust findings in the forecasting literature, that simple extrapolative methods frequently match or beat statistically more sophisticated ones on real data [MH00]. A plausible reading is that individual high-volume series still carry enough month-to-month idiosyncrasy – promotions, calendar effects, one-off spikes – that a seasonal decomposition fitted to only four years of monthly training data (Section 5.3) picks up noise as if it were a repeating pattern, whereas `HistoricAverage`'s flat mean is not exposed to that risk. Whatever the mechanism, the practical implication carried into Section 9.9 is direct: "more sophisticated" is not a safe default assumption for the series a business actually cares most about.

9.5. The Top-Down Advantage on High-Volume Series

Reconciliation interacts with volume segment as strongly as it interacts with hierarchy level. Table 9.5 isolates Bottom-Up and Top-Down for ETS and Theta across the same volume buckets as Table 9.4 (MinTrace

tracks Bottom-Up closely up to the top 80% bucket and is omitted for readability; its full figures, including the larger gap at the top 100% bucket, are given in Tables 10.5 and 10.6 in the appendix).

Table 9.5.: Reconciled sMAPE (%) by volume segment: Bottom-Up vs. Top-Down.

Top % of volume	ETS		Theta	
	Bottom- Up	Top- Down	Bottom- Up	Top- Down
5	22.97	16.98	22.97	17.47
10	29.88	22.91	30.41	23.42
20	33.43	28.14	32.79	28.48
50	31.93	28.99	31.04	29.16
80	31.84	29.69	31.10	29.73
100	37.72	40.74	36.32	40.54

Figure 9.2 plots the same numbers directly: Top-Down (dashed) sits clearly below Bottom-Up (solid) across the narrow, high-volume buckets, then crosses above it by the top 100% bucket – the reversal described in Section 9.6 is visible here as the two line styles trading places, rather than only as a change of sign in a table of numbers.

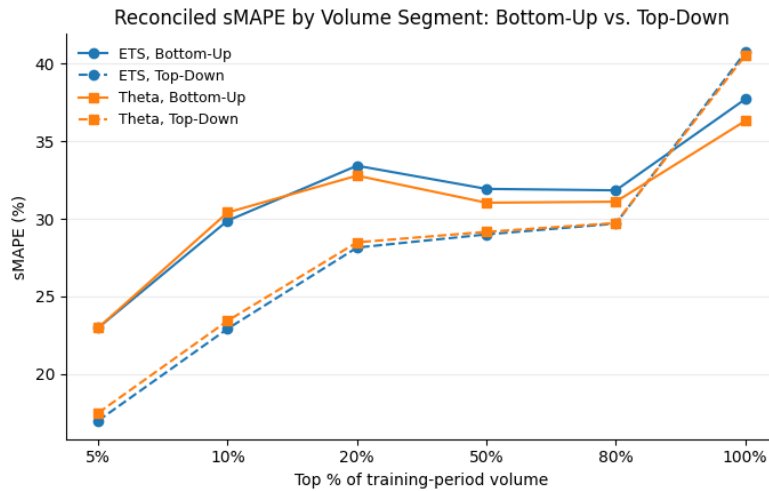


Figure 9.2.: Reconciled sMAPE by volume segment for ETS and Theta, Bottom-Up (solid) vs. Top-Down (dashed).

At the top 5% of volume, Top-Down cuts ETS's sMAPE from 22.97% (Bottom-Up) to 16.98%, and Theta's from 22.97% to 17.47% – close to HistoricAverage's own 16.61% in Table 9.4, and clearly the best reconciled result in this segment for either model. The advantage persists, though

it narrows, through the top 10%, 20%, 50%, and 80% buckets. This is the inverse of Section 9.2’s finding at the hierarchy-level view: there, Top-Down was almost uniformly the weakest reconciler; here, restricted to the series that carry the most volume, it is the strongest. Top-Down’s proportional redistribution of the Total forecast by historical volume share – the proportions vector p formalised in Section 3.3.2 – is, by construction, most reliable precisely where a series’ historical share of volume is itself a stable, representative quantity – true of high-volume series, and decreasingly true as the buckets widen to include smaller, more erratic ones.

9.6. The Long-Tail Reversal

The advantage identified in Section 9.5 does not merely shrink as the volume buckets widen – it reverses. At the top 100% bucket in Table 9.5, Top-Down is the *worst* of the two reconcilers for both base models (ETS: 40.74% vs. Bottom-Up’s 37.72%; Theta: 40.54% vs. 36.32%), even though it was the best at every narrower, higher-volume bucket. The reversal is driven by the same long tail identified in Section 9.4: the roughly three-quarters of series that carry only the last fifth of total volume (Table 9.3) have volume shares too small and too erratic for Top-Down’s proportional split to track, while Bottom-Up’s item-level fits – however imprecise individually – are not tied to any assumption about historical share at all.

Read together with Section 9.4, this reversal marks out exactly where each method earns its keep: HistoricAverage and Top-Down are strongest on the small number of series that dominate training-period sales volume, while ETS and Theta under Bottom-Up reconciliation only pull ahead once the long tail is counted. The fairness note in Section 5.9 flags the consequence directly – because the volume-percentile framework’s headline comparisons foreground high-volume performance, they structurally under-weight exactly the segment, and exactly the models, this reversal shows to be worth measuring on their own terms.

9.7. Testing & Validation Results

Table 9.6 reports results against the three checks specified in Section 6.7, together with the negative-forecast check that verifies the clipping decision motivated in Section 5.6.2.

All three checks pass. The coherence check’s max difference of 0.146244, rather than an exact 0, reflects ordinary floating-point accumulation across the reconciliation pipeline and is negligible against sMAPE values reported in whole percentage points; the Top-Down root check is exact,

Table 9.6.: Testing and validation results, against the protocol defined in Section 6.7.

Check	Result	Expected
Coherence (<i>HistoricAverage</i> vs. Bottom-Up)	max diff 0.146244	≈ 0
Top-Down root invariance (ETS, Total level)	max diff 0.000000	≈ 0
Fallback rate (ETS)	699 of 30,354 series (2.30%)	monitored
Fallback rate (Theta)	699 of 30,354 series (2.30%)	monitored
Negative-forecast check (all columns, post-clipping)	0 negatives everywhere	0

confirming the single "Total" root added in Section 5.4.4 behaves as required. ETS and Theta fall back to *HistoricAverage* on identically 699 of 30,354 series (2.30%) – the same series in both cases, since both models' fitting routines fail under the same conditions of short or irregular history. This 2.30% figure is not itself a pass/fail criterion but a baseline worth carrying into Section 9.11: a materially higher rate on a future run of this pipeline would be a signal that the item universe or data quality has shifted. The negative-forecast check confirms that clipping base forecasts before reconciliation (Section 5.6.2) succeeded: no base or reconciled column contains a negative value, where before clipping 2,300 ETS and 16,345 Theta test-period values had been negative.

9.8. Answering SQ1–SQ3 and the Primary Research Question

SQ1 – Do Bottom-Up, Top-Down, and MinTrace differ in accuracy patterns across hierarchy levels? Yes, and the difference is not a fixed ranking but one that depends on where in the hierarchy the question is asked. Section 9.2 shows Bottom-Up ahead of MinTrace and Top-Down at every level from Total to Department; Top-Down is the weakest of the three at the hierarchy-level view at every level except Theta's Total, where it simply reproduces Theta's already accurate base forecast. The three methods are not interchangeable, and no single one of them is the safe default across the whole hierarchy.

SQ2 – Does the choice of base model affect reconciliation performance across the hierarchy? Yes, substantially, but not because reconciliation reacts differently to each base model's forecasts as such – rather, because *HistoricAverage* is already linear and therefore already

coherent (Section 9.7), reconciliation changes almost nothing for it, while ETS's and Theta's forecasts are genuinely incoherent across levels and see multi-percentage-point swings under different reconcilers (Table 9.2). The base model determines not just the starting accuracy but whether reconciliation has any real work to do.

SQ3 – As the evaluation widens from the highest-volume series outward to the full population, how does the gap between methods change? It does not merely narrow – it inverts, twice over. Sections 9.4 and 9.5 show HistoricAverage and Top-Down leading on the narrowest, highest-volume buckets; Section 9.6 shows Theta, ETS, and Bottom-Up overtaking them once the long tail is included. Which combination looks best is therefore inseparable from which slice of the business the question is being asked about.

Primary research question – how do foundational hierarchical forecasting methods perform across volume segments, and what practical insights follow? No single base-model/reconciliation combination dominates across both dimensions examined in this thesis. Performance is segment-dependent along business volume (Sections 9.4–9.6) and level-dependent along the hierarchy (Sections 9.1–9.2), and the two dependencies do not point the same direction: the combination that performs best on high-volume series (HistoricAverage or Top-Down-reconciled ETS/Theta) is not the combination that performs best once the long tail is counted (Bottom-Up-reconciled ETS/Theta). Practical insights follow directly in Section 9.9.

9.9. Practical Recommendations for Business Forecasting Practice

Three recommendations follow directly from the results above, aimed at a practitioner choosing a forecasting setup for a similar retail hierarchy rather than at further model development.

First, match the method to the business question rather than adopting one combination hierarchy-wide. Where the decision concerns the small number of high-volume item-store series – the "A" items of an ABC classification, whose replenishment and stock levels carry the most business weight – HistoricAverage or a Top-Down-reconciled base model is both simpler to explain and more accurate than the alternatives (Sections 9.4–9.5). This finding concerns individual high-volume series, not the aggregate levels of the hierarchy: for headline reporting and top-line planning at the Total, State or Store level, Bottom-Up remains the more accurate reconciler and Top-Down generally the least accurate (Section 9.2). Where the audience is item-level replenishment or long-tail inventory decisions, Bottom-Up-reconciled ETS or Theta is the better-performing choice (Sec-

tion 9.6), and this is exactly the segment the fairness note in Section 5.9 warns is otherwise under-weighted by volume-percentile evaluation alone.

Second, do not treat MinTrace’s theoretical optimality as a guarantee of practical superiority without checking the specific variant in use. The `wls_struct` weighting evaluated here underperforms simple Bottom-Up summation at every hierarchy level above Item (Section 9.2); a shrinkage-based variant, flagged as future work in Section 5.8, would need to be checked on the same data before MinTrace could be recommended over Bottom-Up in this setting.

Third, do not assume a more sophisticated, seasonally aware model beats a naive baseline by default. HistoricAverage’s advantage on high-volume series (Section 9.4) is large enough – more than six percentage points in the top 5% bucket – that it should be treated as a genuine competitor rather than a mere reference point in any comparable evaluation.

9.10. Conclusion and Future Work

This thesis set out to compare foundational hierarchical forecasting methods across hierarchy levels and business volume segments on retail demand data, and to draw out what those comparisons imply for practice. The empirical picture that emerges is one of segment-dependence rather than a single winning combination: Bottom-Up reconciliation outperforms MinTrace and Top-Down above the Item level; HistoricAverage and Top-Down outperform ETS, Theta, and Bottom-Up on high-volume series; and this ranking reverses once the long tail is included. Together these findings answer SQ1–SQ3 and the primary research question as set out in Section 9.8.

The limitations set out in Section 5.8 bound how far these conclusions generalise. Most directly, the `wls_struct` MinTrace variant used throughout this thesis is not the only MinTrace variant available, and the Bottom-Up surprise reported in Section 9.2 should be re-examined against a shrinkage-based covariance estimator before concluding that MinTrace itself, rather than this specific weighting choice, underperforms. The single-dataset scope of this thesis (Section 5.3) means these patterns are demonstrated on M5’s retail hierarchy specifically, not established as general properties of hierarchical forecasting; and the monthly aggregation adopted in Section 5.4.1 trades away any intra-month granularity a business might separately need. Each of these is a natural next step rather than a gap in the study as scoped.

9.11. Monitoring & Operationalisation Outlook

The dashboard described in Chapter 8 currently serves forecasts for a live period with no ground truth to check them against (Section 5.3.1); as real sales data for that period becomes available, the same sMAPE evaluation used throughout this chapter could be re-run against it on a rolling basis, at both the hierarchy-level and volume-segment views, to confirm the patterns identified here persist rather than being artifacts of the single train/test split used in this thesis.

The 2.30% fallback rate established in Section 9.7 is a natural health metric to track over time: since it reflects how often ETS and Theta fail to fit and fall back to `HistoricAverage`, a sustained increase would indicate that the item universe, data quality, or model configuration needs attention before accuracy figures are trusted. Similarly, because Section 9.6 shows accuracy ranking to be volume-segment-dependent, any operational monitoring should track sMAPE separately by volume bucket rather than as a single hierarchy-wide average, since a stable overall figure could otherwise mask deterioration concentrated in one segment. Retraining cadence, drift detection thresholds, and extension of the comparison to a shrinkage-based `MinTrace` variant (Section 9.10) are the most direct next steps toward turning this thesis's one-off evaluation into an operational monitoring practice.

Part IX.

Appendix

10. Appendix

This chapter supports the results in Chapter 9 without repeating them: Section 10.1 gives the complete sMAPE tables that Chapter 9’s by-level and by-volume-segment comparisons are drawn from, covering every base-model/reconciler combination rather than only the figures needed to make a specific point there. Section 10.2 then reproduces the artifacts this thesis is built on in full: the two notebooks (`01_eda.ipynb` and `02_pipeline.ipynb`), `config.py`, the Streamlit dashboard, and the two separate requirements files – one for the pipeline environment, one for Streamlit Cloud deployment.

10.1. Full sMAPE Tables

The tables below report test-period sMAPE for all nine base-model/reconciler combinations produced by the pipeline (`02_pipeline.ipynb`, reproduced in full in Section 10.2.2), grouped by base model for readability. Each table’s “Base” column is the unreconciled forecast; “Bottom-Up”, “MinTrace”, and “Top-Down” are that same base model’s forecast after the corresponding reconciliation method (Section 3.3).

10.1.1. By Hierarchy Level

Table 10.1 through Table 10.3 report sMAPE at each of the six hierarchy levels defined in Section 5.5.

Table 10.1.: sMAPE (%) by hierarchy level – HistoricAverage

Level	Base	Bottom-Up	MinTrace	Top-Down
Total	6.58	6.58	6.58	6.58
State	6.85	6.85	6.85	6.85
Store	8.19	8.19	8.19	8.19
Category	11.32	11.32	11.32	11.32
Department	13.07	13.07	13.07	13.07
Item	40.91	40.91	41.76	40.91

Table 10.2.: sMAPE (%) by hierarchy level – AutoETS

Level	Base	Bottom-Up	MinTrace	Top-Down
Total	5.53	3.94	5.19	5.53
State	4.88	3.97	5.11	5.88
Store	5.95	4.77	5.58	7.60
Category	7.31	6.22	7.19	10.63
Department	7.88	7.91	8.00	12.38
Item	37.72	37.72	40.62	40.74

Table 10.3.: sMAPE (%) by hierarchy level – Theta

Level	Base	Bottom-Up	MinTrace	Top-Down
Total	4.24	3.78	4.36	4.24
State	4.32	3.76	4.36	4.77
Store	5.35	4.86	5.25	6.97
Category	6.73	6.18	6.52	9.75
Department	8.26	7.60	7.98	11.49
Item	36.32	36.32	39.24	40.54

10.1.2. By Volume Segment

Table 10.4 through Table 10.6 report sMAPE at the bottom (item $\times$ store) level only, pooled within each cumulative training-volume bucket defined in Section 5.7.2. The series counts per bucket – reproduced once here since they are identical across all three tables – are 29 (top 5%), 94 (top 10%), 326 (top 20%), 2,151 (top 50%), 8,155 (top 80%), and 30,240 (top 100%, i.e. all bottom-level series), out of 30,240 total.

Table 10.4.: sMAPE (%) by volume segment – HistoricAverage

Top % of volume	n series	Base	Bottom-Up	MinTrace	Top-Down
5	29	16.61	16.61	16.61	16.61
10	94	22.51	22.51	22.51	22.51
20	326	27.89	27.89	27.89	27.89
50	2151	28.88	28.88	28.88	28.88
80	8155	29.68	29.68	29.68	29.68
100	30,240	40.91	40.91	41.76	40.91

Table 10.5.: sMAPE (%) by volume segment – AutoETS

Top % of volume	n series	Base	Bottom- Up	MinTrace	Top- Down
5	29	22.97	22.97	23.03	16.98
10	94	29.88	29.88	30.05	22.91
20	326	33.43	33.43	33.69	28.14
50	2151	31.93	31.93	32.51	28.99
80	8155	31.84	31.84	32.55	29.69
100	30,240	37.72	37.72	40.62	40.74

Table 10.6.: sMAPE (%) by volume segment – Theta

Top % of volume	n series	Base	Bottom- Up	MinTrace	Top- Down
5	29	22.97	22.97	23.01	17.47
10	94	30.41	30.41	30.98	23.42
20	326	32.79	32.79	34.38	28.48
50	2151	31.04	31.04	32.85	29.16
80	8155	31.10	31.10	32.87	29.73
100	30,240	36.32	36.32	39.24	40.54

Two things worth noting when reading these tables alongside Chapter 9. First, the by-level tables use a macro-average (sMAPE computed per series, then averaged), while the by-volume-segment tables pool every row in a bucket before computing sMAPE (a micro-average); the two happen to coincide at the Item level / top 100% row because the evaluation grid is perfectly balanced, but they are not the same statistic in general. Second, HistoricAverage’s Base, Bottom-Up, and Top-Down columns are identical to two decimal places at every level and bucket – expected, since HistoricAverage is already coherent before reconciliation and Top-Down never changes the Total root, both of which the pipeline’s own sanity checks (Section 10.2.2) confirm directly.

10.2. Full Code Listings

The two notebooks below are reproduced as executed – code, narrative, and output together – exported directly from Jupyter, so what appears here is the actual run these results come from rather than a re-typeset version of it. `config.py`, the dashboard, and both requirements files follow as plain listings, included directly from `Code/`.

10.2.1. EDA Notebook

`01_eda.ipynb` is the companion notebook referenced throughout Chapters 5 and 9; every cleanup decision justified there traces to the specific pipeline step it feeds.

01_eda

September 27, 2026

1 M5 Data — Exploratory Data Analysis

This notebook looks at the raw M5 data **before** any of the cleanup done in `02_pipeline.ipynb`, and shows *why* each cleanup decision was made. Every section here ends with a short note pointing to the exact step in the pipeline notebook that the finding justifies.

Run this notebook independently - it loads the same two raw CSVs the pipeline uses, and shares the same Parquet cache folder so nothing is loaded twice unnecessarily.

1.0.1 Repository layout

This notebook lives in `Code/notebooks/`. It imports shared paths from `Code/config.py` (so file locations always resolve correctly regardless of where Jupyter is launched from), and expects the raw M5 CSVs in `Code/data/m5/` (see `Code/data/README.md` for download instructions).

1.1 Contents

1. Load the raw files
2. Structure: what does one row actually mean?
3. Reshape a sample to long format, and check the date range
4. How sparse is the data? (Why aggregate to monthly?)
 - 4.5 The first month isn't a real month
5. Why does the hierarchy need a "Total" root?
6. New items: why fix the item universe using training data only?
7. Seasonality: is there a repeating yearly pattern?
8. Volume concentration: the Pareto curve
9. Sales by category and state
 - 9.5 Sales are never negative - so why does the pipeline clip forecasts?
10. Summary — EDA finding → Pipeline decision

```
[1]: import sys
from pathlib import Path
import time

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Code/config.py is the single source of truth for paths shared with
```

```

# 02_pipeline.ipynb and the Streamlit app - importing it here means this
# notebook can never silently point at a different data folder than the
# pipeline does.
sys.path.append(str(Path("..").resolve()))
import config as cfg

M5_DATA_DIR = cfg.DATA_DIR
cache_raw_dir = cfg.RAW_CACHE_DIR

plt.rcParams["figure.figsize"] = (10, 4)
print("Imports OK")

```

Imports OK

1.2 1. Load the raw files

```

[2]: cache_path = cache_raw_dir / "sales_train_validation.parquet"
if not cache_path.is_file():
    df_sales_raw = pd.read_csv(M5_DATA_DIR / "sales_train_validation.csv")
    df_sales_raw.to_parquet(cache_path)
else:
    df_sales_raw = pd.read_parquet(cache_path)

cache_path = cache_raw_dir / "calendar.parquet"
if not cache_path.is_file():
    df_calendar = pd.read_csv(M5_DATA_DIR / "calendar.csv",
    parse_dates=["date"])
    df_calendar.to_parquet(cache_path)
else:
    df_calendar = pd.read_parquet(cache_path)

print(f"Sales file : {df_sales_raw.shape[0]:,} rows x {df_sales_raw.shape[1]:,} cols")
print(f"Calendar file: {df_calendar.shape[0]:,} rows")
df_sales_raw.head()

```

Sales file : 30,490 rows x 1,919 cols

Calendar file: 1,969 rows

```

[2]:
      id      item_id  dept_id  cat_id store_id \
0  HOBBIES_1_001_CA_1_validation  HOBBIES_1_001  HOBBIES_1  HOBBIES  CA_1
1  HOBBIES_1_002_CA_1_validation  HOBBIES_1_002  HOBBIES_1  HOBBIES  CA_1
2  HOBBIES_1_003_CA_1_validation  HOBBIES_1_003  HOBBIES_1  HOBBIES  CA_1
3  HOBBIES_1_004_CA_1_validation  HOBBIES_1_004  HOBBIES_1  HOBBIES  CA_1
4  HOBBIES_1_005_CA_1_validation  HOBBIES_1_005  HOBBIES_1  HOBBIES  CA_1

state_id  d_1  d_2  d_3  d_4  ...  d_1904  d_1905  d_1906  d_1907  d_1908  \
0         CA    0    0    0    0  ...      1        3        0        1        1

```

1	CA	0	0	0	0	...	0	0	0	0	0
2	CA	0	0	0	0	...	2	1	2	1	1
3	CA	0	0	0	0	...	1	0	5	4	1
4	CA	0	0	0	0	...	2	1	1	0	1

	d_1909	d_1910	d_1911	d_1912	d_1913
0	1	3	0	1	1
1	1	0	0	0	0
2	1	0	1	1	1
3	0	1	3	7	2
4	1	2	2	2	4

[5 rows x 1919 columns]

1.3 2. Structure: what does one row actually mean?

Each row is one **item at one store**. The first six columns identify it; everything from `d_1` onward is one column per day (day 1 through day 1913), holding that item's units sold that day.

```
[3]: meta_cols = ["id", "item_id", "dept_id", "cat_id", "store_id", "state_id"]
    day_cols = [c for c in df_sales_raw.columns if c.startswith("d_")]
    print(f"Metadata columns : {meta_cols}")
    print(f"Day columns      : {day_cols[0]} ... {day_cols[-1]} ({len(day_cols):,}
    ↪days total)")

    for col in ["state_id", "store_id", "cat_id", "dept_id"]:
        print(f"\n{col}: {sorted(df_sales_raw[col].unique())}")
    print(f"\nitem_id: {df_sales_raw['item_id'].nunique():,} unique values")
```

```
Metadata columns : ['id', 'item_id', 'dept_id', 'cat_id', 'store_id',
'state_id']
```

```
Day columns      : d_1 ... d_1913 (1,913 days total)
```

```
state_id: ['CA', 'TX', 'WI']
```

```
store_id: ['CA_1', 'CA_2', 'CA_3', 'CA_4', 'TX_1', 'TX_2', 'TX_3', 'WI_1',
'WI_2', 'WI_3']
```

```
cat_id: ['FOODS', 'HOBBIES', 'HOUSEHOLD']
```

```
dept_id: ['FOODS_1', 'FOODS_2', 'FOODS_3', 'HOBBIES_1', 'HOBBIES_2',
'HOUSEHOLD_1', 'HOUSEHOLD_2']
```

```
item_id: 3,049 unique values
```

Why the hierarchy has these levels. The columns above - state, store, category, department, item - are exactly the levels chosen for the pipeline's hierarchy (plus a "Total" root added on top, discussed in Section 5). There are 3 states, 10 stores, 3 categories, and 7 departments, all nested

inside each other in a clean tree, which is what makes a “simple tree” hierarchy (as opposed to a more complex cross-product hierarchy) a natural fit here.

Pipeline step this justifies: `spec` and `hierarchy_cols`, Section 2.

1.4 3. Reshape a sample to long format, and check the date range

Reshaping the *entire* file just to check the date range would be wasteful. Melting a small sample of rows is enough to answer “what dates does this data actually cover?”

```
[4]: sample = df_sales_raw.sample(200, random_state=0)
sample_long = sample[meta_cols + day_cols].melt(
    id_vars=meta_cols, value_vars=day_cols, var_name="d", value_name="sales",
)
sample_long = sample_long.merge(df_calendar[["d", "date"]], on="d", how="left")
sample_long["date"] = pd.to_datetime(sample_long["date"])

print(f"Sales data actually covers : {sample_long['date'].min().date()} ->␣
↳{sample_long['date'].max().date()}")
print(f"Calendar file covers      : {df_calendar['date'].min().date()} ->␣
↳{df_calendar['date'].max().date()}")
```

```
Sales data actually covers : 2011-01-29 -> 2016-04-24
Calendar file covers      : 2011-01-29 -> 2016-06-19
```

Notice the calendar file extends further than the sales data - it includes dates for the competition’s evaluation period, which this project does not have actual sales figures for. Any forecast made beyond ~April 2016 has no real ground truth available in this dataset.

Pipeline step this justifies: the pipeline’s “live” window deliberately runs to July 2016 for the *dashboard’s* benefit (it needs somewhere to put forecasts), but is never scored, because there’s nothing real to score it against past April 2016. See `02_pipeline.ipynb`, Section 6.4, and use “January 2011 - April 2016 (sales data), live forecast window extended to July 2016” as the accurate description of the data range, rather than “January 2011 - June 2016”. Section 3.5 below looks more closely at the *start* of that range, since it isn’t as clean a boundary as the end.

1.5 4. How sparse is the data? (Why aggregate to monthly?)

At the daily, item-level, many products simply don’t sell every single day. This section measures exactly how common that is, using the full dataset (not just the sample), since sparsity is central to the aggregation decision.

```
[5]: print(" Melting full sales data (used for the rest of this notebook) ...")
start_time = time.time()
df_long = df_sales_raw[meta_cols + day_cols].melt(
    id_vars=meta_cols, value_vars=day_cols, var_name="d", value_name="sales",
)
df_long = df_long.merge(df_calendar[["d", "date"]], on="d", how="left")
df_long["date"] = pd.to_datetime(df_long["date"])
df_long.drop(columns=["d", "id"], inplace=True)
```

```
print(f" Done in {time.time() - start_time:.2f}s ({df_long.shape[0]:,} rows)")
```

Melting full sales data (used for the rest of this notebook) ...
Done in 33.01s (58,327,370 rows)

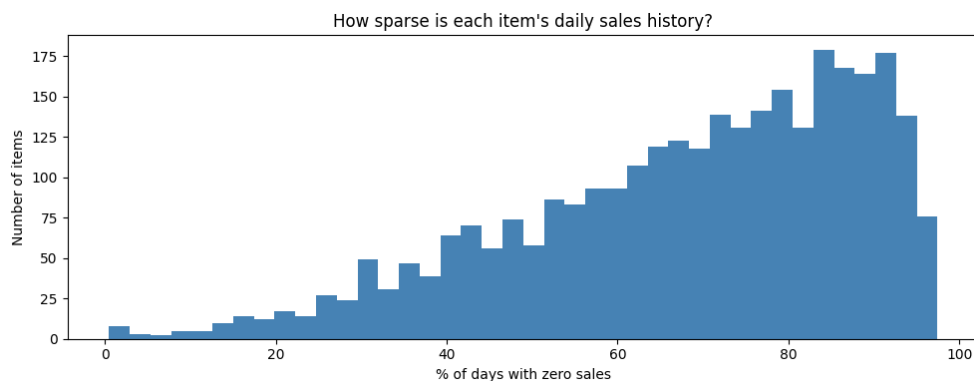
```
[6]: pct_zero_daily_item = (df_long["sales"] == 0).mean() * 100

state_daily = df_long.groupby(["state_id", "date"])["sales"].sum().reset_index()
pct_zero_daily_state = (state_daily["sales"] == 0).mean() * 100

print(f"Daily, ITEM level : {pct_zero_daily_item:5.1f}% of all (item, day)␣
      ↳ observations are zero sales")
print(f"Daily, STATE level : {pct_zero_daily_state:5.1f}% of all (state, day)␣
      ↳ observations are zero sales")
```

Daily, ITEM level : 68.2% of all (item, day) observations are zero sales
Daily, STATE level : 0.0% of all (state, day) observations are zero sales

```
[7]: fig, ax = plt.subplots()
zero_share_by_item = df_long.groupby("item_id")["sales"].apply(lambda s: (s == 0).mean() * 100)
ax.hist(zero_share_by_item, bins=40, color="steelblue")
ax.set_xlabel("% of days with zero sales")
ax.set_ylabel("Number of items")
ax.set_title("How sparse is each item's daily sales history?")
plt.tight_layout()
plt.show()
```



Most items sit far to the right - i.e. most items have zero sales on a large fraction of days, especially compared to state-level totals, which are almost never exactly zero (someone always buys *something* in an entire state on a given day). This is “intermittent demand” - a defining feature of item-level retail data, and one of the reasons item-level sMAPE (Section 6 of 02_pipeline.ipynb) is so much higher than state-level sMAPE: a handful of correct zero predictions can’t rescue an item series

with, say, only 3 non-zero days out of 60 months.

Aggregating to monthly doesn't remove this sparsity, but it substantially reduces it - a month with any sales at all is no longer zero, even if most of the days in it were.

Pipeline step this justifies: daily -> monthly aggregation, 02_pipeline.ipynb Section 6.1.

1.5.1 4.5 The first month isn't a real month

Section 3 showed sales data starts on 2011-01-29 - only 3 days before February. When the pipeline resamples daily data to monthly with `pd.Grouper(freq="MS")`, those 3 days get bucketed into a 2011-01-01 "month" alongside nothing else. That bucket isn't a partial observation of January demand, it's a heavily undercounted one - and it's the very first data point of four training Januaries the seasonal models will ever see.

```
[8]: first_month_end = pd.Timestamp("2011-02-01")
days_in_first_bucket = (df_calendar["date"] >= "2011-01-29") &
    (df_calendar["date"] < first_month_end)
n_days_first_bucket = int(days_in_first_bucket.sum())
print(f"Days actually observed in the 2011-01-01 monthly bucket:
    {n_days_first_bucket} "
      f"(a normal January has 31)")

first_month_sales = df_long.loc[df_long["date"] < first_month_end, "sales"].
    sum()
second_month_sales = df_long.loc[
    (df_long["date"] >= first_month_end) & (df_long["date"] < "2011-03-01"),
    "sales"
].sum()
print(f"Total sales in the (3-day) 2011-01 bucket : {first_month_sales:,.0f}")
print(f"Total sales in the (full) 2011-02 bucket : {second_month_sales:,.0f}")
print(f"Ratio: the first bucket holds only {first_month_sales /
    second_month_sales:.1%} "
      f"of a normal month's volume")
```

Days actually observed in the 2011-01-01 monthly bucket: 3 (a normal January has 31)

Total sales in the (3-day) 2011-01 bucket : 88,163

Total sales in the (full) 2011-02 bucket : 726,375

Ratio: the first bucket holds only 12.1% of a normal month's volume

With `season_length=12`, ETS and Theta look for a pattern that repeats every 12 points, using whatever monthly history they're given as the reference. Training runs 2011-2014, so this 3-day bucket is one of only four "Januaries" the models see - a full quarter of the seasonal reference for that month is built from a number roughly a tenth the size it should be. Dropping it is not discarding data so much as refusing to mislabel 3 days as a month.

Pipeline step this justifies: dropping the partial first month, 02_pipeline.ipynb Section 6.2.

1.6 5. Why does the hierarchy need a “Total” root?

Look at what the three states actually roll up into.

```
[9]: n_stores_per_state = df_sales_raw.groupby("state_id")["store_id"].nunique()
      print("Stores per state:")
      print(n_stores_per_state)
```

```
Stores per state:
state_id
CA      4
TX      3
WI      3
Name: store_id, dtype: int64
```

CA has 4 stores, TX and WI have 3 each. Without an explicit “Total” node sitting above State, the hierarchy is really **three separate trees** (one rooted at CA, one at TX, one at WI) with no single top. Top-Down reconciliation is only well-defined when there is one root to distribute downward from - without it, a reconciler asked to do “Top-Down” has to guess which of the three roots is “the top”, and in practice ends up picking whichever one aggregates the most series (CA, since it has the most stores), then incorrectly redistributing CA’s forecast across TX and WI as well. Adding an explicit Total node - a single constant column, “Total”, sitting above State - removes the ambiguity entirely.

Pipeline step this justifies: `col_total / spec, 02_pipeline.ipynb` Section 2. Sanity check [2] in Section 14 of that notebook confirms the fix worked, by checking that Top-Down reconciliation leaves the Total-level forecast completely unchanged.

1.7 6. New items: why fix the item universe using training data only?

Not every item was on sale for the entire 2011-2016 window - some were introduced partway through. This section finds items with **no sales at all** before the train/test split (2015-01-01).

```
[10]: monthly_check = (
        df_long.groupby(["item_id", pd.Grouper(key="date", freq="MS")])["sales"].
        ↪sum().reset_index()
    )
    train_active = monthly_check.loc[
        (monthly_check["date"] < "2015-01-01") & (monthly_check["sales"] != 0),
        ↪"item_id"
    ].unique()
    all_items = monthly_check["item_id"].unique()
    new_items = sorted(set(all_items) - set(train_active))

    print(f"Items with zero sales before 2015-01-01: {len(new_items)} of
    ↪{len(all_items):,}")
    print(f"Example items: {new_items[:5]}")

    if new_items:
        example = new_items[0]
```

```

first_sale = monthly_check.loc[
    (monthly_check["item_id"] == example) & (monthly_check["sales"] > 0),
    "date"
].min()
print(f"\nExample - '{example}' first recorded a sale in: {first_sale.
date() if pd.notna(first_sale) else 'never'}")

```

Items with zero sales before 2015-01-01: 25 of 3,049

Example items: ['FOODS_2_185', 'FOODS_2_390', 'FOODS_3_003', 'FOODS_3_038', 'FOODS_3_047']

Example - 'FOODS_2_185' first recorded a sale in: 2015-01-01

These items simply have no training history for any model to learn from - forecasting them would mean the model is really just guessing, not extrapolating from a pattern. Removing them keeps the evaluation honest: every scored forecast is for an item the models had a genuine chance to learn from. Because this filter is applied once, at the training boundary, before anything else is built, the test, live, and evaluation hierarchies all end up using the exact same item set automatically - there's no later step where they could drift apart.

Pipeline step this justifies: the item-universe fix in `02_pipeline.ipynb`, Section 6.2. The count above should match that section's printed "dropped" number.

1.8 7. Seasonality: is there a repeating yearly pattern?

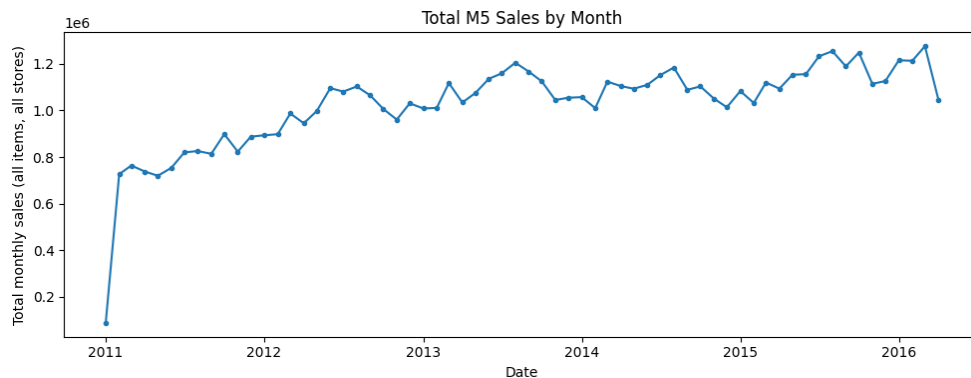
If total sales just trend up and down randomly, `season_length=12` wouldn't help the forecasting models at all. This plots total monthly sales across every item and store, to check.

```

[11]: monthly_total = df_long.groupby(pd.Grouper(key="date", freq="MS"))["sales"].
      sum()

fig, ax = plt.subplots()
ax.plot(monthly_total.index, monthly_total.values, marker="o", markersize=3)
ax.set_xlabel("Date")
ax.set_ylabel("Total monthly sales (all items, all stores)")
ax.set_title("Total M5 Sales by Month")
plt.tight_layout()
plt.show()

```



A repeating annual pattern is visible, confirming a 12-month cycle... this justifies `season_length=12`

Pipeline step this justifies: `SEASON_LENGTH = 12`, `02_pipeline.ipynb` Section 2 and the model setup in Section 8.

1.9 8. Volume concentration: the Pareto curve

Rank every item x store series by its total training-period sales, and plot the cumulative share of volume against the cumulative share of series. This is the same ranking the pipeline uses to build its “top p% of volume” buckets in Section 12 of `02_pipeline.ipynb` - here it’s just visualised directly.

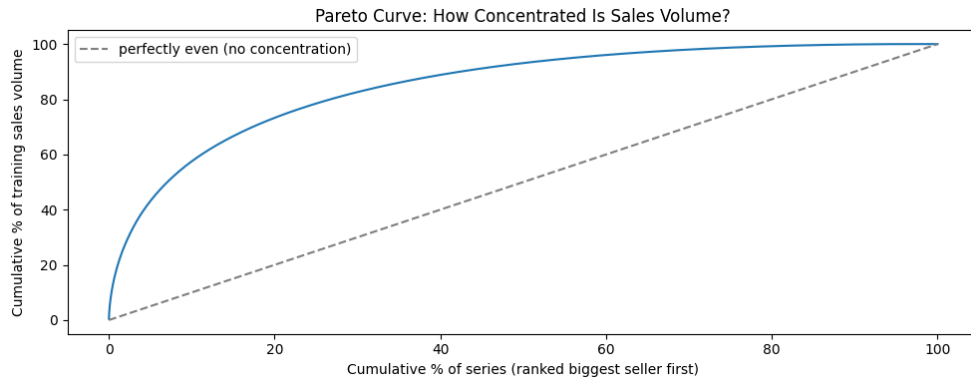
```
[12]: train_volume = (
    df_long[df_long["date"] < "2015-01-01"]
    .groupby(["item_id", "store_id"])["sales"].sum()
    .sort_values(ascending=False)
)

cum_share_volume = train_volume.cumsum() / train_volume.sum() * 100
cum_share_series = np.arange(1, len(train_volume) + 1) / len(train_volume) * 100

fig, ax = plt.subplots()
ax.plot(cum_share_series, cum_share_volume.values)
ax.plot([0, 100], [0, 100], linestyle="--", color="gray", label="perfectly even,
↳(no concentration)")
ax.set_xlabel("Cumulative % of series (ranked biggest seller first)")
ax.set_ylabel("Cumulative % of training sales volume")
ax.set_title("Pareto Curve: How Concentrated Is Sales Volume?")
ax.legend()
plt.tight_layout()
plt.show()

for p in [5, 10, 20, 50, 80]:
    n_series = (cum_share_volume <= p).sum()
```

```
print(f" The biggest {n_series:>6,} series ({n_series/len(train_volume):5.
↳2%} of all "
      f"{len(train_volume):,}) already account for {p}% of total training_
↳volume")
```



The biggest 29 series (0.10% of all 30,490) already account for 5% of total training volume

The biggest 94 series (0.31% of all 30,490) already account for 10% of total training volume

The biggest 326 series (1.07% of all 30,490) already account for 20% of total training volume

The biggest 2,151 series (7.05% of all 30,490) already account for 50% of total training volume

The biggest 8,153 series (26.74% of all 30,490) already account for 80% of total training volume

The steep curve is the point: a small number of series account for a disproportionate share of total sales, and the vast majority of series (the long tail) contribute very little individually. This is the concrete evidence behind the thesis's founding motivation - that forecast accuracy for high-volume, business-critical series matters more than treating every series equally.

Pipeline step this justifies: the volume-bucket evaluation in `02_pipeline.ipynb`, Section 12 - and explains why “top 5% of volume” ends up being a tiny number of series, not 5% of the item list.

1.10 9. Sales by category and state

A simple descriptive breakdown, useful for background context in the thesis (e.g. which category dominates revenue, whether states are comparable in scale).

```
[13]: fig, axes = plt.subplots(1, 2, figsize=(12, 4))

df_long.groupby("cat_id")["sales"].sum().sort_values(ascending=False).plot(
```

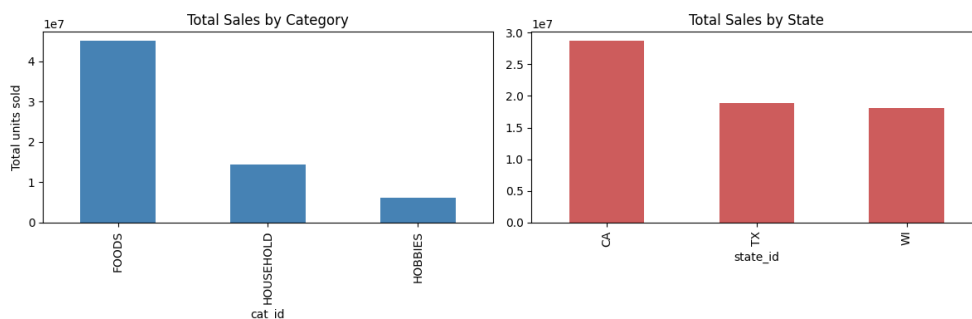
```

    kind="bar", ax=axes[0], color="steelblue"
)
axes[0].set_title("Total Sales by Category")
axes[0].set_ylabel("Total units sold")

df_long.groupby("state_id")["sales"].sum().sort_values(ascending=False).plot(
    kind="bar", ax=axes[1], color="indianred"
)
axes[1].set_title("Total Sales by State")

plt.tight_layout()
plt.show()

```



1.10.1 9.5 Sales are never negative - so why does the pipeline clip forecasts?

Every check so far has been about the shape of demand. This one is about a constraint on it: units sold cannot be negative. That sounds obvious, but it matters because the base forecasting models (ETS, Theta) have no awareness of it - they are free to extrapolate a downward trend straight through zero. This section simply confirms the raw data itself respects the constraint the models don't.

```

[14]: n_negative_raw = (df_long["sales"] < 0).sum()
      print(f"Negative values in the raw daily sales data: {n_negative_raw:,} "
            f"(of {len(df_long):,} observations)")
      print(f"Minimum observed daily sales value: {df_long['sales'].min()}")

```

```

Negative values in the raw daily sales data: 0 (of 58,327,370 observations)
Minimum observed daily sales value: 0

```

As expected, the raw data has no negative values - a day either has 0 or more units sold, nothing else. Any negative number that later shows up in a *forecast* is therefore known, with certainty, to be an artefact of the model rather than a real possibility being predicted. That's a stronger justification for clipping than "negative sales look odd" - it's "negative sales are not a value this series can take".

Pipeline step this justifies: clipping base forecasts at 0 before reconciliation,

02_pipeline.ipynb Section 8.5. (Section 13 of that notebook shows how often ETS and Theta actually produced negative values before the clip was added: 2,796 and 21,768 rows respectively.)

1.11 10. Summary: EDA Findings → Pipeline Decisions

- **Sales data ends ~April 2016; calendar extends further** → Live window described honestly as “extended beyond available data”, never scored.
- **Sales data *starts* mid-month, leaving a 3-day “January” (Section 4.5)** → Drop the partial first calendar month before modelling.
- **Item-level daily data is highly sparse (Section 4)** → Aggregate daily to monthly before modelling.
- **CA/TX/WI are three separate hierarchy roots (Section 5)** → Add an explicit “Total” root above State.
- **Some items have zero sales before 2015 (Section 6)** → Fix the item universe once, from training data only.
- **Total sales show a repeating yearly pattern (Section 7)** → Use `season_length=12` in ETS and Theta.
- **Sales volume is highly concentrated (Section 8)** → Evaluate accuracy separately by volume segment, ranked on training data only.
- **Raw sales are never negative (Section 9.5)** → Clip base forecasts at 0 before reconciliation.

Everything in 02_pipeline.ipynb traces back to one of the mappings above.

10.2.2. Pipeline Notebook

`02_pipeline.ipynb` produces the base forecasts, reconciled forecasts, and sMAPE tables reported in Section 10.1 and discussed in Chapter 9.

02_pipeline

September 20, 2026

1 M5 Hierarchical Forecasting Pipeline

This notebook forecasts monthly Walmart sales at six levels of a hierarchy — **Total** → **State** → **Store** → **Category** → **Department** → **Item** — using three simple forecasting models, three ways of making those forecasts hierarchically coherent, and evaluates the results with a single accuracy metric (sMAPE).

Read the companion notebook 01_eda.ipynb first. Every cleanup decision made here (which items to keep, how far the date range should go, why the data is aggregated to monthly, why the models use `season_length=12`) is justified with real numbers and charts over there. This notebook assumes you have already seen that reasoning and just applies it.

1.0.1 Repository layout

This notebook lives in `Code/notebooks/`. It imports shared configuration from `Code/config.py` - the same file `app/streamlit_app.py` reads - so column names, hierarchy structure, and the reconciler-to-column-suffix mapping can never silently drift between the pipeline and the dashboard.

1.0.2 What this notebook produces

1. Three **base forecasts** per series: `HistoricAverage` (a naive baseline), `AutoETS` (exponential smoothing), and `Theta` (a decomposition-based method).
2. Three **reconciliation methods** that make those forecasts add up correctly through the hierarchy: `Bottom-Up`, `MinTrace`, and `Top-Down`.
3. Accuracy scores (sMAPE) sliced two ways: by hierarchy level, and by sales-volume segment (top sellers vs. the long tail).

1.0.3 Data required

Download from Kaggle (`m5-forecasting-accuracy` competition) into `Code/data/m5/` (see `Code/data/README.md`): - `sales_train_validation.csv` - `calendar.csv`

Reproducibility. All outputs in this notebook were generated by executing it top-to-bottom in the `m5_forecast` environment. If you change the data, `config.py`, or any modelling step, re-run the whole notebook rather than individual cells - several steps (the item universe fix in Section 6.3 in particular) depend on state built by earlier cells.

1.1 Contents

1. Imports

2. Configuration
3. sMAPE — the one accuracy metric used everywhere
4. Data Loading
5. Reshape: wide $\rightarrow$ long, then attach real dates
6. Data Preparation
7. Build the Hierarchy
8. Base Forecasting Models
 - 8.5 Clip Negative Base Forecasts Before Reconciliation
9. Reconciliation
10. Combine Results and Attach Ground Truth
11. Evaluation — Accuracy by Hierarchy Level
12. Evaluation — Accuracy by Volume Segment
13. Negative Forecast Check
14. Sanity Checks
15. Optional Visualisation
16. Export Data for the Dashboard

1.2 1. Imports

- `statsforecast` fits the three forecasting models, fast, across thousands of series at once.
- `hierarchicalforecast` builds the hierarchy (the “summing matrix” that says how items roll up into departments, departments into categories, and so on) and applies the reconciliation methods.
- `utilsforecast` supplies `smape`, the one accuracy metric used everywhere in this notebook, and a plotting helper.

```
[1]: from pathlib import Path
import time
import inspect
import warnings

import numpy as np
import pandas as pd
from IPython.display import display

from statsforecast import StatsForecast
from statsforecast.models import HistoricAverage, AutoETS, Theta

from hierarchicalforecast.utils import aggregate
from hierarchicalforecast.core import HierarchicalReconciliation
from hierarchicalforecast.methods import BottomUpSparse, MinTraceSparse,
↳ TopDownSparse
from hierarchicalforecast.evaluation import evaluate as hierarchical_evaluate

from utilsforecast.plotting import plot_series
from utilsforecast import losses as uloss

# AutoETS checks each series' autocorrelation before fitting. Series that
```

```
# are constant throughout training (e.g. an item with zero sales the whole
# time) have zero variance, which makes this check divide by zero. This is
# harmless - such series simply fall back to HistoricAverage (see Section
# 14, check [3]) - so the warning is suppressed here to keep the output
# readable. It does NOT suppress errors, only this one specific message.
warnings.filterwarnings("ignore", message="invalid value encountered in divide")

print("Imports OK")
```

```
/opt/miniconda3/envs/m5_forecast/lib/python3.10/site-packages/tqdm/auto.py:21:
TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
    from .autonotebook import tqdm as notebook_tqdm
```

Imports OK

1.3 2. Configuration

Why a “Total” root exists. Without it, the hierarchy is really three separate trees (CA, TX, WI) with no single top. Top-Down reconciliation is only mathematically defined when there is one root to distribute downward from - so a “Total” node is added above State. See 01_eda.ipynb, Section 5 for the effect this has (before this fix, Top-Down was silently changing state-level forecasts, which it should never do).

Why SEASON_LENGTH = 12. The data is monthly, and 01_eda.ipynb, Section 7 shows a clear, repeating November/December sales spike every year. A `season_length` of 12 tells ETS and Theta “look for a pattern that repeats every 12 points” - without it, both models default to `season_length=1`, i.e. no seasonality at all, and produce almost flat forecasts.

Why these three time windows. Train ends where Test begins; Test ends where Live begins. “Live” extends beyond the actual data (which stops in April 2016) purely to give the pipeline something to write forecasts into - there is no real ground truth for May/June/July 2016 in this dataset, so the Live period is never used for evaluation, only for the dashboard.

```
[2]: import sys
sys.path.append(str(Path(".").resolve()))
import config as cfg

# --- Time windows -----
train_test_split_date: str = cfg.TRAIN_TEST_SPLIT_DATE    # train ends, test
↳begins
forecast_start_date: str = cfg.FORECAST_START_DATE        # test ends, live
↳begins
forecast_end_date: str = cfg.FORECAST_END_DATE            # live ends (no real
↳data past Apr 2016)

# --- Column names -----
date_col: str = cfg.DATE_COL
target_col: str = cfg.TARGET_COL
```

```

cat_label_col: str = cfg.CAT_LABEL_COL      # will hold "train" / "test" /   

↳ "live"
id_col: str = cfg.ID_COL                    # hierarchicalforecast's series   

↳ identifier

# --- Hierarchy columns, Total → ... → Item -----
col_total: str = cfg.COL_TOTAL
col_state: str = cfg.COL_STATE
col_store: str = cfg.COL_STORE
col_cat: str = cfg.COL_CAT
col_dept: str = cfg.COL_DEPT
col_item: str = cfg.COL_ITEM

hierarchy_cols: list = cfg.HIERARCHY_COLS

# `spec` tells hierarchicalforecast's aggregate() function which columns
# define each level, from the top down. Built once in config.py so it can
# never drift from hierarchy_cols above, or from what app/streamlit_app.py
# assumes about the hierarchy's shape.
spec: list = cfg.SPEC

# Carries the train/test/live label through the aggregation step. "min" is
# a safe, no-op choice here: since the label only depends on the date (not
# on which item/store it is), every row being aggregated for a given date
# already shares the same label, so "min" just returns that shared value.
exog_vars: dict = cfg.EXOG_VARS

SEASON_LENGTH: int = cfg.SEASON_LENGTH      # monthly data -> yearly   

↳ seasonality
PERCENTILES: list = cfg.PERCENTILES         # cumulative share of TRAINING   

↳ volume

# --- Folders   

↳ -----
M5_DATA_DIR = cfg.DATA_DIR
cache_raw_dir = cfg.RAW_CACHE_DIR

print("Configuration loaded")
print(f" - Train period : start -> {train_test_split_date}")
print(f" - Test period  : {train_test_split_date} -> {forecast_start_date}")
print(f" - Live period   : {forecast_start_date} -> {forecast_end_date}")
print(f" - Hierarchy     : {' -> '.join(c.replace('_id', '').title() for c in   

↳ hierarchy_cols)}")

```

Configuration loaded

- Train period : start -> 2015-01-01
- Test period : 2015-01-01 -> 2016-01-01

- Live period : 2016-01-01 -> 2016-07-01
- Hierarchy : Total -> State -> Store -> Cat -> Dept -> Item

1.4 3. sMAPE — the one accuracy metric used everywhere

The formula:

$$\text{sMAPE} = \text{mean} \left(\frac{|F - A|}{|F| + |A|} \right) \times 100$$

where F is the forecast and A is the actual value.

Worked example. Actual sales = 10, forecast = 12: - Error: $|12 - 10| = 2$ - Denominator: $|12| + |10| = 22$ - sMAPE for this point: $2/22 = 0.0909 \rightarrow 9.09\%$

The zero/zero case. If both the actual and the forecast are 0 (a dead item, correctly predicted as dead), the formula would divide 0 by 0. This is treated as **0% error** - a perfect forecast - rather than undefined.

Why this scale (0-100%) and not the classic 0-200% version. Some textbooks add a /2 to the denominator, which doubles the whole metric to a 0-200% scale. Neither version is more “correct” - they’re just different scaling conventions for the same underlying idea. This notebook uses the 0-100% version because it’s exactly what `utilsforecast.losses.smape` computes (confirmed below by printing its source), so there is only ever one scale in play, instead of two different numbers that look comparable but aren’t.

```
[3]: def smapecp(actual, forecast) -> float:
      """
      Symmetric Mean Absolute Percentage Error, as a percentage (0-100%).
      Formula: mean( |F - A| / (|F| + |A|) ) * 100
      Points where both actual and forecast are 0 contribute 0 error.
      """
      actual = np.asarray(actual, dtype=float)
      forecast = np.asarray(forecast, dtype=float)
      denom = np.abs(actual) + np.abs(forecast)
      ratio = np.divide(
          np.abs(forecast - actual), denom,
          out=np.zeros_like(denom), where=denom != 0,
      )
      return float(ratio.mean() * 100)

      # Proof that our formula matches the library's formula exactly - same
      # |F-A| / (|F|+|A|) fraction, same zero-handling. We just multiply by 100
      # afterwards to express it as a percentage, since the library returns the
      # raw, unscaled fraction.
      print("utilsforecast.losses.smape source (for comparison):")
      print(inspect.getsource(uloss.smape))
```

```

utilsforecast.losses.smape source (for comparison):
@_base_docstring
def smape(
    df: IntoDataFrameT,
    models: List[str],
    id_col: str = "unique_id",
    target_col: str = "y",
    cutoff_col: str = "cutoff",
) -> IntoDataFrameT:
    """Symmetric Mean Absolute Percentage Error (SMAPE)

    SMAPE measures the relative prediction
    accuracy of a forecasting method by calculating the relative deviation
    of the prediction and the observed value scaled by the sum of the
    absolute values for the prediction and observed value at a
    given time, then averages these deviations over the length
    of the series. This allows the SMAPE to have bounds between
    0% and 100% which is desirable compared to normal MAPE that
    may be undetermined when the target is zero."""

    def gen_expr(model):
        abs_err = (nw.col(model) - nw.col(target_col)).abs()
        denominator = _zero_to_nan(nw.col(model).abs() +
nw.col(target_col).abs())
        return (abs_err / denominator).alias(model).fill_null(0)

    return _nw_agg_expr(
        df=df,
        models=models,
        id_col=id_col,
        gen_expr=gen_expr,
        cutoff_col=cutoff_col,
    )

```

1.5 4. Data Loading

Raw M5 files are large (the sales file alone is ~450k rows x 1919 columns), so both files are cached to Parquet after the first load. Parquet is a columnar binary format: much faster to read back than re-parsing a CSV every time, and it preserves data types (dates stay dates, not strings).

```

[4]: cache_path = cache_raw_dir / "sales_train_validation.parquet"
if not cache_path.is_file():
    print(" Loading sales_train_validation.csv ...")
    df_sales_raw = pd.read_csv(M5_DATA_DIR / "sales_train_validation.csv")
    df_sales_raw.to_parquet(cache_path)
else:
    df_sales_raw = pd.read_parquet(cache_path)

```



```

print(f" Loaded sales data ({df_sales_raw.shape[0]:,} rows, {df_sales_raw.
    ↳shape[1]:,} cols)")

cache_path = cache_raw_dir / "calendar.parquet"
if not cache_path.is_file():
    print(" Loading calendar.csv ...")
    df_calendar = pd.read_csv(M5_DATA_DIR / "calendar.csv",
    ↳parse_dates=["date"])
    df_calendar.to_parquet(cache_path)
else:
    df_calendar = pd.read_parquet(cache_path)
print(f" Loaded calendar ({df_calendar.shape[0]:,} rows)")

```

```

Loaded sales data (30,490 rows, 1,919 cols)
Loaded calendar (1,969 rows)

```

1.6 5. Reshape: wide → long, then attach real dates

The raw sales file has one row per item-store, and one column per day (`d_1`, `d_2`, ... `d_1913`) - this is “wide” format. Forecasting tools expect “long” format instead: one row per (series, date, value). `melt()` does that reshape. The `calendar.csv` file is what actually maps `d_1`, `d_2`, etc. to real calendar dates, so it’s joined on immediately after.

```

[5]: print(" Melting sales data from wide to long format ...")
start_time = time.time()
meta_cols = ["id", "item_id", "dept_id", "cat_id", "store_id", "state_id"]
day_cols = [c for c in df_sales_raw.columns if c.startswith("d_")]
df_long = df_sales_raw[meta_cols + day_cols].melt(
    id_vars=meta_cols, value_vars=day_cols, var_name="d", value_name="sales",
)
print(f" Melted in {time.time() - start_time:.2f}s ({df_long.shape[0]:,} rows)")
del df_sales_raw

df_long = df_long.merge(df_calendar[["d", "date"]], on="d", how="left")
df_long.drop(columns=["d", "id"], inplace=True)
df_long["date"] = pd.to_datetime(df_long["date"])
print(f" Calendar joined - date range: {df_long['date'].min().date()} ->
    ↳{df_long['date'].max().date()}")
del df_calendar

```

```

Melting sales data from wide to long format ...
Melted in 42.00s (58,327,370 rows)
Calendar joined - date range: 2011-01-29 -> 2016-04-24

```

1.7 6. Data Preparation

Five steps happen here, in order, and the order matters:

1. **Aggregate daily → monthly.** Forecasting monthly instead of daily smooths out day-to-day

noise and matches the planning horizon most retail businesses actually use (see `01_eda.ipynb`, Section 6 for how sparse the daily data is at item level - monthly aggregation is one of the main ways this pipeline copes with that sparsity).

2. **Drop the partial first calendar month.** Sales data starts 2011-01-29, so the 2011-01-01 monthly bucket built in step 1 only contains 3 real days, not a full January. Left in, this understates January sales in one of the four training years, which distorts the January seasonal index that `season_length=12` relies on for ETS and Theta (see `01_eda.ipynb`, Section 3, for the underlying date-range check this relies on). The fix simply excludes any monthly bucket before the first *complete* month.
3. **Fix the item universe once, using training data only.** An item that first sold *after* the train/test split has no training history - no model here could ever have forecast it, so it's removed. Doing this now, before anything else is built, means test, live, and evaluation automatically end up using the exact same set of items - there is no later step where their item sets could quietly drift apart.
4. **Build the full monthly grid.** Every item-store combination needs a row for every month, including months with zero sales (so the models see the true shape of demand, not a shortened series), and including months beyond the real data (so the pipeline has a place to write live forecasts into).
5. **Label each row train / test / live**, based on its date.

```
[6]: # --- 6.1 Aggregate daily -> monthly -----
print(" Resampling daily sales to monthly ...")
start_time = time.time()
df_monthly = (
    df_long.groupby(
        [col_state, col_store, col_cat, col_dept, col_item, pd.
        ↳Grouper(key="date", freq="MS")]
    )["sales"].sum().reset_index()
)
print(f" Resampled in {time.time() - start_time:.2f}s ({df_monthly.shape[0]:,}
    ↳rows)")
del df_long
```

```
Resampling daily sales to monthly ...
Resampled in 28.78s (1,951,360 rows)
```

```
[7]: # --- 6.2 Drop the partial first calendar month -----
# Sales data starts 2011-01-29 (see 01_eda.ipynb, Section 3), so the
# 2011-01-01 monthly bucket built above only contains 3 real days, not a
# full January. Left in, this corrupts the January seasonal index for
# ETS/Theta, since season_length=12 would see a near-empty January in one
# of the four training years. This keeps every OTHER month untouched -
# only the single, genuinely partial first bucket is dropped.
first_full_month = df_monthly["date"].min() + pd.offsets.MonthBegin(1)
n_before = len(df_monthly)
df_monthly = df_monthly[df_monthly["date"] >= first_full_month].copy()
print(f" Dropped {n_before - len(df_monthly):,} rows from the partial first
    ↳month ")
```

```
f"(excluding monthly buckets before {first_full_month.date()}))"
```

Dropped 30,490 rows from the partial first month (excluding monthly buckets before 2011-02-01)

```
[8]: # --- 6.3 Fix the item universe, using TRAINING data only -----
# "Active" = had at least one month of non-zero sales before the train/test
# split. Items that never sold anything in that window are dropped, because
# no amount of modelling could forecast a series with zero training history.
train_active_items = (
    df_monthly.loc[
        (df_monthly["date"] < train_test_split_date) & (df_monthly["sales"] != 0),
        col_item,
    ].unique()
)
n_items_before = df_monthly[col_item].nunique()
df_monthly = df_monthly[df_monthly[col_item].isin(train_active_items)].copy()
n_items_after = df_monthly[col_item].nunique()
print(f" Item universe fixed at the training boundary.
      ({train_test_split_date})")
print(f"   - items before : {n_items_before:,}")
print(f"   - items after  : {n_items_after:,}")
print(f"   - dropped      : {n_items_before - n_items_after:,} (no sales before
      the split)")
```

Item universe fixed at the training boundary (2015-01-01)

```
- items before : 3,049
- items after  : 3,024
- dropped      : 25 (no sales before the split)
```

```
[9]: # --- 6.4 Build the complete monthly grid -----
# Cross-join every remaining item-store combination with every month from
# the earliest date in the data out to forecast_end_date, then fill any
# missing (item, month) combination with 0 sales. Months past the real
# data (after ~April 2016) get 0 here purely as a placeholder - there is
# no real sales data for them, and they are never used for evaluation.
print(" Building complete monthly grid ...")
start_time = time.time()
all_dates = pd.date_range(start=df_monthly["date"].min(),
                          end=forecast_end_date, freq="MS")
all_items = df_monthly[[col_state, col_store, col_cat, col_dept, col_item]].
drop_duplicates()
full_grid = all_items.merge(pd.DataFrame({"date": all_dates}), how="cross")
df_grid = full_grid.merge(
    df_monthly, on=[col_state, col_store, col_cat, col_dept, col_item, "date"],
    how="left",
)
```

```

df_grid["sales"] = df_grid["sales"].fillna(0).astype(np.float32)
df_grid["date"] = df_grid["date"].astype("datetime64[us]")
df_grid[col_total] = "Total" # constant column - the root of the hierarchy
print(f" Grid built in {time.time() - start_time:.2f}s ({df_grid.shape[0]:,}
    ↳rows)")
del df_monthly, full_grid, all_items

```

Building complete monthly grid ...
 Grid built in 1.46s (1,995,840 rows)

```

[10]: # --- 6.5 Label every row train / test / live, by date -----
def generate_cat_label(df, date_col, train_end, test_end, live_end,
    ↳label_col="_cat"):
    conditions = [
        df[date_col] < pd.Timestamp(train_end),
        (df[date_col] >= pd.Timestamp(train_end)) & (df[date_col] < pd.
    ↳Timestamp(test_end)),
        (df[date_col] >= pd.Timestamp(test_end)) & (df[date_col] <= pd.
    ↳Timestamp(live_end)),
    ]
    df[label_col] = np.select(conditions, ["train", "test", "live"],
    ↳default="unknown")
    return df

df = generate_cat_label(
    df_grid, date_col=date_col,
    train_end=train_test_split_date, test_end=forecast_start_date,
    ↳live_end=forecast_end_date,
)
del df_grid

print(" Data preparation completed")
print(f"   - Train rows : {(df[cat_label_col] == 'train').sum():,}")
print(f"   - Test rows  : {(df[cat_label_col] == 'test').sum():,}")
print(f"   - Live rows   : {(df[cat_label_col] == 'live').sum():,}")
print(f"   - Items       : {df[col_item].nunique():,}")
print(f"   - Bottom-level series (item x store): {df[col_item].nunique() *
    ↳df[col_store].nunique():,}")

```

Data preparation completed

- Train rows : 1,421,280
- Test rows : 362,880
- Live rows : 211,680
- Items : 3,024
- Bottom-level series (item x store): 30,240

1.8 7. Build the Hierarchy

`aggregate()` does three things at once, using the `spec` from Section 2:

1. Builds `Y_df`: the target values (sales) rolled up to **every** level of the hierarchy, not just the bottom. A “CA” row is the sum of every store in California, for example - `aggregate()` computes this automatically for all six levels.
2. Builds `S_df`: the “summing matrix”, a compact representation of exactly how bottom-level series (items) roll up into every higher node. This is what the reconciliation methods use in Section 9 to keep forecasts consistent across levels.
3. Builds `tags`: a lookup of which `unique_ids` belong to which level, used later when evaluating accuracy separately at each level.

This only needs to run **once**. Because the item universe was already fixed to a single, training-based set of items in Section 6, the same hierarchy is valid for the test period, the live period, and evaluation - there is no need to rebuild it three times with three different item sets.

```
[11]: print(" Building hierarchy ...")
start_time = time.time()
Y_df, S_df, tags = aggregate(
    df, spec=spec, exog_vars=exog_vars, id_col=id_col, time_col=date_col,
    target_cols=[target_col], sparse_s=True,
)
Y_df = Y_df.rename(columns={k + f"_{v}": k for k, v in exog_vars.items()})
print(f" Hierarchy built in {time.time() - start_time:.2f}s")
print(f" - Total nodes : {Y_df[id_col].nunique():,}")
for lvl_cols in spec:
    lvl_depth = len(lvl_cols) - 1
    n = (Y_df[id_col].str.count("/") == lvl_depth).sum() / Y_df[date_col].
    nunique()
    print(f" - {'/'.join(lvl_cols):<50} {int(round(n)):>8,} series")
```

```
Building hierarchy ...
Hierarchy built in 51.10s
- Total nodes : 30,354
- total_id                                     1 series
- total_id/state_id                           3 series
- total_id/state_id/store_id                  10 series
- total_id/state_id/store_id/cat_id           30 series
- total_id/state_id/store_id/cat_id/dept_id   70 series
- total_id/state_id/store_id/cat_id/dept_id/item_id 30,240 series
```

1.9 8. Base Forecasting Models

Three models, in increasing order of sophistication:

- **HistoricAverage** — forecasts the mean of everything seen so far. Flat, no trend, no seasonality. Included as a reference point rather than a hurdle every other model is assumed to clear: on short or highly seasonal series in particular, a flat baseline can be competitive with - or beat - more sophisticated models, and Section 12 checks whether that happens here.

- **AutoETS** — exponential smoothing. Automatically picks how much weight to give trend and seasonality based on the data.
- **Theta** — decomposes the series and blends a short-term view with a long-term trend view. Won the M3 forecasting competition.

`season_length=12` tells ETS and Theta to look for a pattern repeating every 12 months (see Section 2 and 01_eda.ipynb Section 7 for why).

`fallback_model=HistoricAverage()` is a safety net: on 30,000+ series, some will be too short, too sparse, or too irregular for ETS/Theta to fit. Rather than the whole run crashing on one bad series, that series simply gets the HistoricAverage forecast instead. Section 15 measures exactly how often this happens.

Two separate forecasts are produced: one trained on Train only (to score against the real Test-period actuals), and one trained on Train+Test (to produce the Live forecasts, which have no ground truth to check against).

```
[12]: sf = StatsForecast(
    models=[
        HistoricAverage(alias="histavg"),
        AutoETS(season_length=SEASON_LENGTH, alias="ets"),
        Theta(season_length=SEASON_LENGTH, alias="theta"),
    ],
    freq="MS",
    n_jobs=-1,
    fallback_model=HistoricAverage(),
)

# --- Test period: trained on 'train' rows only -----
h_test = pd.Period(forecast_start_date, "M").ordinal - pd.
    .Period(train_test_split_date, "M").ordinal
print(f" Forecasting test period ({h_test} months) ...")
start_time = time.time()
pred_test = sf.forecast(
    h=h_test,
    df=Y_df.query(f"{cat_label_col} == 'train'").drop(columns=[cat_label_col]),
    id_col=id_col, time_col=date_col, target_col=target_col,
)
pred_test = pred_test[pred_test[date_col] < forecast_start_date]
print(f" Test forecast done in {time.time() - start_time:.2f}s")
```

Forecasting test period (12 months) ...

```
/opt/miniconda3/envs/m5_forecast/lib/python3.10/site-
packages/statsmodels/tsa/stattools.py:702: RuntimeWarning: invalid value
encountered in divide
    acf = avf[: nlags + 1] / avf[0]
/opt/miniconda3/envs/m5_forecast/lib/python3.10/site-
packages/statsmodels/tsa/stattools.py:702: RuntimeWarning: invalid value
encountered in divide
```

```

    acf = avf[: nlags + 1] / avf[0]
/opt/miniconda3/envs/m5_forecast/lib/python3.10/site-
packages/statsmodels/tsa/stattools.py:702: RuntimeWarning: invalid value
encountered in divide
    acf = avf[: nlags + 1] / avf[0]
/opt/miniconda3/envs/m5_forecast/lib/python3.10/site-
packages/statsmodels/tsa/stattools.py:702: RuntimeWarning: invalid value
encountered in divide
    acf = avf[: nlags + 1] / avf[0]
/opt/miniconda3/envs/m5_forecast/lib/python3.10/site-
packages/statsmodels/tsa/stattools.py:702: RuntimeWarning: invalid value
encountered in divide
    acf = avf[: nlags + 1] / avf[0]
/opt/miniconda3/envs/m5_forecast/lib/python3.10/site-
packages/statsmodels/tsa/stattools.py:702: RuntimeWarning: invalid value
encountered in divide
    acf = avf[: nlags + 1] / avf[0]
/opt/miniconda3/envs/m5_forecast/lib/python3.10/site-
packages/statsmodels/tsa/stattools.py:702: RuntimeWarning: invalid value
encountered in divide
    acf = avf[: nlags + 1] / avf[0]
Test forecast done in 253.69s

```

```

[13]: # --- Live period: trained on 'train' + 'test' rows -----
h_live = (
    pd.Period(forecast_end_date, "M").ordinal - pd.Period(forecast_start_date,
    ↵ "M").ordinal + 1
)
print(f" Forecasting live period ({h_live} months) ...")
start_time = time.time()
pred_live = sf.forecast(
    h=h_live,
    df=Y_df.query(f"{cat_label_col}.isin(['train', 'test'])").
    ↵ drop(columns=[cat_label_col]),
    id_col=id_col, time_col=date_col, target_col=target_col,
)
pred_live = pred_live[pred_live[date_col] <= forecast_end_date]
print(f" Live forecast done in {time.time() - start_time:.2f}s")

```

```

Forecasting live period (7 months) ...
/opt/miniconda3/envs/m5_forecast/lib/python3.10/site-
packages/statsmodels/tsa/stattools.py:702: RuntimeWarning: invalid value
encountered in divide

```

```

    acf = avf[: nlags + 1] / avf[0]
/opt/miniconda3/envs/m5_forecast/lib/python3.10/site-
packages/statsmodels/tsa/stattools.py:702: RuntimeWarning: invalid value
encountered in divide
    acf = avf[: nlags + 1] / avf[0]
/opt/miniconda3/envs/m5_forecast/lib/python3.10/site-
packages/statsmodels/tsa/stattools.py:702: RuntimeWarning: invalid value
encountered in divide
    acf = avf[: nlags + 1] / avf[0]
/opt/miniconda3/envs/m5_forecast/lib/python3.10/site-
packages/statsmodels/tsa/stattools.py:702: RuntimeWarning: invalid value
encountered in divide
    acf = avf[: nlags + 1] / avf[0]
/opt/miniconda3/envs/m5_forecast/lib/python3.10/site-
packages/statsmodels/tsa/stattools.py:702: RuntimeWarning: invalid value
encountered in divide
    acf = avf[: nlags + 1] / avf[0]
/opt/miniconda3/envs/m5_forecast/lib/python3.10/site-
packages/statsmodels/tsa/stattools.py:702: RuntimeWarning: invalid value
encountered in divide
    acf = avf[: nlags + 1] / avf[0]
Live forecast done in 255.43s

```

1.10 8.5 Clip Negative Base Forecasts Before Reconciliation

ETS and Theta are free to predict below zero (Section 13 shows how often this happens - 2,796 and 21,768 rows respectively, pre-fix). Left alone, this creates an unfair comparison between reconcilers: `BottomUpSparse` simply inherits whatever the base model produced, negatives included, while `MinTraceSparse(nonnegative=True)` forces every value to `0`. Part of any accuracy gap between the two would then reflect the nonnegativity constraint itself, not the reconciliation method. Clipping every base forecast at `0` **before** reconciliation puts both reconcilers on equal footing - `BottomUpSparse` and `MinTraceSparse` now start from the same, already-nonnegative base forecasts.

```

[14]: # Clip negative base forecasts before reconciling. Sales cannot be
      # negative, so any negative base forecast is already known to be wrong;
      # leaving it in place would only benefit whichever reconciler doesn't
      # separately enforce nonnegativity. Applied to both the test and live
      # forecast tables, for all three base models.
      for _df in (pred_test, pred_live):
          for _col in ["histavg", "ets", "theta"]:
              _n_clipped = (_df[_col] < 0).sum()
              _df[_col] = _df[_col].clip(lower=0)

```



```
print(f" Clipped {_n_clipped:,} negative values in {_col} "
      f"({'test' if _df is pred_test else 'live'})")
```

```
Clipped 0 negative values in histavg (test)
Clipped 2,300 negative values in ets (test)
Clipped 16,345 negative values in theta (test)
Clipped 0 negative values in histavg (live)
Clipped 712 negative values in ets (live)
Clipped 6,752 negative values in theta (live)
```

1.11 9. Reconciliation

Each base model above was fit **independently** at every hierarchy node - the “CA” forecast and the “CA_1” (a specific store) forecast have no mathematical relationship, so summing up all the stores in California generally will **not** equal the separately-fit CA forecast. This is “incoherence”, and reconciliation fixes it.

- **Bottom-Up** — trust the item-level forecasts completely, and simply sum them upward to get every higher level.
- **Top-Down** — trust the top-level (Total) forecast completely, and split it downward using each series’ historical share of sales.
- **MinTrace** — a compromise: adjusts forecasts at every level simultaneously to minimise overall error, using information from the whole hierarchy rather than committing entirely to top or bottom. `nonnegative=True` additionally forces every reconciled forecast to be 0. As of Section 8.5, the base forecasts fed into every reconciler here are already clipped at 0 (Section 13 shows how often ETS and Theta produced negative values before that clipping was applied), so this flag is now a belt-and-braces guarantee rather than the sole source of nonnegativity - it also keeps MinTrace’s *reconciliation adjustment* itself from reintroducing a negative value.

Why `wls_struct` and not a shrinkage-based variant. `MinTraceSparse` supports structural scaling (`wls_struct`, used here) but does not expose a shrinkage-based covariance estimator. Rather than approximate one - which would mean the results reported as “MinTrace” no longer match the specific, well-defined variant described in the thesis methodology chapter - this pipeline uses `wls_struct` throughout, and a comparison against a shrinkage-based MinTrace variant is left as future work.

```
[15]: reconciler = HierarchicalReconciliation(reconcilers=[
    BottomUpSparse(),
    MinTraceSparse(method="wls_struct", nonnegative=True),
    TopDownSparse(method="proportion_averages"),
])

print(" Reconciling test forecasts ...")
start_time = time.time()
rec_test = reconciler.reconcile(
    Y_hat_df=pred_test,
    Y_df=Y_df.query(f"{cat_label_col} == 'train'").
    <-drop(columns=[cat_label_col]),
```

```

    S_df=S_df, tags=tags, id_col=id_col, time_col=date_col,
    ↪target_col=target_col,
)
print(f" Test reconciliation done in {time.time() - start_time:.2f}s")

print(" Reconciling live forecasts ...")
start_time = time.time()
rec_live = reconciler.reconcile(
    Y_hat_df=pred_live,
    Y_df=Y_df.query(f"{cat_label_col}.isin(['train', 'test'])").
    ↪drop(columns=[cat_label_col]),
    S_df=S_df, tags=tags, id_col=id_col, time_col=date_col,
    ↪target_col=target_col,
)
print(f" Live reconciliation done in {time.time() - start_time:.2f}s")

```

```

Reconciling test forecasts ...
Test reconciliation done in 52.70s
Reconciling live forecasts ...
Live reconciliation done in 33.34s

```

1.12 10. Combine Results and Attach Ground Truth

Test and Live forecasts are stacked into one table each (base, and reconciled). Ground truth comes directly from `Y_df` - the same table built once in Section 7 - so there is no separate “evaluation hierarchy” to build or keep in sync; every (unique_id, date) in the forecasts is guaranteed to already exist in `Y_df`.

```

[16]: pred = pd.concat(
    [pred_test.assign(**{cat_label_col: "test"}), pred_live.
    ↪assign(**{cat_label_col: "live"})],
    ignore_index=True,
)
rec_fc = pd.concat(
    [rec_test.assign(**{cat_label_col: "test"}), rec_live.
    ↪assign(**{cat_label_col: "live"})],
    ignore_index=True,
)
assert rec_fc.shape[0] == pred.shape[0], "Shape mismatch between rec_fc and
    ↪pred."
print(f" Base predictions collected : {pred.shape[0]:,} rows")
print(f" Reconciled forecasts collected: {rec_fc.shape[0]:,} rows")

ground_truth = Y_df[[id_col, date_col, target_col]]
pred_with_gt = pred.merge(ground_truth, on=[id_col, date_col], how="left",
    ↪validate="1:1")

```

```

rec_with_gt = rec_fc.merge(ground_truth, on=[id_col, date_col], how="left",
    validate="1:1")

# Safety net only - should never actually fire, since every (id, date) in
# pred/rec_fc originated from Y_df in the first place.
n_missing = pred_with_gt[target_col].isna().sum() + rec_with_gt[target_col].
    isna().sum()
print(f" Missing ground-truth rows after merge: {n_missing} (expected 0)")
pred_with_gt[target_col] = pred_with_gt[target_col].fillna(0)
rec_with_gt[target_col] = rec_with_gt[target_col].fillna(0)

```

```

Base predictions collected : 576,726 rows
Reconciled forecasts collected: 576,726 rows
Missing ground-truth rows after merge: 0 (expected 0)

```

1.13 11. Evaluation — Accuracy by Hierarchy Level

`hierarchical_evaluate` computes sMAPE for every series, then averages within each hierarchy level (Total, State, Store, Category, Department, Item), for every model column requested. The result is multiplied by 100 here, since the library returns the raw, unscaled fraction (see Section 3).

```

[17]: rec_methods = [
    c for c in rec_with_gt.columns
    if c.startswith("histavg/") or c.startswith("ets/") or c.startswith("theta/
    ")
]
print(f" Reconciled methods found: {rec_methods}")

print("\n" + "=" * 80)
print("EVALUATION: BASE FORECASTS (sMAPE %, by hierarchy level)")
print("=" * 80)
metrics_base = hierarchical_evaluate(
    df=pred_with_gt.query(f"{cat_label_col} == 'test'"),
    metrics=[uloss.smape], tags=tags, models=["histavg", "ets", "theta"],
    id_col=id_col, time_col=date_col, target_col=target_col, agg_fn="mean",
).query("level != 'Overall'")
metrics_base[["histavg", "ets", "theta"]] *= 100
display(metrics_base)

```

```

Reconciled methods found: ['histavg/BottomUpSparse', 'ets/BottomUpSparse',
'theta/BottomUpSparse', 'histavg/MinTraceSparse_method-wls_struct_nonnegative-
True', 'ets/MinTraceSparse_method-wls_struct_nonnegative-True',
'theta/MinTraceSparse_method-wls_struct_nonnegative-True',
'histavg/TopDownSparse_method-proportion_averages', 'ets/TopDownSparse_method-
proportion_averages', 'theta/TopDownSparse_method-proportion_averages']

```

```

=====
EVALUATION: BASE FORECASTS (sMAPE %, by hierarchy level)

```

```
=====
```

	level	metric	histavg \
0	total_id	smape	6.579472
1	total_id/state_id	smape	6.853592
2	total_id/state_id/store_id	smape	8.186627
3	total_id/state_id/store_id/cat_id	smape	11.324590
4	total_id/state_id/store_id/cat_id/dept_id	smape	13.073418
5	total_id/state_id/store_id/cat_id/dept_id/item_id	smape	40.908002

	ets	theta
0	5.530957	4.236969
1	4.879873	4.316024
2	5.953599	5.348969
3	7.306784	6.733565
4	7.880821	8.263848
5	37.720347	36.317889

```
[18]: print("\n" + "=" * 80)
print("EVALUATION: RECONCILED FORECASTS (sMAPE %, by hierarchy level)")
print("=" * 80)
metrics_rec = hierarchical_evaluate(
    df=rec_with_gt.query(f"{cat_label_col} == 'test'"),
    metrics=[uloss.smape], tags=tags, models=rec_methods,
    id_col=id_col, time_col=date_col, target_col=target_col, agg_fn="mean",
).query("level != 'Overall'")
metrics_rec[rec_methods] *= 100
display(metrics_rec)
```

```
=====
```

EVALUATION: RECONCILED FORECASTS (sMAPE %, by hierarchy level)

```
=====
```

	level	metric \
0	total_id	smape
1	total_id/state_id	smape
2	total_id/state_id/store_id	smape
3	total_id/state_id/store_id/cat_id	smape
4	total_id/state_id/store_id/cat_id/dept_id	smape
5	total_id/state_id/store_id/cat_id/dept_id/item_id	smape

	histavg/BottomUpSparse	ets/BottomUpSparse	theta/BottomUpSparse \
0	6.579479	3.940593	3.780967
1	6.853595	3.966725	3.758364
2	8.186627	4.773414	4.864666
3	11.324590	6.218255	6.181708
4	13.073418	7.914683	7.597197
5	40.908002	37.720347	36.317889

```
histavg/MinTraceSparse_method-wls_struct_nonnegative-True \
0      6.579392
1      6.853528
2      8.186590
3     11.324554
4     13.073411
5     41.759892
```

```
ets/MinTraceSparse_method-wls_struct_nonnegative-True \
0      5.190602
1      5.114232
2      5.577928
3      7.187220
4      8.002738
5     40.620891
```

```
theta/MinTraceSparse_method-wls_struct_nonnegative-True \
0      4.362197
1      4.356591
2      5.251183
3      6.523516
4      7.984296
5     39.241883
```

```
histavg/TopDownSparse_method-proportion_averages \
0      6.579472
1      6.853588
2      8.186622
3     11.324585
4     13.073413
5     40.908000
```

```
ets/TopDownSparse_method-proportion_averages \
0      5.530957
1      5.875760
2      7.595910
3     10.628312
4     12.377877
5     40.739535
```

```
theta/TopDownSparse_method-proportion_averages
0      4.236969
1      4.765819
2      6.968626
3      9.748501
4     11.487926
5     40.539253
```

A pattern worth carrying into the results chapter. For both ETS and Theta, **Bottom-Up beats MinTrace at every level above Item** in the table above (e.g. ETS at `total_id`: Bottom-Up 3.94% vs. MinTrace 5.19% vs. the unreconciled base 5.53%). This runs somewhat against the usual framing of MinTrace as the theoretically “optimal” reconciler (Wickramasuriya et al., 2019), so it’s worth discussing explicitly rather than treating MinTrace’s superiority as a given. Two plausible contributors to explore in the discussion chapter:

1. `wls_struct` weights each node purely by how many bottom-level series sum into it, not by an estimate of each series’ forecast-error variance/covariance - a coarser signal than a shrinkage-based MinTrace variant would use.
2. The upper levels of this hierarchy are already fairly stable and non-intermittent (see `01_eda.ipynb`, Section 4), so Bottom-Up’s plain summation has little incoherence left to correct - while MinTrace’s cross-level adjustment has more room to introduce noise than to remove it.

1.14 12. Evaluation — Accuracy by Volume Segment

Real businesses don’t care equally about every product - a handful of top sellers usually drive most of the revenue (the Pareto pattern explored in `01_eda.ipynb`, Section 8). This section checks whether forecast accuracy differs between the biggest sellers and the long tail.

How “top p% of volume” is defined. Rank every item x store series by how much it sold **during training** (never on test data - that would be using information a real forecaster wouldn’t have yet). Then, walking down that list from the biggest seller, keep adding series until their combined sales reach p% of everyone’s total training sales. Whichever series were needed to get there make up the “top p%” bucket.

This means “top 5%” is a *sales share*, not a *series count* - it is typically a small number of series, not 5% of all 30,240 of them. The printout below shows exactly how many series land in each bucket.

Unlike the level-based evaluation above, this is deliberately done **only** at the bottom (item x store) level, never re-aggregated into fake “state” or “category” totals built from a handful of selected items - a state built from 5% of its items isn’t a real state.

```
[19]: bottom_depth = len(spec[-1]) - 1
      bottom_ids = Y_df.loc[Y_df[id_col].str.count("/") == bottom_depth, id_col].
      ↪unique()
      print(f" Bottom-level (item x store) series: {len(bottom_ids):,}")

      train_volume = (
          Y_df[(Y_df[id_col].isin(bottom_ids)) & (Y_df[cat_label_col] == "train")]
          .groupby(id_col)[target_col].sum()
          .sort_values(ascending=False)
      )

      def bucket_ids_by_volume(volume: pd.Series, percentiles: list) -> dict:
          """
          Rank series by volume (largest first). For each percentile p, return
```

```

the smallest set of top series whose combined volume reaches at least
p% of the total.
"""
cum_share = volume.cumsum() / volume.sum() * 100
buckets = {}
for p in percentiles:
    if p >= 100:
        buckets[p] = volume.index.tolist()
        continue
    selected = cum_share[cum_share <= p].index.tolist()
    if not selected: # the single biggest series alone already exceeds p%
        selected = [volume.index[0]]
    buckets[p] = selected
return buckets

volume_buckets = bucket_ids_by_volume(train_volume, PERCENTILES)

print("\n Volume buckets (share of TRAINING volume):")
for p, ids in volume_buckets.items():
    print(f"    top {p:>3}% of volume -> {len(ids):>6,} series "
          f"({len(ids) / len(bottom_ids):6.2%} of all {len(bottom_ids):,}
          ↵bottom-level series)")

```

Bottom-level (item x store) series: 30,240

```

Volume buckets (share of TRAINING volume):
    top    5% of volume ->      29 series ( 0.10% of all 30,240 bottom-level
series)
    top   10% of volume ->     94 series ( 0.31% of all 30,240 bottom-level
series)
    top   20% of volume ->    326 series ( 1.08% of all 30,240 bottom-level
series)
    top   50% of volume ->  2,151 series ( 7.11% of all 30,240 bottom-level
series)
    top   80% of volume ->  8,155 series (26.97% of all 30,240 bottom-level
series)
    top  100% of volume -> 30,240 series (100.00% of all 30,240 bottom-level
series)

```

```

[20]: def evaluate_volume_buckets(df_with_gt, model_cols, buckets, test_label="test") ↵
      ↵-> pd.DataFrame:
      """sMAPE per volume bucket, over the test period, bottom-level series only.
      ↵"""
      test_df = df_with_gt[df_with_gt[cat_label_col] == test_label]
      rows = []
      for p, ids in buckets.items():

```

```

        bucket_df = test_df[test_df[id_col].isin(ids)]
        row = {"top_%_of_volume": p, "n_series": len(ids)}
        for col in model_cols:
            row[col] = smape_pct(bucket_df[target_col], bucket_df[col])
        rows.append(row)
    return pd.DataFrame(rows)

print("\n" + "=" * 80)
print("EVALUATION: BASE FORECASTS BY VOLUME SEGMENT (sMAPE %)")
print("=" * 80)
volume_results_base = evaluate_volume_buckets(pred_with_gt, ["histavg", "ets",
    ↪ "theta"], volume_buckets)
display(volume_results_base)

print("\n" + "=" * 80)
print("EVALUATION: RECONCILED FORECASTS BY VOLUME SEGMENT (sMAPE %)")
print("=" * 80)
volume_results_rec = evaluate_volume_buckets(rec_with_gt, rec_methods,
    ↪ volume_buckets)
display(volume_results_rec)

```

```

=====
EVALUATION: BASE FORECASTS BY VOLUME SEGMENT (sMAPE %)
=====

```

	top_%_of_volume	n_series	histavg	ets	theta
0	5	29	16.612222	22.968469	22.969570
1	10	94	22.507600	29.878109	30.409768
2	20	326	27.891465	33.426580	32.786666
3	50	2151	28.881365	31.930420	31.039820
4	80	8155	29.676837	31.838834	31.104829
5	100	30240	40.908002	37.720347	36.317889

```

=====
EVALUATION: RECONCILED FORECASTS BY VOLUME SEGMENT (sMAPE %)
=====

```

	top_%_of_volume	n_series	histavg/BottomUpSparse	ets/BottomUpSparse	\
0	5	29	16.612222	22.968469	
1	10	94	22.507600	29.878109	
2	20	326	27.891465	33.426580	
3	50	2151	28.881365	31.930420	
4	80	8155	29.676837	31.838834	
5	100	30240	40.908002	37.720347	

theta/BottomUpSparse \

0	22.969570
1	30.409768
2	32.786666
3	31.039820
4	31.104829
5	36.317889

	histavg/MinTraceSparse_method-wls_struct_nonnegative-True \
0	16.612220
1	22.507596
2	27.891461
3	28.881356
4	29.676839
5	41.759892

	ets/MinTraceSparse_method-wls_struct_nonnegative-True \
0	23.028875
1	30.048930
2	33.690437
3	32.509668
4	32.548459
5	40.620891

	theta/MinTraceSparse_method-wls_struct_nonnegative-True \
0	23.005153
1	30.981419
2	34.380797
3	32.850072
4	32.867587
5	39.241883

	histavg/TopDownSparse_method-proportion_averages \
0	16.612224
1	22.507602
2	27.891466
3	28.881366
4	29.676837
5	40.908000

	ets/TopDownSparse_method-proportion_averages \
0	16.975081
1	22.906617
2	28.144875
3	28.994753
4	29.688648
5	40.739535

theta/TopDownSparse_method-proportion_averages

0	17.474691
1	23.423231
2	28.482957
3	29.160204
4	29.727582
5	40.539253

Visualizing the reversal. The reconciled volume-bucket table above shows Top-Down beating Bottom-Up at every bucket up to 80% of volume, then losing at 100% — plotting this directly makes the crossover visible as the dashed (Top-Down) and solid (Bottom-Up) lines trade places.

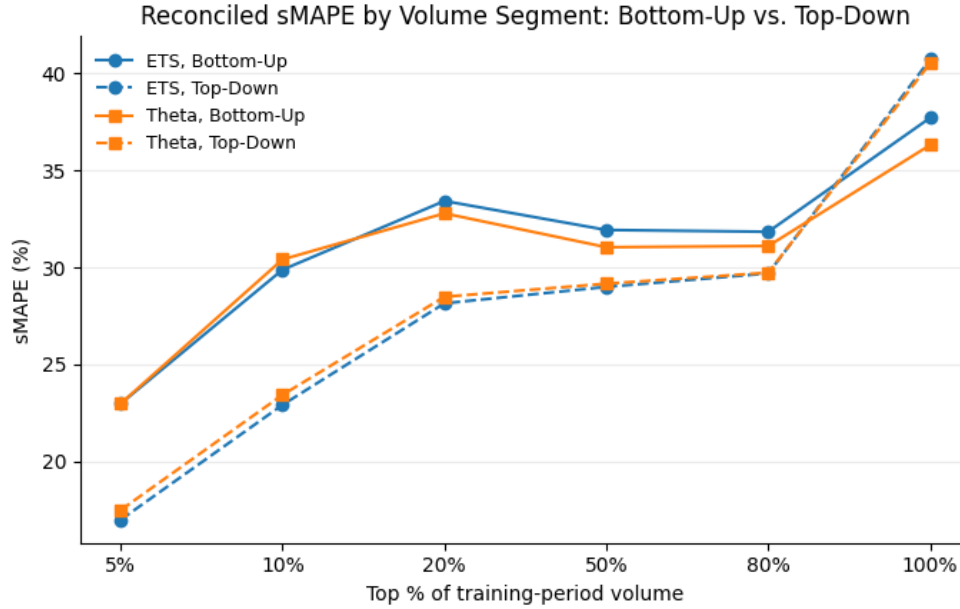
```
[23]: import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(7, 4.5))

buckets = volume_results_rec["top_%_of_volume"].astype(str)
x = range(len(buckets))

ax.plot(x, volume_results_rec["ets/BottomUpSparse"],
        marker="o", color="tab:blue", linestyle="-", label="ETS, Bottom-Up")
ax.plot(x, volume_results_rec["ets/TopDownSparse_method-proportion_averages"],
        marker="o", color="tab:blue", linestyle="--", label="ETS, Top-Down")
ax.plot(x, volume_results_rec["theta/BottomUpSparse"],
        marker="s", color="tab:orange", linestyle="-", label="Theta, Bottom-Up")
ax.plot(x, volume_results_rec["theta/TopDownSparse_method-proportion_averages"],
        marker="s", color="tab:orange", linestyle="--", label="Theta, Top-Down")

ax.set_xticks(list(x))
ax.set_xticklabels([b + "%" for b in buckets])
ax.set_xlabel("Top % of training-period volume")
ax.set_ylabel("sMAPE (%)")
ax.set_title("Reconciled sMAPE by Volume Segment: Bottom-Up vs. Top-Down")
ax.legend(frameon=False, fontsize=9)
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.grid(axis="y", alpha=0.25)

plt.tight_layout()
plt.savefig("volumeReconciledChart.png", dpi=200)
plt.show()
```



Note on averaging convention. The by-level evaluation in Section 11 uses `hierarchical_evaluate`, which computes sMAPE per series first and then averages the per-series values (a **macro-average** - every series counts equally). `evaluate_volume_buckets` here instead pools every row in a bucket before computing sMAPE (a **micro-average**). These are not the same statistic in general - but they coincide here because the evaluation grid is perfectly balanced: every series contributes exactly the same 12 test-period rows (Section 6.4 zero-fills every item-store-month rather than dropping missing periods), so pooling reduces algebraically to averaging the per-series means. This is confirmed by the “top 100%” row above matching the Item-level row in Section 11 to 6 decimal places. If a future version of this pipeline used a ragged panel (e.g. dropped rather than zero-filled missing months), this equivalence would no longer hold and the two evaluations would need to be reconciled explicitly.

1.15 13. Negative Forecast Check

Sales can’t be negative, but ETS and Theta are free to predict below zero if the underlying math (e.g. a downward trend) pushes them there. Section 8.5 now clips every base forecast at 0 before reconciliation runs, so the `histavg/ets/theta` columns below are expected to show **zero** negatives - this check exists to confirm that clipping step actually worked, not to discover the original problem (the print statements in Section 8.5 report the pre-clip counts for the record). `MinTraceSparse(nonnegative=True)` additionally guarantees its own reconciliation adjustment can’t reintroduce a negative value; `TopDownSparse` is safe by construction (proportions of a non-negative total are always non-negative); `BottomUpSparse` simply inherits whatever the (now-clipped) base model produced.

```
[24]: print("\n" + "=" * 80)
print("VALIDATION: Negative Forecast Check")
print("=" * 80)
for base_model in ["histavg", "ets", "theta"]:
    neg = (pred_with_gt[base_model] < 0).sum()
    print(f" {base_model:<8} base negatives: {neg}")
for col in rec_methods:
    neg = (rec_with_gt[col] < 0).sum()
    print(f" {col} negatives: {neg}")
```

```
=====
VALIDATION: Negative Forecast Check
=====
```

```
histavg  base negatives: 0
ets      base negatives: 0
theta    base negatives: 0
histavg/BottomUpSparse negatives: 0
ets/BottomUpSparse negatives: 0
theta/BottomUpSparse negatives: 0
histavg/MinTraceSparse_method-wls_struct_nonnegative-True negatives: 0
ets/MinTraceSparse_method-wls_struct_nonnegative-True negatives: 0
theta/MinTraceSparse_method-wls_struct_nonnegative-True negatives: 0
histavg/TopDownSparse_method-proportion_averages negatives: 0
ets/TopDownSparse_method-proportion_averages negatives: 0
theta/TopDownSparse_method-proportion_averages negatives: 0
```

1.16 14. Sanity Checks

Four quick checks that the pipeline is doing what it claims to, before trusting any of the numbers above.

[1] **Coherence check.** HistoricAverage is a *linear* forecast - the mean of a sum equals the sum of means. That makes its base forecasts already perfectly coherent, *before* any reconciliation runs. So Bottom-Up (and MinTrace, mostly) applied to HistoricAverage should barely change anything - if this check shows a big difference, something in the merge or the reconciliation step is broken. (Note: forecasts must be matched by **unique_id and date** before comparing - the two dataframes are not guaranteed to be in the same row order.)

[2] **Top-Down root check.** With a single “Total” root now in place, Top-Down reconciliation should never change the top-level forecast - it can only redistribute it downward. A non-zero result here would mean the “Total” root fix from Section 2 isn’t actually working.

[3] **Fallback rate.** How often ETS/Theta produced exactly the HistoricAverage forecast at every single horizon - a proxy for “this series could not be fit, so it fell back to the safety net” (Section 8).

[4] **Top-Down vs. base HistoricAverage, at a non-root level.** Top-Down redistributes the Total forecast using each series’ *historical average share* of volume - a different quantity from the ratios implied by HistoricAverage’s own independently-fit per-series forecasts. So below the Total

level, these two should genuinely differ; a near-zero result here would indicate the reconciler is silently not doing anything (e.g. a no-op on the sparse root, or a merge picking up the wrong column).

```
[25]: print("\n" + "=" * 80)
print("SANITY CHECKS")
print("=" * 80)

# [1] Coherence check - must merge on (id, date) first; the two
# dataframes are not guaranteed to share the same row order.
check1 = pred_test[[id_col, date_col, "histavg"]].merge(
    rec_test[[id_col, date_col, "histavg/BottomUpSparse"]], on=[id_col,
    ↵date_col],
)
diff = (check1["histavg/BottomUpSparse"] - check1["histavg"]).abs().max()
print(f" [1] HistoricAverage vs BottomUp(HistoricAverage): max diff = {diff:.
    ↵6f} (expect ~0)")

# [2] Top-Down must not change the "Total" root.
root_base = pred_test.loc[pred_test[id_col] == "Total", "ets"].to_numpy()
root_td = rec_test.loc[
    rec_test[id_col] == "Total", "ets/TopDownSparse_method-proportion_averages"
].to_numpy()
print(f" [2] Top-Down root check (ETS): max diff = {np.abs(root_td - root_base).
    ↵max():.6f} (expect ~0)")

# [3] Fallback rate, counted per series (not per row).
for m in ["ets", "theta"]:
    match = pred_test[m] == pred_test["histavg"]
    same_as_fallback = match.groupby(pred_test[id_col]).all()
    print(f" [3] {m:<6} fell back on {same_as_fallback.sum():,} of
    ↵{len(same_as_fallback):,} "
        f"series ({same_as_fallback.mean():.2%})")

# [4] Top-Down vs base histavg, at a NON-root level - should NOT be ~0.
# Top-Down redistributes the Total forecast by historical volume share,
# not by the ratios implied by histavg's own per-series forecasts, so at
# any level below Total these should genuinely differ.
diff_nonroot = (
    rec_test["histavg/TopDownSparse_method-proportion_averages"] -
    ↵pred_test["histavg"]
).abs().max()
print(f" [4] Top-Down vs base histavg (non-root check): max diff =
    ↵{diff_nonroot:,.2f} (expect >> 0)")
```

=====

SANITY CHECKS

```
=====
[1] HistoricAverage vs BottomUp(HistoricAverage): max diff = 0.146244 (expect
~0)
[2] Top-Down root check (ETS): max diff = 0.000000 (expect ~0)
[3] ets fell back on 699 of 30,354 series (2.30%)
[3] theta fell back on 699 of 30,354 series (2.30%)
[4] Top-Down vs base histavg (non-root check): max diff = 292,505.69 (expect >>
0)
```

1.17 15. Optional Visualisation

Plots a handful of series (random by default) with their actual history and every reconciled forecast overlaid. To look at specific nodes instead of random ones, pass `ids=[...]` - remember every `unique_id` now starts with "Total/", e.g. `ids=["Total/CA", "Total/CA/CA_1"]`.

```
[26]: plot_series(
        df=Y_df.query(f"{cat_label_col}.isin(['train', 'test'])").
        drop(columns=[cat_label_col]),
        forecasts_df=rec_fc.drop(columns=[cat_label_col]),
        time_col=date_col, target_col=target_col,
        max_ids=8, plot_random=True, engine="plotly",
        # ids=["Total/CA", "Total/CA/CA_1"], # uncomment to pin specific nodes
    )
```

```
[27]: plot_series(
        df=Y_df.query(f"{cat_label_col}.isin(['train', 'test'])").
        drop(columns=[cat_label_col]),
        forecasts_df=rec_fc.drop(columns=[cat_label_col]),
        time_col=date_col, target_col=target_col,
        max_ids=8, plot_random=True, engine="plotly",
        ids=["Total", "Total/CA/CA_1"], # uncomment to pin specific nodes
    )
```

1.18 16. Export Data for the Dashboard

Four Parquet files feed the Streamlit dashboard (`app/streamlit_app.py`): a lookup table for the dropdown menus, the full actual sales history, and the base/reconciled forecasts for the test and live horizons.

```
[28]: df[[col_state, col_store, col_cat, col_dept, col_item]].drop_duplicates().
        to_parquet(
            cfg.DASHBOARD_DATA_DIR / "hierarchy.parquet", index=False
        )
Y_df[[id_col, date_col, target_col, cat_label_col]].to_parquet(
    cfg.DASHBOARD_DATA_DIR / "actuals.parquet", index=False
)
pred_with_gt.to_parquet(cfg.DASHBOARD_DATA_DIR / "pred_with_gt.parquet",
    index=False)
```

```
rec_with_gt.to_parquet(cfg.DASHBOARD_DATA_DIR / "rec_with_gt.parquet",  
                        index=False)  
  
print(f"\n Dashboard data exported to {cfg.DASHBOARD_DATA_DIR.resolve()}")
```

Dashboard data exported to /Users/hemanth_jp/Documents/GitHub/MBIDA_Master_Thesis/Code/artifacts/dashboard_data

10.2.3. config.py

The single shared configuration module read by both notebooks above and the Streamlit dashboard below, as introduced in Section 7.1.

Listing 10.1: Full code listing: `Code/config.py`

```
"""
Shared configuration for the M5 hierarchical forecasting project.

Imported by BOTH 'notebooks/02_pipeline.ipynb' (which produces the
    ↳ dashboard
parquet files) and 'app/streamlit_app.py' (which reads them).
    ↳ Keeping these
constants in one file means the two can never silently drift apart
    ↳ e.g. the
reconciler column-name suffixes below must match EXACTLY what
'hierarchicalforecast' produces, and previously that string was
    ↳ typed out
separately in the app.

All paths are resolved relative to this file's own location, so both
    ↳ the
notebooks and the app work correctly regardless of the working
    ↳ directory
they're launched from (this matters especially for Streamlit
    ↳ Community
Cloud, which always sets the working directory to the repo root).
"""

from pathlib import Path

#
    ↳ -----
    ↳ -----
# Paths
#
    ↳ -----
    ↳ -----
CODE_DIR = Path(__file__).resolve().parent

DATA_DIR = CODE_DIR / "data" / "m5" # raw
    ↳ Kaggle CSVs (gitignored)
RAW_CACHE_DIR = CODE_DIR / "artifacts" / "raw_cache" # parsed
    ↳ -CSV parquet cache (gitignored)
DASHBOARD_DATA_DIR = CODE_DIR / "artifacts" / "dashboard_data" #
    ↳ committed - what the app reads

for _d in (RAW_CACHE_DIR, DASHBOARD_DATA_DIR):
    _d.mkdir(parents=True, exist_ok=True)

#
    ↳ -----
    ↳ -----
# Column names
#
    ↳ -----
    ↳ -----
COL_STATE = "state_id"
```



```

COL_STORE = "store_id"
COL_CAT = "cat_id"
COL_DEPT = "dept_id"
COL_ITEM = "item_id"
COL_TOTAL = "total_id"

DATE_COL = "date"
TARGET_COL = "sales"
CAT_LABEL_COL = "_cat"           # holds "train" / "test" / "live"
ID_COL = "unique_id"            # hierarchicalforecast's series
    ↪ identifier

HIERARCHY_COLS = [COL_TOTAL, COL_STATE, COL_STORE, COL_CAT, COL_DEPT
    ↪ , COL_ITEM]
ROOT_LABEL = "Total"            # the constant value written into
    ↪ COL_TOTAL
LEVEL_SEP = "/"                # hierarchicalforecast's default
    ↪ unique_id separator

# 'spec' for hierarchicalforecast's aggregate(): each inner list is
    ↪ one level,
# top-down, always including every column of the levels above it.
SPEC = [HIERARCHY_COLS[: i + 1] for i in range(len(HIERARCHY_COLS))]

# Carries the train/test/live label through aggregation. "min" is a
    ↪ safe
# no-op here since the label only depends on date, not on item/store
    ↪ .
EXOG_VARS = {CAT_LABEL_COL: "min"}

#
    ↪ -----
    ↪
# Time windows
#
    ↪ -----
    ↪
TRAIN_TEST_SPLIT_DATE = "2015-01-01" # train ends, test begins
FORECAST_START_DATE = "2016-01-01"   # test ends, live begins
FORECAST_END_DATE = "2016-07-01"     # live ends (no real data)
    ↪ past Apr 2016

#
    ↪ -----
    ↪
# Modeling
#
    ↪ -----
    ↪
SEASON_LENGTH = 12                # monthly data ->
    ↪ yearly seasonality
PERCENTILES = [5, 10, 20, 50, 80, 100] # cumulative share of
    ↪ TRAINING volume

#
    ↪ -----
    ↪
# Reconciliation methods
#

```

```
# Label -> exact column suffix produced by hierarchicalforecast
    ↪ after
# reconcile(). If you change a reconciler's parameters in the
    ↪ pipeline
# notebook, update the matching suffix here too - this is the ONE
    ↪ place
# both sides read from, so there's only one string to keep in sync.
#
    ↪ -----
    ↪
RECONCILERS = {
    "Bottom-Up": "BottomUpSparse",
    "MinTrace (wls_struct)": "MinTraceSparse_method-
    ↪ wls_struct_nonnegative-True",
    "Top-Down": "TopDownSparse_method-proportion_averages",
}

BASE_MODELS = ["histavg", "ets", "theta"]
```

10.2.4. Streamlit Dashboard

The dashboard application described in Chapter 8, reading the four exported Parquet artifacts written by Section 16 of the pipeline notebook above.

Listing 10.2: Full code listing: `Code/app/streamlit_app.py`

```
"""
M5 Hierarchical Forecast Explorer (Streamlit)
=====

    Total -> State -> Store -> Category -> Department -> Item

Cascading dropdowns (st.selectbox is searchable out of the box -
    ↪ just
click and start typing). Leave any level on "(All)" and the app
resolves that to the correct hierarchy node's 'unique_id'
(e.g. "Total", "Total/CA", "Total/CA/CA_1", ...) and plots whatever
aggregate (or bottom-level) series that represents.

The chart shows:
- actuals (train + test, shaded backgrounds; "live" is a forecast-
    ↪ only
window with no real ground truth, so it's excluded from the
    ↪ actual line)
- the base forecast (histavg / ets / theta) you pick
- the reconciled forecast(s) you pick (BottomUp / MinTrace /
    ↪ TopDown)

-----
DATA

This app reads from Code/artifacts/dashboard_data/ - four parquet
    ↪ files
produced by notebooks/02_pipeline.ipynb. It never re-runs the
    ↪ pipeline
itself (Streamlit Community Cloud only ever executes this file).
```

```

Run locally:
  pip install -r requirements.txt
  streamlit run app/streamlit_app.py
-----
"""

import sys
from pathlib import Path

# Make Code/config.py importable regardless of the working directory
#   ↳ this
# app is launched from (matters especially on Streamlit Community
#   ↳ Cloud,
# which always sets the cwd to the repo root, not to app/).
sys.path.append(str(Path(__file__).resolve().parent.parent))
import config # noqa: E402

import pandas as pd
import plotly.graph_objects as go
import streamlit as st
from streamlit_theme import st_theme

ALL_LABEL = "(All)"

st.set_page_config(page_title="M5 Hierarchy Explorer", layout="wide"
#   ↳ )

# Reads Streamlit's actual active theme (light/dark, including "Use
#   ↳ system
# setting") straight from the browser - no manual toggle needed.
_theme = st_theme()
if _theme is not None:
    st.session_state["_chart_base_theme"] = _theme.get("base", "
#   ↳ light")
dark = st.session_state.get("_chart_base_theme", "light") == "dark"

#
#   ↳ =====
#   ↳
# LOAD DATA (cached so switching dropdowns doesn't re-read parquet
#   ↳ each time)
#
#   ↳ =====
#   ↳
@st.cache_data
def load_data():
    hierarchy_df = pd.read_parquet(config.DASHBOARD_DATA_DIR / "
#   ↳ hierarchy.parquet")
    actuals_df = pd.read_parquet(config.DASHBOARD_DATA_DIR / "
#   ↳ actuals.parquet")
    pred_df = pd.read_parquet(config.DASHBOARD_DATA_DIR / "
#   ↳ pred_with_gt.parquet")
    rec_df = pd.read_parquet(config.DASHBOARD_DATA_DIR / "
#   ↳ rec_with_gt.parquet")

    for df_ in (actuals_df, pred_df, rec_df):
        df_[config.DATE_COL] = pd.to_datetime(df_[config.DATE_COL])

```

```

    return hierarchy_df, actuals_df, pred_df, rec_df

try:
    hierarchy_df, actuals_df, pred_df, rec_df = load_data()
except FileNotFoundError as e:
    st.error(
        f"Couldn't find one of the required parquet files in "
        f"'{config.DASHBOARD_DATA_DIR}'.\n\n{e}\n\n"
        "Run notebooks/02_pipeline.ipynb first - its final section
        ↳ writes "
        "hierarchy.parquet, actuals.parquet, pred_with_gt.parquet,
        ↳ and "
        "rec_with_gt.parquet into that folder."
    )
    st.stop()

rec_cols_available = set(rec_df.columns)

def reconciled_col(base_model: str, reconciler_suffix: str) -> str |
    ↳ None:
    candidate = f"{base_model}/{reconciler_suffix}"
    return candidate if candidate in rec_cols_available else None

#
↳ =====
↳
# HELPERS: cascading dropdown option builders + unique_id resolution
#
↳ =====
↳

def options_with_all(values):
    values = sorted(v for v in values if pd.notna(v))
    return [ALL_LABEL] + list(values)

def unique_id_for_selection(state, store, cat, dept, item) -> str:
    """
    Build the hierarchy node's unique_id from the (possibly partial)
    selection. Every id starts with the "Total" root. Stops at the
    first level left on "(All)" - deeper dropdowns are ignored,
    matching how the pipeline's 'SPEC' (config.py) builds aggregate
    ↳ nodes:
        [Total]
        [Total, state]
        [Total, state, store]
        [Total, state, store, cat]
        [Total, state, store, cat, dept]
        [Total, state, store, cat, dept, item]

    Leaving every dropdown on "(All)" resolves to "Total" itself
    (the grand total).
    """
    parts = [config.ROOT_LABEL]
    for val in (state, store, cat, dept, item):
        if val in (None, ALL_LABEL):

```

```

        break
    parts.append(val)
    return config.LEVEL_SEP.join(parts)

#
# SIDE BAR - cascading dropdowns + model / reconciler controls
#
st.title("M5 Hierarchical Forecast Explorer")
st.caption(
    "Drill down Total \u2192 State \u2192 Store \u2192 Category \u2192 Department \u2192 Item. "
    "Leave a level on \u201c(All)\u201d to view that aggregate node instead of a "
    "single item. Click a dropdown and type to search."
)

with st.sidebar:
    st.header("Hierarchy")

    state = st.selectbox("State", options_with_all(hierarchy_df[
        config.COL_STATE].unique()))

    if state == ALL_LABEL:
        store_opts = [ALL_LABEL]
    else:
        subset = hierarchy_df[hierarchy_df[config.COL_STATE] == state]
        store_opts = options_with_all(subset[config.COL_STORE].unique())
    store = st.selectbox("Store", store_opts)

    if ALL_LABEL in (state, store):
        cat_opts = [ALL_LABEL]
    else:
        subset = hierarchy_df[
            (hierarchy_df[config.COL_STATE] == state) & (
                hierarchy_df[config.COL_STORE] == store)
        ]
        cat_opts = options_with_all(subset[config.COL_CAT].unique())
    cat = st.selectbox("Category", cat_opts)

    if ALL_LABEL in (state, store, cat):
        dept_opts = [ALL_LABEL]
    else:
        subset = hierarchy_df[
            (hierarchy_df[config.COL_STATE] == state)
            & (hierarchy_df[config.COL_STORE] == store)
            & (hierarchy_df[config.COL_CAT] == cat)
        ]
        dept_opts = options_with_all(subset[config.COL_DEPT].unique())
    dept = st.selectbox("Department", dept_opts)

    if ALL_LABEL in (state, store, cat, dept):

```

```

        item_opts = [ALL_LABEL]
    else:
        subset = hierarchy_df[
            (hierarchy_df[config.COL_STATE] == state)
            & (hierarchy_df[config.COL_STORE] == store)
            & (hierarchy_df[config.COL_CAT] == cat)
            & (hierarchy_df[config.COL_DEPT] == dept)
        ]
        item_opts = options_with_all(subset[config.COL_ITEM].unique
        ↪ ())
    item = st.selectbox("Item", item_opts)

    st.header("Forecast")
    base_model = st.radio("Base model", config.BASE_MODELS,
    ↪ horizontal=True)
    chosen_reconcilers = st.multiselect(
        "Reconciliation method(s)", list(config.RECONCILERS.keys()),
        ↪ default=["MinTrace (wls_struct)"]
    )

#
    ↪ =====
    ↪
# BUILD THE CHART
#
    ↪ =====
    ↪
uid = unique_id_for_selection(state, store, cat, dept, item)
level_label = f"Selected node: **{uid}** (level = {uid.count(config
    ↪ .LEVEL_SEP) + 1})"

# --- full actual history, but only through "test" -- the "live"
    ↪ window has
# no real ground truth, so plotting it as if it were observed sales
    ↪ is
# misleading. Cut the line there. ---
actual_full = actuals_df[actuals_df[config.ID_COL] == uid].
    ↪ sort_values(config.DATE_COL)
if actual_full.empty:
    st.warning(f"No data found for '{uid}'.")
    st.stop()

actual = actual_full[actual_full[config.CAT_LABEL_COL] != "live"]

# --- colors that stay readable in both chart themes ---
if dark:
    actual_color = "#F2F2F2"
    grid_color = "rgba(255,255,255,0.15)"
    font_color = "#E6E6E6"
    annotation_color = "#CFCFCF"
    train_fill = "rgba(255,255,255,0.06)"
else:
    actual_color = "black"
    grid_color = "rgba(0,0,0,0.12)"
    font_color = "#262730"
    annotation_color = "#666666"
    train_fill = "rgba(0,0,0,0.04)"

```

```

fig = go.Figure()

fig.add_trace(
    go.Scatter(
        x=actual[config.DATE_COL], y=actual[config.TARGET_COL], mode
        ↪="lines+markers",
        name="Actual", line=dict(color=actual_color, width=2),
        ↪ marker=dict(size=6),
    )
)

# --- base forecast (pred_df already only contains the horizon) ---
fc = pred_df[pred_df[config.ID_COL] == uid].sort_values(config.
    ↪ DATE_COL)
fig.add_trace(
    go.Scatter(
        x=fc[config.DATE_COL], y=fc[base_model], mode="lines+markers
        ↪ ",
        name=f"Base: {base_model}", line=dict(dash="dot", width=2),
        ↪ marker=dict(size=6),
    )
)

# --- reconciled forecast(s) (rec_df already only contains the
    ↪ horizon) ---
rec_sub = rec_df[rec_df[config.ID_COL] == uid].sort_values(config.
    ↪ DATE_COL)
missing = []
for label_name in chosen_reconcilers:
    col = reconciled_col(base_model, config.RECONCILERS[label_name])
    if col is None:
        missing.append(label_name)
        continue
    fig.add_trace(
        go.Scatter(
            x=rec_sub[config.DATE_COL], y=rec_sub[col], mode="lines+
            ↪ markers",
            name=f"Reconciled: {label_name}", line=dict(width=2),
            ↪ marker=dict(size=6),
        )
    )

# --- shade + label train / test / live regions ---
phase_style = {
    "train": dict(fill=train_fill, text="Train"),
    "test": dict(fill="rgba(255,165,0,0.12)", text="Test"),
    "live": dict(fill="rgba(0,178,89,0.12)", text="Live"),
}
for phase, style in phase_style.items():
    phase_dates = actual_full.loc[actual_full[config.CAT_LABEL_COL]
        ↪ == phase, config.DATE_COL]
    if phase_dates.empty:
        continue
    fig.add_vrect(
        x0=phase_dates.min(), x1=phase_dates.max(),
        fillcolor=style["fill"], line_width=0,
        annotation_text=style["text"], annotation_position="top left
        ↪ ",
        annotation_font_size=10, annotation_font_color=

```

```

        ↪ annotation_color,
    )

fig.update_layout(
    xaxis_title="Date",
    yaxis_title="Sales",
    legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="
        ↪ right", x=1),
    margin=dict(t=60, l=40, r=20, b=40),
    height=560,
    paper_bgcolor="rgba(0,0,0,0)",
    plot_bgcolor="rgba(0,0,0,0)",
    font=dict(color=font_color),
    xaxis=dict(gridcolor=grid_color, zerolinecolor=grid_color),
    yaxis=dict(gridcolor=grid_color, zerolinecolor=grid_color),
)

if missing:
    level_label += (
        f" | No column found for: {', '.join(missing)} "
        "(check config.RECONCILERS suffixes vs. rec_with_gt.columns)
        ↪ "
    )

st.markdown(level_label)
st.plotly_chart(fig, width="stretch")

```

10.2.5. Pipeline Requirements

The full pinned dependency set used to execute both notebooks locally in the `m5_forecast` environment.

Listing 10.3: Full listing: `Code/requirements-pipeline-raw.txt`

```

adagio==0.2.6
altair==6.2.2
annotated-types==0.8.0
anyio==4.14.2
appnope==0.1.4
asttokens==3.0.2
attrs==26.1.0
blinker==1.9.0
certifi==2026.7.22
cffi==2.1.0
charset-normalizer==3.4.9
clarabel==0.11.1
click==8.4.2
cloudpickle==3.1.2
comm==0.2.3
contourpy==1.3.2
coreforecast==0.0.18

```



```
cycler==0.12.1
dash==4.4.1
debugpy==1.8.21
decorator==5.3.1
exceptiongroup==1.3.1
executing==2.2.1
fastjsonschema==2.22.1
Flask==3.1.3
fonttools==4.63.0
fsspec==2026.7.0
fugue==0.9.7
h11==0.16.0
hierarchicalforecast==1.5.1
httptools==0.8.0
idna==3.18
importlib_metadata==9.0.0
ipykernel==7.3.0
ipython==8.37.0
itsdangerous==2.2.0
janus==2.0.0
jedi==0.20.0
Jinja2==3.1.6
joblib==1.5.3
jsonschema==4.26.0
jsonschema-specifications==2025.9.1
jupyter_client==8.9.1
jupyter_core==5.9.1
kiwisolver==1.5.0
MarkupSafe==3.0.3
matplotlib==3.10.9
matplotlib-inline==0.2.2
narwhals==2.24.0
nbformat==5.10.4
nest-asyncio==1.6.0
nest-asyncio2==1.7.2
numpy==2.2.6
packaging==26.0
pandas==2.3.3
parso==0.8.7
patsy==1.0.2
pexpect==4.9.0
pickleshare==0.7.5
pillow==12.3.0
pip==26.1.2
```

```
platformdirs==4.11.0
plotly==6.9.0
prompt_toolkit==3.0.53
protobuf==7.35.1
psutil==7.2.2
ptyprocess==0.7.0
pure_eval==0.2.3
pyarrow==24.0.0
pyparser==3.0
pydantic==2.13.4
pydantic_core==2.46.4
pydeck==0.9.3
Pygments==2.20.0
pyparsing==3.3.2
python-dateutil==2.9.0.post0
python-multipart==0.0.32
pytz==2026.3.post1
pyzmq==27.1.0
qpsolvers==4.13.0
referencing==0.37.0
requests==2.34.2
retrying==1.4.2
rpds-py==0.30.0
scikit-learn==1.7.2
scipy==1.15.3
setuptools==83.0.0
six==1.17.0
st-theme==1.2.3
stack_data==0.6.3
starlette==1.3.1
statsforecast==2.1.1
statsmodels==0.14.6
streamlit==1.61.1
tenacity==9.1.4
threadpoolctl==3.6.0
toml==0.10.2
tornado==6.5.7
tqdm==4.70.0
traitlets==5.15.1
triad==1.0.2
typing_extensions==4.16.0
typing-inspection==0.4.2
tzdata==2026.3
urllib3==2.7.0
```

```
utilsforecast==0.2.16
uvicorn==0.52.1
wcwidth==0.8.2
websockets==16.1.1
Werkzeug==3.1.8
wheel==0.47.0
zipp==4.1.0
```

10.2.6. Deployment Requirements

The separate, smaller requirements file that Streamlit Community Cloud actually installs when building the dashboard (Chapter 8) – deliberately kept independent of the pipeline requirements above so that the deployed app is never held back by, or accidentally dependent on, packages only the notebooks need.

Listing 10.4: Full listing: `Code/app/requirements.txt`

```
# Requirements for the deployed Streamlit dashboard ONLY.
# Streamlit Community Cloud installs from this file (Cloud looks for
# requirements file at the repo root OR next to the entrypoint – this
# lives next to streamlit_app.py). Versions pinned to what's proven t
# in the m5_forecast conda env.
#
# NOTE: the installed package is 'st-theme' (imported as 'streamlit_t
# not 'streamlit-theme' – confirmed against the working environment's

streamlit==1.61.1
st-theme==1.2.3
plotly==6.9.0
pandas==2.3.3
pyarrow==24.0.0
```

Bibliography

- [AN00] V. Assimakopoulos and K. Nikolopoulos. “The theta model: a decomposition approach to forecasting”. In: *International Journal of Forecasting* 16.4 (2000), pp. 521–530. DOI: [10.1016/S0169-2070\(00\)00066-2](https://doi.org/10.1016/S0169-2070(00)00066-2). URL: [https://doi.org/10.1016/S0169-2070\(00\)00066-2](https://doi.org/10.1016/S0169-2070(00)00066-2).
- [Apa26] Apache Software Foundation. *Apache Arrow: A Cross-Language Development Platform for In-Memory Data*. Accessed: 18 September 2026. 2026. URL: <https://arrow.apache.org/docs/python/>.
- [Ath+17] G. Athanasopoulos et al. “Forecasting with Temporal Hierarchies”. In: *European Journal of Operational Research* 262.1 (2017), pp. 60–74.
- [CM15] S. Chopra and P. Meindl. *Supply Chain Management: Strategy, Planning, and Operation*. 6th, Global. Harlow, UK: Pearson Education Limited, 2015. ISBN: 9781292093574.
- [Cro72] J. D. Croston. “Forecasting and stock control for intermittent demands”. In: *Operational Research Quarterly* 23.3 (1972), pp. 289–303. DOI: [10.2307/3007885](https://doi.org/10.2307/3007885).
- [DWD76] D. M. Dunn, W. H. Williams, and T. L. Dechaine. “Aggregate versus subaggregate models in local area forecasting”. In: *Journal of the American Statistical Association* 71.353 (1976), pp. 68–71.
- [FPSS96] U. Fayyad, G. Piatetsky-Shapiro, and P. Smyth. “From data mining to knowledge discovery in databases”. In: *AI Magazine* 17.3 (1996), pp. 37–54. DOI: [10.1609/aimag.v17i3.1230](https://doi.org/10.1609/aimag.v17i3.1230). URL: <https://ojs.aaai.org/aimagazine/index.php/aimagazine/article/view/1230>.
- [GS90] C. W. Gross and J. E. Sohl. “Disaggregation methods to expedite product line forecasting”. In: *Journal of Forecasting* 9.3 (1990), pp. 233–254.
- [Gar+22] A. Garza et al. *StatsForecast: Lightning fast forecasting with statistical and econometric models*. PyCon Salt Lake City, Utah, US 2022. 2022. URL: <https://github.com/Nixtla/statsforecast>.

-
- [HA18] R. J. Hyndman and G. Athanasopoulos. *Forecasting: Principles and Practice*. 2nd. Melbourne, Australia: OTexts, 2018. URL: <https://otexts.com/fpp2/>.
 - [HK06] R. J. Hyndman and A. B. Koehler. “Another look at measures of forecast accuracy”. In: *International Journal of Forecasting* 22.4 (2006), pp. 679–688. DOI: [10.1016/j.ijforecast.2006.03.001](https://doi.org/10.1016/j.ijforecast.2006.03.001).
 - [HT99] A. Hunt and D. Thomas. *The Pragmatic Programmer: From Journeyman to Master*. Boston, MA: Addison-Wesley, 1999. ISBN: 0201616224.
 - [Har+20] C. R. Harris et al. “Array programming with NumPy”. In: *Nature* 585.7825 (2020), pp. 357–362. DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2).
 - [Hew+21] H. Hewamalage et al. “A look at the evaluation setup of the M5 forecasting competition”. In: *arXiv preprint arXiv:2108.03588* (2021). URL: <https://arxiv.org/abs/2108.03588>.
 - [Hun07] J. D. Hunter. “Matplotlib: A 2D graphics environment”. In: *Computing in Science & Engineering* 9.3 (2007). Accessed: 18 September 2026, pp. 90–95. DOI: [10.1109/MCSE.2007.55](https://doi.org/10.1109/MCSE.2007.55). URL: <https://matplotlib.org/stable/>.
 - [Hyn+08] R. J. Hyndman et al. *Forecasting with Exponential Smoothing: The State Space Approach*. Springer Series in Statistics. Berlin, Germany: Springer, 2008. DOI: [10.1007/978-3-540-71918-2](https://doi.org/10.1007/978-3-540-71918-2). URL: <https://doi.org/10.1007/978-3-540-71918-2>.
 - [Hyn+11] R. J. Hyndman et al. “Optimal combination forecasts for hierarchical time series”. In: *Computational Statistics & Data Analysis* 55.9 (2011), pp. 2579–2589. DOI: [10.1016/j.csda.2011.03.006](https://doi.org/10.1016/j.csda.2011.03.006).
 - [MH00] S. Makridakis and M. Hibon. “The M3-Competition: Results, Conclusions and Implications”. In: *International Journal of Forecasting* 16.4 (2000), pp. 451–476. DOI: [10.1016/S0169-2070\(00\)00057-1](https://doi.org/10.1016/S0169-2070(00)00057-1).
 - [MSA22] S. Makridakis, E. Spiliotis, and V. Assimakopoulos. “M5 accuracy competition: Results, findings, and conclusions”. In: *International Journal of Forecasting* 38.4 (2022), pp. 1346–1364. DOI: [10.1016/j.ijforecast.2021.11.013](https://doi.org/10.1016/j.ijforecast.2021.11.013). URL: <https://doi.org/10.1016/j.ijforecast.2021.11.013>.

- [McK10] W. McKinney. “Data Structures for Statistical Computing in Python”. In: *Proceedings of the 9th Python in Science Conference*. 2010, pp. 56–61. DOI: [10.25080/Majora-92bf1922-00a](https://doi.org/10.25080/Majora-92bf1922-00a).
- [Mon04] D. C. Montgomery. *Design and Analysis of Experiments*. 6th. New York: John Wiley & Sons, 2004. ISBN: 9780471487357.
- [Nix26] Nixtla. *utilsforecast: Forecasting Utilities*. Accessed: 18 September 2026. 2026. URL: <https://github.com/Nixtla/utilsforecast>.
- [OWE68] G. H. Orcutt, H. W. Watts, and J. B. Edwards. “Data aggregation and information loss”. In: *The American Economic Review* 58.4 (1968), pp. 773–787.
- [Oli+22] K. G. Olivares et al. *HierarchicalForecast: A Reference Framework for Hierarchical Forecasting in Python*. Work in progress paper, submitted to the Journal of Machine Learning Research. 2022. arXiv: [2207.03517](https://arxiv.org/abs/2207.03517) [stat.ML]. URL: <https://arxiv.org/abs/2207.03517>.
- [Par96] V. Pareto. *Cours d’économie politique*. Vol. 1. Lausanne: F. Rouge, 1896. URL: <https://archive.org/details/fp-0148-1>.
- [Plø26] Plotly Technologies Inc. *Plotly Python Open Source Graphing Library*. Accessed: 18 September 2026. 2026. URL: <https://plotly.com/python/>.
- [SBC05] A. A. Syntetos, J. E. Boylan, and J. D. Croston. “On the categorization of demand patterns”. In: *Journal of the Operational Research Society* 56.5 (2005), pp. 495–503. DOI: [10.1057/palgrave.jors.2601841](https://doi.org/10.1057/palgrave.jors.2601841). URL: <https://doi.org/10.1057/palgrave.jors.2601841>.
- [SPP98] E. A. Silver, D. F. Pyke, and R. Peterson. *Inventory Management and Production Planning and Scheduling*. 3rd. New York: John Wiley & Sons, 1998. ISBN: 0471119474.
- [San+13] G. K. Sandve et al. “Ten Simple Rules for Reproducible Computational Research”. In: *PLoS Computational Biology* 9.10 (2013), e1003285. DOI: [10.1371/journal.pcbi.1003285](https://doi.org/10.1371/journal.pcbi.1003285).
- [Str26] Streamlit Inc. *Streamlit Documentation*. Accessed: 18 September 2026. 2026. URL: <https://docs.streamlit.io>.

- [WAH19] S. L. Wickramasuriya, G. Athanasopoulos, and R. J. Hyndman. “Optimal forecast reconciliation for hierarchical and grouped time series through trace minimization”. In: *Journal of the American Statistical Association* 114.526 (2019), pp. 804–819. DOI: **10.1080/01621459.2018.1448825**. URL: <https://doi.org/10.1080/01621459.2018.1448825>.
- [WTH20] S. L. Wickramasuriya, B. A. Turlach, and R. J. Hyndman. “Optimal non-negative forecast reconciliation”. In: *Statistics and Computing* 30.5 (2020), pp. 1167–1182. DOI: **10.1007/s11222-020-09930-0**.
- [tem26] tempass, Gabriel. *st-theme: A Streamlit Component that Returns the Active Theme of a Streamlit App*. Accessed: 19 September 2026. 2026. URL: <https://pypi.org/project/st-theme/>.